

AppSphere

שם בית הספר: תיכון אלון

שם העבודה: AppSphere

שם התלמידה: תמר הופמן

ת.ז. תלמידה: 216728576

שם המנחים: עידן פלדברג, רום רפסון, עודי רז וקובי שוצמן

שם החלופה: הגנת סייבר

תאריך ההגשה: 24.5.2025



תוכן עניינים:

5	מבוא
5	ייזום
5	תיאור ראשוני של המערכת:
6	הגדרת הלקוח:
6	הגדרת יעדים/מטרות:
6	בעיות, תועלות וחסכונות:
6	סקירת פתרונות קיימים:
6	סקירת טכנולוגית הפרויקט:
7	תיחום פרויקט:
7	פירוט תיאור המערכת (אפיון)
7	פירוט מפורט יותר של המערכת:
8	פירוט היכולת שהמערכת נותנת לכל סוג משתמש:
8	פירוט הבדיקות (קופסה שחורה) שמתוכננות לפרויקט:
9	תכנון וניהול לוח זמנים לפיתוח המערכת:
9	ניהול סיכונים ודרכי התמודדות:
10	תיאור תחום הידע
10	פירוט מעמיק של היכולות שהוצגו בשלב הקודם ומעבר
10	פירוט היכולות בצד השרת:
17	פירוט היכולות בצד לקוח:
22	מבנה/ ארכיטקטורה של הפרויקט
22	תיאור הארכיטקטורה של המערכת המוצעת
22	תיאור החומרה – רכיבים שונים והקשרים ביניהם:
23	שרטוט המציג את הקשרים בין רכיבי החומרה:
23	תיאור טכנולוגיה רלוונטית
24	תיאור זרימת המידע במערכת
28	תיאור האלגוריתמים מרכזיים בפרויקט
31	תיאור סביבת הפיתוח
31	פירוט כלי הפיתוח הנדרשים לפיתוח:
31	פרוט הסביבה והכלים הנדרשים לבדיקות:
32	תיאור פרוטוקול התקשורת
32	תיאור מילולי של פרוטוקול התקשורת:
32	פירוט כלל ההודעות הזורמות במערכת:

38	תיאור מסכי המערכת
38	תיאור כל מסך:
47	תרשים מסכים:
48	תיאור מבני הנתונים
48	פירוט מבני הנתונים:
48	פירוט מאגרי המידע של המערכת:
52	סקירת חולשות והאיומים
55	מימוש הפרויקט
55	סקירת מודולים/ מחלקות מיובאים
56	מודולים/מחלקות שפיתחתי
80	קטעי קוד האלגוריתמים המרכזיים
89	מסמך בדיקות מלא
92	מדריך למשתמש
92	עץ קבצי המערכת:
94	התקנת המערכת
95	מדריך להפעלת השרת:
95	מדריך למשתמש AppSphere:
106	רפלקציה/ סיכום אישי
108	ביבליוגרפיה
109	נספחים – קוד הפרויקט
109	מצד השרת:
109	:server
119	:RSA
121	:My_database2
124	מצד הלקוח:
124	:Main
125	:Client2
130	:RSA
132	:OpeningFrame
134	:CreatorFrame
136	:InputDatabaseFrame
143	:Tools
147	:Homepage
150	:Catalog

151.....:Meeting_System

154.....:User

מבוא

ייזום

תיאור ראשוני של המערכת:

הפרויקט שבחרתי לעשות הוא מעין ספרייה של אפליקציות שאליה כל אחד יכול להוסיף אפליקציה משלו ולהתאים אותה לשימוש. כל אפליקציה הנוצרת דרך הפרויקט שלי תכנס ללא יוצא מן הכלל לספריית האפליקציות אליה כל אדם שיש לו גישה לפרויקט יוכל להיכנס ולהשתמש.

מראה האפליקציה שתיווצר יהיה נתון להחלטות היוצר, כאשר הפרויקט שלי מציע מגוון דרכים לעצב ולהתאים אותה אישית, כמו בחירת ערכת הנושא, בחירת התבנית עליה האפליקציה תיבנה ובחירת מרכיבי האפליקציה. מרכיבי האפליקציה האפשריים הם:

- עמוד בית – מכיל תמונות, תכנים ופרטים שבעל האפליקציה רוצה להציג.
- קטלוג – אם בעל האפליקציה מעוניין להציג מוצרים למכירה, הוא יכול להיעזר בקטלוג, שם לכל מוצר תופיע תמונה, תיאור ומחיר (דרך האפליקציה לא יהיה ניתן להעביר כסף או להזמין מוצרים אלא רק להסתכל).
- עמוד לקביעת ותיאום זמנים לפי רצונותיו של בעל האפליקציה – עמוד שיכיל טבלה שבועית של תאריכים ושעות ומותאמת לצרכי בעל האפליקציה. לאחר שהתבצעה הקביעה עם המשתמש, המערכת בצורה אוטומטית תעדכן את הטבלה.

כדי להיכנס לאתר אין צורך להתחבר עם שם משתמש וסיסמה, אך כדי לקבוע פגישה, המשתמש חייב להתחבר עם שם משתמש וסיסמה כדי להיכנס למערכת הפגישות.

למה בחרתי בפרויקט?

הרעיון המקורי של הפרויקט הגיע אלי דרך חיי היום יום שראיתי שיש ביקוש למשהו כזה. אני השתתפתי בסדנת תפירה בה המורה העלתה את הצורך ליצור לעצמה אפליקציה לקביעת הסדנאות שלה ומכירת הציוד שיש לה. דרך הצורך הזה אני חשבת על הרעיון שלי לפרויקט שיהיה פלטפורמה נגישה ליצירת אתרים שדומים לאלו שהיא צריכה.

לדעתי האתגרים המרכזיים שיהיו לי בפרויקט יהיו בתחומי האבטחה והרשתות, עקב השימוש הקריטי והגודל שלהם בפרויקט: מקום כל הפרטים של ספריית האפליקציות נמצא בשרת, לכן השימוש המרובה בתקשורת עם הדאגה שהיא מאובטחת ומוצפנת, נראה לי מאתגר.

הגדרת הלקוח:

למערכת שלי יש שני סוגים שונים של לקוחות:

- היוצר - האדם שיוצר דרך הפרויקט שלי אפליקציה משלו.
התבנית שלי לאפליקציה היא מאוד כללית ולכן היא יכולה להיות יעילה למגוון רחב של אנשים, לדוגמה: הצגת מוצרי תפירה וקביעת סדנאות, קביעת שיעורי נהיגה או טסטים, קביעת זמנים לחדרי בריחה, הזמנת מקום במסעדה, ועוד...
- המשתמש - כל אדם עם גישה לתוכנה יכול להיכנס לאפליקציות מספריית האפליקציות ולהשתמש בהן.
המשתמש יכול לעיין בקטלוג ולקבוע פגישה בצורה קלה.

הגדרת יעדים/מטרות:

מטרת הפרויקט שלי היא שבזמן יצירת האפליקציות, יהיו כמה שיותר דרכים לעצב אותם אישית וליצור אפליקציות מיוחדות, בזמן שהן הולכות על פי תבנית בנויה. בנוסף אני רוצה שהאפליקציות שיווצרו יהיו נוחות ואסתטיות לשימוש המשתמשים, בזמן שהן גם פועלות ברשת עם אבטחה מלאה.

בעיות, תועלות וחסכונות:

הפרויקט שלי יהיה לתועלת רבה לאנשים שמעוניינים באפליקציה בעלת המבנה שאני מציעה. הפרויקט שלי יכול לחסוך לאותם היוצרים שעות רבות של עבודה כדי ליצור אפליקציה כזו או לחסוך כסף לאותם אנשים שאחרת היו משתמשים בבעל מקצוע בשביל ליצור אפליקציה עם מבנה יחסית פשוט.

סקירת פתרונות קיימים:

חקרתי באינטרנט וגיליתי שיש אתרים ואפליקציות שמטרתן מאוד דומה לפרויקט שלי בנושא יצירת האפליקציות המותאמות אישית. לדוגמה האפליקציה AppMachine או Jotform Web App Builder. הפתרונות הקיימים השונים שמצאתי נראים מאוד טובים והם נותנים מגוון רחב של אפשרויות לאפליקציה שתיצור. לעומתם, הפרויקט שלי הוא יותר ממוקד לצורת אתר ספציפי יותר, וכך יהיה למשתמש פשוט יותר לפתח את האפליקציה שלו. אומנם הפלטפורמה שלי היא מוגבלת יותר, אבל היא יכולה לשמש למגוון מאוד רחב של דברים המצוינים למעלה.

סקירת טכנולוגית הפרויקט:

הפרויקט משתמש בטכנולוגיות קיימות, אך מציע גישה חדשנית ליצירת אפליקציות מותאמות אישית בצורה פשוטה ונגישה. הוא אינו מבוסס על טכנולוגיה חדשה, אבל הייחודיות שלו נמצאת בתבנית העבודה הממוקדת, שמקלה על המשתמשים לעצב ולבנות את האפליקציות שלהם במהירות וללא צורך בידע מתקדם בתכנות.

תיחום פרויקט:

- Python - שפת התכנות בה ייכתב רוב הפרויקט.
- Pycharm - סביבת תכנות לפיתוח.
- Sqlite3 - סוג sql ליצירת database.
- Db browser for sqlite - סביבת הרצה של sql.
- Tkinter - ממשק של פיתוח ליצירת גרפיקה.
- HASHING/RSA - מערכות ההצפנה.
- Client server networking - דרך השימוש ברשתות.
- Threading - שימוש במערכות הפעלה.
- Windows - מערכת ההפעלה שהפרויקט עובד עליה.

תחומים בהם המערכת לא מטפלת:

- המערכת רצה רק על PC/מחשב אישי/מחשב נייד, ואינה פועלת כאפליקציית מובייל.

פירוט תיאור המערכת (אפיון)

פירוט מפורט יותר של המערכת:

כאשר משתמש נכנס אל תוך הפרויקט שלי, מוצגות בפניו שתי אופציות:

- **יצירת אפליקציה חדשה** - שם המשתמש מתחיל מלבחור את ההגדרות של האפליקציה שלו, כמו אילו חלקים הוא רוצה שהאפליקציה תכיל (עמוד בית/מערכת פגישות/קטלוג), צבע ערכת הנושא שלו והעיצוב שהוא בוחר לכל אחד מהחלקים. לאחר שהיוצר סיים עם ההגדרות, הוא נדרש להכניס את כל הערכים והנתונים שהוא מעוניין שיהיו באפליקציה שלו, כמו לדוגמה תמונות ותכני עמוד הבית, מוצרי הקטלוג או שעות העבודה שלו עבור מערכת הפגישות. המערכת תבדוק את כל התכנים שהוא הזין ובמקרה של שגיאה היא תעיר על כך. אם התכנים תקינים, אז תפתח לו האפליקציה והאפליקציה שלו תכנס לספריית האפליקציות לשימוש של כולם.
- **כניסה לספריית האפליקציות** - יוצג מול המשתמש רשימה של שמות האפליקציות הקיימות בפרויקט שלי, והמשתמש יכול להיכנס לאיזו אפליקציה שירצה ולהנות מהתכנים שלה. בשביל שיטוט כללי באתרים של אחרים, אין למשתמש צורך להתחבר לכל אתר, אך אם ברצונו לקבוע פגישה אז הוא נדרש להתחבר. ההתחברות הראשונית נעשית בצורה של אימות דו שלבי בעזרת האימייל, וכל פעולה שהמשתמש יעשה באפליקציה (קביעה/ביטול פגישה) תדווח לו לאימייל גם כן.

פירוט היכולת שהמערכת נותנת לכל סוג משתמש:

היוצר - לעצב אתר משלו עם תכנים כרצונו (צבע נושא, תמונות, בחירת בעיצוב שירצה, החלטה על התכנים...).
המשתמש - המשתמש יכול לגלול ולהסתכל על כל האפליקציות מספריית האפליקציות, ולהשתמש בפונקציות הנמצאות בהן. זאת אומרת שהמשתמש יכול להסתכל על הקטלוג כפי שהוא, והוא יכול להתחבר למערכת ולקבוע לעצמו פגישה על בסיס מקומות פנויים שמוצגים באפליקציה.

פירוט הבדיקות (קופסה שחורה) שמתוכננות לפרויקט:

- האם המידע עובר נכון משני צדדי התקשורת?
 כדי לגלות אני אדפיס את המידע בכל אחד מהצדדים ואוודא שהמידע זהה בשני הצדדים.
- תקינות מנגנון ההצפנה והפיענוח:
 אדפיס את המידע לפני ההצפנה ואחרי ההצפנה ואוודא שהוא זהה.
- תפקוד בסיס הנתונים – האם הוא מתעדכן לפי פעולות המערכת והאם המערכת פועלת בעקבות המידע שנמצא בבסיס הנתונים?
 איעזר ב - db browser כדי לראות שהערכים נכנסים בצורה תקינה, ואת בדיקת הוצאת הערכים אבצע באמצעות הדפסות והשוואת תוצאת ההדפסה למראה ב - db browser.
- בדיקת הצפנה ה - Hash :
 כשמתחברים לפרויקט, הסיסמאות נשלחות לבסיס הנתונים בצורה של hash. בעזרת הדפסת הסיסמאות לפני שהן נשלחות ובדיקת תוכן הסיסמה בתוך הטבלאות של בסיס הנתונים בעזרת db browser, אוכל לראות שפונקציית ה - hash עובדת.
- האם האימיילים אכן נשלחים ועם תוכן תקין?
 אבדוק על ידי שאשלח את המיילים למייל שלי ואבחן את התוצאות.
- ווידוא יצירת אפליקציה תקינה:
 אריץ את התוכנה ואראה שהאפליקציה שתיווצר זהה לזו שהיוצר התכוון ליצור.
- ווידוא העלאה של אפליקציות מספריית האפליקציות בצורה תקינה:
 אצור כמה אפליקציות שונות ואנסה להיכנס אליהן, בתור יוצרת האפליקציות אראה שהן זהות למבוקש. בנוסף, אעזר במידע השמור בבסיס הנתונים בעזרת הסתכלות עליו מ - db browser.
- בדיקת מקרי קצה:
 תוך כדי יצירת הקוד והרצתו אנסה למצוא את מקרי הקצה בעזרת פעולות מסוימות או פלטים מסוימים ואבדוק שתוצאת המערכת זהה למצופה.
- בדיקת הגנה מ - SQL injection :
 אזין מחרוזות עם התנאים המתאימים לSQL injection ואבדוק שהמערכת מגנה מזה.
- בדיקת הגנה מדידוס:
 אקטין את כמות הלקוחות המקסימלית ואפתח מספר לקוחות הגדול באחד מהמספר המקסימלי. אראה

שהשרת יודע לקבל רק עד הכמות המקסימלית ושם מתפנה (אחרי סגירת אחד מהלקוחות), אז השרת יידע לקבל לקוח חדש.

- בדיקת שרת מרובה משתמשים:
אפתח את השרת במחשב אחד ובנוסף אחבר כמה משתמשים במחשבים אחרים, ואראה שהשרת מתפקד בצורה תקינה.
- בדיקת טיפול הלקוח כאשר השרת הפסיק לפעול:
אפעיל את השרת ואחבר את הלקוח אליו, אפסיק את הרצת השרת ואראה שצד הלקוח מתמודד עם זה.

תכנון וניהול לוח זמנים לפיתוח המערכת:

1. יצירת ממשק גרפי אותו היוצר יראה, בו הוא יוכל להתאים אישית את מראה האפליקציה שלו.
2. יצירת ממשק גרפי של הטבלאות אליהן היוצר יכניס את נתוניו, שלאחר מכן יגיעו לבסיס הנתונים.
3. יצירת בסיס הנתונים וחיבורו לממשק הגרפי.
4. יצירת עיצוב אחד לאפליקציה (הכולל עמוד בית, קטלוג ודף קביעת פגישות) וחיבורו להמשך תהליך יצירת האפליקציה.
5. יצירת הממשק הגרפי של התחברות למערכת וקביעת פגישות, ולחבר את נתוני ההתחברות ל - database (בשלב זה ללא אבטחה).
6. יצירת ממשק גרפי של עמוד פתיח המפנה ליצירת אפליקציה או ספריית האפליקציות.
7. יצירת תקשורת (עדיין ללא אבטחה).
8. אבטוח האתר.
9. מימוש שליחת אימיילים לפי פעולות המשתמש באתר ויצירת התחברות בעלת אימות דו שלבי.
10. יצירת עוד שני עיצובים.
11. הוספת גיף טעינה, פיצ'רים נוספים ותיקונים, שיפורים ותיקון בעיות אחרונות.

ניהול סיכונים ודרכי התמודדות:

- המערכת מתמודדת עם סוג ההתקפה של man in the middle בכך שכל התקשורת מוצפנת בעזרת הצפנה א-סימטרית RSA, שמצפינה את תוכן ההודעה ומונעת פריצת אבטחה למידע שנשלח.
- מניעת דרך לקחת מידע אישי ורגיש של משתמשים הנמצא בבסיס הנתונים בעזרת שימוש בפונקציית hash.
- מניעת SQL injection בעזרת הגנה על בסיס הנתונים ועל התכנים שנכנסים אליו.
- מניעת תקיפת דידוס בעזרת הגדרת כמות לקוחות מקסימלית.

תיאור תחום הידע

פירוט מעמיק של היכולות שהוצגו בשלב הקודם ומעבר

פירוט היכולות בצד השרת:

תפקידו העיקרי של השרת הוא לתפקד מול בסיס הנתונים, בכל דרך שהיא.

○ יצירת תקשורת בסיסית עם הלקוחות.

- **מהות:** חיבור השרת לכמות רבה של לקוחות ויצירת תקשורת בסיסית (ללא הצפנה או הודעות מיוחדות).
- **אוסף היכולות הנדרשות:**
 - פתיחות socket.
 - הקשבה ללקוחות.
 - קבלת ושליחת הודעות.
 - טיפול בשגיאות ומקרי קצה.
 - וידוא שכמות הלקוחות לא עוברת את המקסימום על מנת להגן מהתקפת דידוס, ושליחה מיידית של תשובה על כך ללקוח.
- **אובייקטים נחוצים:**
 - המחלקות המיובאות socket, select.
 - הרשימות: rlist, wlist, xlist שומרות את הלקוחות.
 - הרשימה client_sockets שמכילה בתוכה את כל הלקוחות המחוברים לשרת.
 - הרשימה message_to_send המכילה בתוכה את כל ההודעות שהשרת צריך לשלוח ללקוח כאשר כל איבר ברשימה מורכב מטאפל המכיל את הלקוח וההודעה.

○ החלפת מפתחות והצפנה א סימטרית RSA.

- **מהות:** על מנת שהתקשורת תהיה מאובטחת, נצפין אותה בעזרת הצפנת RSA. בהצפנה זו כל אחד ממרכיבי התקשורת צריכים לשמור שני מפתחות (פרטי וציבורי), ולהחליף את המפתחות הציבוריים שלהם.
- **אוסף היכולות הנדרשות:**
 - קליטת ההודעה של הלקוח.
 - לא לפענח אותה כמו את שאר ההודעות (כי ללקוח אין עדיין את המפתח הציבורי של השרת אז הוא לא יכל להצפין את ההודעה).

- יצירת זוג מפתחות (ציבורי ופרטי) בצורה רנדומלית – נוצר לכל לקוח חדש.
- להצפין את המפתח הציבורי של השרת ולשלוח אותו ללקוח.

• אובייקטים נחוצים:

- ספריית random על מנת ליצור את המפתחות.
- שימוש בקובץ RSA שבו יש פונקציה שיוצרת מפתחות.
- גישה למסמך prime2 שמכיל את כל המספרים הראשוניים של המיליון השני.
- הצפנת RSA.
- המילון client_and_keys המכיל כמפתח את הלקוח, וכערך שמורה רשימה המכילה באיבר הראשון את המפתח הציבורי של הלקוח ובאיבר השני את המפתח הפרטי של השרת.
- ספריית Json על מנת לשלוח טאפל של המפתח הציבורי בפורמט של מחרוזת.
- שימוש בספרייה ast כדי להפוך את קלט של הלקוח לצורה של טאפל.

○ טיפול בבקשות הלקוח הדורשות מידע מבסיס הנתונים.

- **מהות:** כל הפרטים הקשורים לאפליקציות נשמרים בבסיס הנתונים שבשרת. בהתאם לכך, השרת צריך לטפל בכל הבדיקות או בקשות הלקוח לפרטים.
- *פה מדובר על כל סוג העברה שהיא, כאשר אם יש לה טיפול מיוחד אז היא תתבצע בנוסף לכך, בהתאם לפרוט בפעולות אחרות שיופיעו בהמשך (כמו חלוקה לפקטות/טיפול בתמונות).
- **פה לא מדובר על הכנסת נתונים לבסיס הנתונים אלא רק הוצאתם.
- שימושים:
 - החזרת רשימת שמות האפליקציות.
 - בדיקה האם שם האפליקציה שהתקבל מהלקוח הוא ייחודי והחזרה תשובה בהתאם.
 - החזרת תוכן עמוד הבית והקטלוג (קשור ליכולות העברת הודעות גדולות ותמונות) לפי שם האפליקציה שהתקבל מהלקוח.
 - התחברות – כאשר הלקוח שולח את האימייל והסיסמה ואת שם האפליקציה והשרת בודק ומחזיר תשובה בהתאם.
 - בדיקה האם האימייל שהתקבל מהלקוח נמצא במאגר הנתונים של האפליקציה, ששמה התקבל מהלקוח והחזרת הודעה מתאימה.
 - בדיקת זמינות של פגישה מסוימת, שזמנה ושם האפליקציה ממנה היא באה התקבלו מהלקוח והחזרת הודעה מתאימה.
 - ניקוי כל הפרטים של האפליקציה ששמה התקבל מהלקוח.
 - החזרת הערכים של מערכת הפגישות של אפליקציה מסוימת, כאשר שם האפליקציה התקבל מהשרת.

- ספירת השורות בטבלת בסיס הנתונים, של טבלה מסוימת באפליקציה מסוימת, שהתקבלה מהשרת, והחזרת כמות השורות.
- החזרת שם העיצוב של אפליקציה מסוימת שהתקבלה מהשרת.
- **אוסף היכולות הנדרשות:**
 - קליטת הודעת המשתמש.
 - פיענוח ההודעה.
 - חלוקת ההודעה לצורה הסטנדרטית בעזרת פיצול ההודעה לפי התו @ כך ש: האיבר הראשון הוא הפקודה, האיבר השני הוא שם האפליקציה והאיברים הבאים הם פרמטרים הנשמרים ברשימה (יש התייחסות ספציפית למקרה של אימייל).
 - ביצוע הפקודה שהלקוח ביקש (בדרך כלל תהיה מול בסיס הנתונים).
 - הצפנת ושליחת הודעה מתאימה בחזרה ללקוח.
- **אובייקטים נחוצים:**
 - גישה לבסיס הנתונים, תקשורת והצפנה.
 - שימוש בספרייה sqlite3.
 - שימוש בספרייה Json על מנת לשלוח דברים שהם לא רק מחרוזות בצורת מחרוזת.
- **שליחה וקבלה של תוכן מאוד גדול.**
 - **מהות:** בין הלקוח לשרת הרבה פעמים עוברות הודעות המכילות תוכן מאוד גדול (כמו למשל תמונות), לכן צריך לטפל בהודעות אלו בצורה קצת שונה. שימושים:
 - כאשר מעבירים ללקוח את כל תוכן עמוד הבית או את תוכן הקטלוג.
 - כאשר הלקוח מעביר לשרת את כל תוכן עמוד הבית או הקטלוג.
- **אוסף היכולות הנדרשות:**
 - שליחת הודעה גדולה:
 - ❖ הרכבת ההודעה והצפנתה.
 - ❖ בדיקה האם גודל ההודעה עולה על גודל המקסימלי? (נתייחס למצב שכן)
 - ❖ השרת שולח ללקוח בצורה מוצפנת הודעה המכילה רק את כמות הפקטות שהוא ישלח.
 - ❖ השרת מחלק את ההודעה שלו לפקטות ושולח את כולן אחת אחרי השנייה.
 - קבלת הודעה גדולה:
 - ❖ קליטת ההודעה מהלקוח ופענוחה.
 - ❖ כל הודעה שהשרת מקבל מהלקוח, הוא בודק האם היא מתחילה במספר? (נתייחס למקרה שכן).
 - ❖ המספר שבתחילת ההודעה מעיד על כמות הפקטות שיבואו מיד אחריה.
 - ❖ השרת מקבל את כל הפקטות בצורת לולאה ומחבר אותן להודעה מלאה.

❖ השרת מפענח את ההודעה המלאה שהתקבל.

• **אובייקטים נחוצים:**

- הצפנת/פיענוח RSA.
- תקשורת סוקטים.
- התכונה self.MAX_MSG_LENGTH שמעידה על אורך ההודעה המקסימלי.
- המשתנה content אליו נבנית כל ההודעה המתקבלת דרך הפקטות.
- שימוש ב - threading על מנת לאפשר פעולות גדולות במקביל ללקוחות שונים.

○ **שליחה וקבלה של תמונות.**

- **מהות:** כאשר מעבירים הודעה שמכילה בין השאר גם תמונות, חשוב להעביר אותן לפורמט המתאים לכך.

• **אוסף היכולות הנדרשות:**

- הפיכת תמונות לצורת מחרוזת כדי לשלוח אותה.
- או
- פיענוח תמונה לאופן בינארי.

• **אובייקטים נחוצים:**

- תקשורת והצפנה.
- שימוש בספריית base64.

○ **יצירת אפליקציה חדשה.**

- **מהות:** הוספת אפליקציה חדשה לספריית האפליקציות בעזרת הכנסת הפרטים לבסיס הנתונים.

• **אוסף יכולות הנדרשות:**

- קבלת הנתונים מהלקוח שמרכיבים את האפליקציה שלו ופיענוחם.
- פיצול ההודעה שנשלחה לפי התו @, כך שהחלק הראשון הוא הפקודה, השני הוא שם האפליקציה והשלישי הם הפרמטרים בצורה של רשימה (יש התייחסות ספציפית למקרה של אימייל).
- בדיקה מול בסיס הנתונים שהנתונים שהתקבלו הם אכן תקינים.
- במקרה של הצלחה – הם יכנסו לבסיס הנתונים, ובמקרה של כישלון הם לא.
- החזרה ללקוח הודעה על הצלחה או כישלון (עם הסבר על הבעיה) והכנסת הנתונים (רק אם זה הצלחה כמובן).

• **אובייקטים נחוצים:**

- גישה לבסיס הנתונים, תקשורת והצפנה.
- שימוש בספריית sqlite3.

- שימוש בפעולה `validate_and_insert_data` שנמצאת אצלי בתוך מחלקת יצירת בסיס הנתונים.

○ הרשמה למערכת:

- **מהות:** כניסה של משתמש חדש למערכת.
- **אוסף היכולות הנדרשות:**
 - קבלת הנתונים ופענוחם.
 - פיצול ההודעה שנשלחה לפי התו `@`, כך שהחלק הראשון הוא הפקודה, השני הוא שם האפליקציה והשלישי הם הפרמטרים בצורה של רשימה (יש התייחסות ספציפית למקרה של אימייל).
 - בדיקה מול בסיס הנתונים שהנתונים שהתקבלו הם אכן תקינים.
 - *אין פה הכנסה לבסיס הנתונים עדיין.
 - החזרה ללקוח הודעה על הצלחת או כישלון (עם הסבר על הבעיה).
 - במקרה של הצלחה ישלח אל האימייל של הלקוח קוד אימות.
- **אובייקטים נחוצים:**
 - גישה לבסיס הנתונים, תקשורת והצפנה.
 - שימוש בספריית `sqlite3`.
 - שימוש בפעולה `validate_and_insert_data` שנמצאת אצלי בתוך מחלקת יצירת בסיס הנתונים.
 - שימוש בספריות `smtplib`, `EmailMessage` על מנת לשלוח אימיילים.
 - שימוש בספרייה `random` על מנת ליצור קוד.
 - שימוש במילון `codes` שמכיל כמפתח את הלקוח (`client`) וכערך את הקוד שנשלח.

○ בדיקת קוד האימות:

- **מהות:** כדי לבצע אימות דו שלבי ולוודא שהאימייל שהמשתמש מזין תקין, המערכת שולחת אימייל המכיל קוד אימות בתוכו. את קוד האימות הזה הלקוח מחזיר ותפקידו של השרת לאמת אותו.
- **אוסף היכולות הנדרשות:**
 - קבלת קוד האימות והאימייל של הלקוח ופיענוחו.
 - בדיקה האם הקוד תקין.
 - החזרת תשובה מתאימה מוצפנת.
- **אובייקטים נחוצים:**
 - תקשורת והצפנה.

- שימוש במילון codes שמכיל כמפתח את הלקוח וכערך את הקוד שנשלח.

○ הכנסה של משתמש חדש:

- **מהות:** הכנסת פרטי המשתמש לבסיס הנתונים אחרי שעבר אימות דו שלבי.
- **אוסף היכולות הנדרשות:**
 - קבלת נתונים ופענוחם.
 - פיצול ההודעה שנשלחה לפי התו @, כך שהחלק הראשון הוא הפקודה, השני הוא שם האפליקציה והשלישי הם הפרמטרים בצורה של רשימה (יש התייחסות ספציפית למקרה של אימייל).
 - להעביר את ההודעה ב - Json מכיוון שהפרטים מגיעים דרך מילון.
 - הכנסת הנתונים לבסיס הנתונים.
- **אובייקטים נחוצים:**
 - גישה לבסיס הנתונים, תקשורת והצפנה.
 - שימוש בספריית sqlite3.
 - שימוש בספרייה Json.

○ שליחה מחדש של קוד אימות.

- **מהות:** למקרה שקוד האימות התעכב או לא הגיע, השרת יכול לשלוח אימייל נוסף, כאשר הוא מקבל מהלקוח את האימייל של המשתמש.
- **אוסף היכולות הנדרשות:**
 - קבלת נתונים, פענוחם והפרדתם לחלקים.
 - הגרלת קוד חדש ועדכון המילון עם הקודים.
 - שליחת אימייל נוסף עם הקוד.
- **אובייקטים נחוצים:**
 - גישה לבסיס הנתונים, תקשורת והצפנה.
 - שימוש בספריית sqlite3.
 - שימוש בספריות EmailMessage, smtplib על מנת לשלוח אימיילים.
 - שימוש בספרייה random על מנת ליצור קוד.
 - שימוש במילון codes שמכיל כמפתח את הלקוח וכערך את הקוד שנשלח.

○ קביעת פגישה ומחיקתה:

- **מהות:** כאשר המשתמש קובע פגישה דרך אחת האפליקציות או מוחק פגישה שהוא קבע בעבר, השרת מעדכן את בסיס הנתונים ומעדכן את המשתמש דרך האימייל. השרת מקבל מהלקוח את שם האפליקציה מולה המשתמש קובע/מוחק פגישות, את ה-ID של המשתמש ואת זמן הפגישה.

- **אוסף היכולות הנדרשות:**

- קבלת הנתונים, פיענוחם והפרדתם לחלקים.
- עדכון בסיס הנתונים בשינויים בפגישות.
- בעזרת בסיס הנתונים, השגת האימייל של המשתמש, והשם שלו.
- שליחת אימייל על השינויים.

- **אובייקטים נחוצים:**

- גישה לבסיס הנתונים, תקשורת והצפנה.
- שימוש בספריית sqlite3.
- שימוש בספריות EmailMessage, smtp lib, על מנת לשלוח אימיילים.

- **שכתי סיסמה:**

- **מהות:** אם המשתמש התחבר בעבר לאפליקציה אבל שכח את הסיסמה שלו, השרת יכול לשלוח אליו באימייל סיסמה חדשה ולעדכן אותה בבסיס הנתונים. השרת מקבל מהלקוח את שם האפליקציה אליה המשתמש מנסה להתחבר ואת האימייל של המשתמש.

- **אוסף היכולות הנדרשות:**

- קבלת הנתונים, פיענוחם והפרדתם לחלקים.
- בעזרת בסיס הנתונים להוציא את שמו של המשתמש.
- יצירת סיסמה חדשה למשתמש.
- הצפנת הסיסמה החדשה בעזרת פונקציית Hash.
- החלפת הסיסמה הישנה בבסיס הנתונים בסיסמה החדשה.
- שליחת אימייל למשתמש המכיל את הסיסמה החדשה.

- **אובייקטים נחוצים:**

- גישה לבסיס הנתונים, תקשורת והצפנה.
- שימוש בספריית sqlite3.
- שימוש בספריות EmailMessage, smtp lib, על מנת לשלוח אימיילים.
- שימוש בספריית string על מנת ליצור את הסיסמה החדשה.
- שימוש בהצפנת Hash.

פירוט היכולות בצד לקוח:

כל משימה שיש בצד המשתמש הדורשת גישה לבסיס הנתונים או לאימילים, עוברת דרך הלקוח ששולח פקודה לשרת עם מבוקשו וממשיך עם ההחזר של השרת.

○ **יצירת תקשורת בסיסית עם השרת.**

- **מהות:** חיבור הלקוח לשרת מרובה משתמשים ויצירת תקשורת בסיסית (ללא הצפנה או הודעות מיוחדות).

• **אוסף היכולות הנדרשות:**

- יצירת לקוח.
- ניסיון חיבור לקוח לשרת.
- קבלת הודעה בהתאם על הצלחת החיבור (נפרט על המצב שהחיבור הצליח אחרת אין המשך והמשתמש יעבור למסך מתאים).
- שליחת בקשות לשרת.
- קבלת תשובות מהשרת.
- לבצע פעולות מול השרת באופן הבדוק כל פעם האם השרת עדיין פעיל.
- שליחת הודעה מוצפנת לשרת.
- קבלת תשובה מהשרת ופיענוח שלה.

• **אובייקטים נחוצים:**

- המחלקה socket.
- האובייקט lock שיהיה פתוח וכך יאפשר סדר בין בקשות הלקוח, שייך למחלקה threading.
- התכונה self.status שמכילה את סטטוס החיבור ללקוח.
- בעזרת הפעולות try, except ניתן לבדוק כל פעולה האם השרת פתוח ויכול לבצע אותה.

○ **החלפת מפתחות והצפנה א-סימטרית RSA.**

- **מהות:** על מנת שהתקשורת תהיה מאובטחת, נצפין אותה בעזרת הצפנת RSA. בהצפנה זו כל אחד ממרכיבי התקשורת צריכים לשמור שני מפתחות (פרטי וציבורי), ולהחליף את המפתחות הציבוריים שלהם.

• **אוסף היכולות הנדרשות:**

- יצירת זוג מפתחות (ציבורי ופרטי)
- שליחת המפתח הציבורי לשרת עם בקשה לקבל את המפתח הציבורי שלו.
- קבלת המפתח הציבורי של השרת ופיענוחו.

• **אובייקטים נחוצים:**

- ספריית random על מנת ליצור את המפתחות.

- שימוש בקובץ RSA שבו יש פונקציה שיוצרת מפתחות.
- גישה למסמך prime2 שמכיל את כל המספרים הראשוניים של המיליון השני.
- הצפנת RSA.
- ספריית Json על מנת להפוך את המפתח הציבורי של השרת ממחרוזת לטאפל.

○ קבלת מידע והשמת מידע השמור בבסיס הנתונים:

- **מהות:** כל המידע שנוגע לאפליקציות או להתחברות משתמשים נמצא בבסיס הנתונים שרק לשרת יש יכולת להגיע אליו.
שימושים:

- השגת שמות האפליקציות הקיימות.
- בדיקה האם שם האפליקציה החדשה הוא ייחודי, כאשר הלקוח שולח ביחד עם הפקודה את שם האפליקציה החדשה ומחזיר הודעה מתאימה.
- קבלת תוכן עמוד הביתה ותוכן הקטלוג (משתמשים במקרה זה גם בפקטות והצפנת תמונות), כאשר הלקוח שולח עם הפקודה את שם האפליקציה.
- התחברות למערכת, כאשר עם הפקודה הלקוח שולח את שם האפליקציה, האימייל והסיסמה אחרי שהיא עברה hash ומחזיר הודעה מתאימה.
- הכנסת משתמש חדש לבסיס הנתונים, הלקוח שולח עם הפקודה את שם האפליקציה והפרטים של המשתמש.
- בדיקה האם אימייל ספציפי נמצא בבסיס הנתונים של אפליקציה מסוימת ומחזיר הודעה מתאימה.
- בדיקת סטטוס פגישה, כאשר הלקוח שולח בנוסף לפקודה את שם האפליקציה וזמן הפגישה ומחזיר הודעה מתאימה.
- קבלת כל הפגישות שיש למשתמש אחד. הלקוח שולח עם הפקודה את שם המשתמש ומחזיר רשימה עם כך הפגישות.
- הכנסת ערכי עמוד הבית והקטלוג לבסיס הנתונים. הלקוח שולח עם הפקודה את שם האפליקציה, את הטבלה אליה הוא מעוניין להכניס את הערכים ואת הערכים שהוא מעוניין להכניס, כאשר הערכים בצורה של טאפל והמפתחות הם שמות העמודות בטבלה אליה הוא יכניס את הערכים. הלקוח מחזיר הודעה מתאימה.
- ניקוי כל הטבלאות והפרטים של טבלה מסוימת שהלקוח ישלח את שמה עם הפקודה.
- קבלת הערכים של מערכת הפגישות, כאשר הלקוח שולח עם הפקודה את שם האפליקציה ומחזיר רשימה של הערכים.
- קבלת העיצוב של אפליקציה מבוקשת.
- ספירת כמות השורות בטבלה מסוימת, כאשר הלקוח שולח עם הפקודה את שם האפליקציה ושם הטבלה ומחזיר את כמות השורות.

- קביעת פגישה או מחיקת פגישה, כאשר הלקוח שולח עם הפקודה את שם האפליקציה, זמן הפגישה וה-ID של הלקוח.

• אוסף היכולות הנדרשות:

- הרכבת הודעה המכילה את הפקודה שהלקוח מעוניין מהשרת לבצע ואת הפרמטרים הנחוצים לכך שמופרדת על ידי התו "@".
- הצפנת ההודעה ושליחתה.
- המתנה להחזרה של השרת (רק אם יש כמובן, מעודכן לכל פעולה).
- פיענוח תגובתו של השרת.
- החזרת תגובתו של השרת לתוכנה שביקשה את זה.

• אובייקטים נחוצים:

- ממשק משתמש.
- תקשורת והצפנה/פיענוח.
- שימוש בספרייה Json על מנת לשלוח דברים שהם לא רק מחרוזות בצורת מחרוזת.

○ שליחה וקבלה של תוכן מאוד גדול.

- **מהות:** בין הלקוח לשרת הרבה פעמים עוברות הודעות המכילות תוכן מקוד גדול (כמו למשל תמונות), לכן צריך לטפל בהודעות אלו בצורה קצת שונה.
**לעומת השרת, אצל הלקוח הוא יודע כבר מראש אילו פעולות צריכות את הפקטות. שימושים:

- כאשר מעבירים ללקוח את כל תוכן עמוד הבית או את תוכן הקטלוג.
- כאשר הלקוח מעביר לשרת את כל תוכן עמוד הבית או הקטלוג.

• אוסף היכולות הנדרשות:

- שליחה הודעה גדולה:
- ❖ הרכבת ההודעה והצפנתה.
- ❖ הלקוח שולח לשרת בצורה מוצפנת הודעה המכילה את ה"הגדרות" ולא את התוכן, למשל, רק את הפקודה ושם האפליקציה ולא את התוכן שרוצים להכניס לבסיס הנתונים, כאשר לפני הפקודה יש את מספר הפקטות שהולכות להישלח.
- ❖ הלקוח מחלק את ההודעה שלו לפקטות ושולח את כולן אחת אחרי השנייה.
- קבלת הודעה גדולה:
- ❖ קליטת הודעה מהשרת ופענוחה.
- ❖ הודעה זו הולכת להכיל רק את מספר הפקטות שהולכות להישלח.
- ❖ הלקוח מקבל את כל הפקטות בצורת לולאה ומחבר אותן להודעה מלאה.
- ❖ הלקוח מפענח את ההודעה המלאה שהתקבלה.

• אובייקטים נחוצים:

- הצפנת/פיענוח RSA.
- תקשורת סוקטים.
- התכונה self.MAX_MSG_LENGTH שמעידה על אורך ההודעה המקסימלי.
- המשתנה content אליו נבנית כל ההודעה המתקבל דרך פקטות.

○ שליחה וקבלה של תמונות.

- **מהות:** כאשר מעבירים הודעה שמכילה בין השאר גם תמונות, חשוב להעביר אותן לפורמט המתאים לכך.
- **אוסף היכולות הנדרשות:**
 - הקטנת גודל התמונה לתמונה "קלה יותר" שגודלה לא יהיה יותר מ – 15 KB.
 - הפיכת תמונות לצורת מחרוזת כדי לשלוח אותה.
 - או
 - פיענוח תמונה לאופן בינארי.
- **אובייקטים נחוצים:**
 - תקשורת והצפנה.
 - שימוש בספריית base64.
 - שימוש בספריות IO ו-Image בפעולה reduce_image_binary.

○ בדיקת קוד האימות:

- **מהות:** כדי לבצע אימות דו שלבי ולוודא שהאימייל שהמשתמש מזין תקין, המערכת שולחת אימייל המכיל קוד אימות בתוכו, שאותו רק השרת יכול לאמת, לכן הלקוח צריך להעביר את הקוד אל השרת.
- **אוסף היכולות הנדרשות:**
 - קבלת קוד האימות מממשק המשתמש.
 - הרכבת בקשה מהשרת.
 - הצפנת הבקשה ושליחתה לשרת.
 - פיענוח תשובת השרת והחזרתו.
- **אובייקטים נחוצים:**
 - ממשק משתמש.
 - תקשורת והצפנה.

○ שליחה מחדש של קוד אימות.

- **מהות:** למקרה שקוד האימות התעכב או לא הגיע, הלקוח יכול לבקש שישלח שוב פעם.
- **אוסף היכולות הנדרשות:**

- שליחת נתונים לשרת הכוללים את הפקודה, והאימייל בצורה מוצפנת.

- **אובייקטים נחוצים:**

- תקשורת והצפנה.

- **שכחתי סיסמה:**

- **מהות:** אם המשתמש התחבר בעבר לאפליקציה אבל שכח את הסיסמה שלו, הלקוח יכול לבקש מהשרת לשלוח סיסמה חדשה דרך האימייל.

- **אוסף היכולות הנדרשות:**

- שליחת נתונים לשרת הכוללים את הפקודה והאימייל.

- **אובייקטים נחוצים:**

- תקשורת והצפנה.

- **בדיקת תקינות השרת:**

- **מהות:** במקרה והשרת אינו תקין או לא מחובר, על הלקוח להתכונן לכך ולדעת איך להתמודד עם מצב זה.

- **אוסף היכולות הנדרשות:**

- עטיפת כל שליחת הודעה/קבלת הודעה ב – try/except (נתייחס למקרה של except).
- סגירת הלקוח והעברה לפעולת ה – retry שמפעילה את ה – thread הראשי.
- בדיקה האם מדובר ב – thread הראשי?

- **אם כן:**

שליחה של ערך דיפולטי על מנת להמשיך את זרימת הקוד.

- **אם לא:**

סגירת ה – thread.

- **אובייקטים נחוצים:**

- Try – I except.
- הפעולה Retry_act.
- שימוש ב – threading.

מבנה/ ארכיטקטורה של הפרויקט

תיאור הארכיטקטורה של המערכת המוצעת

תיאור החומרה – רכיבים שונים והקשרים ביניהם:

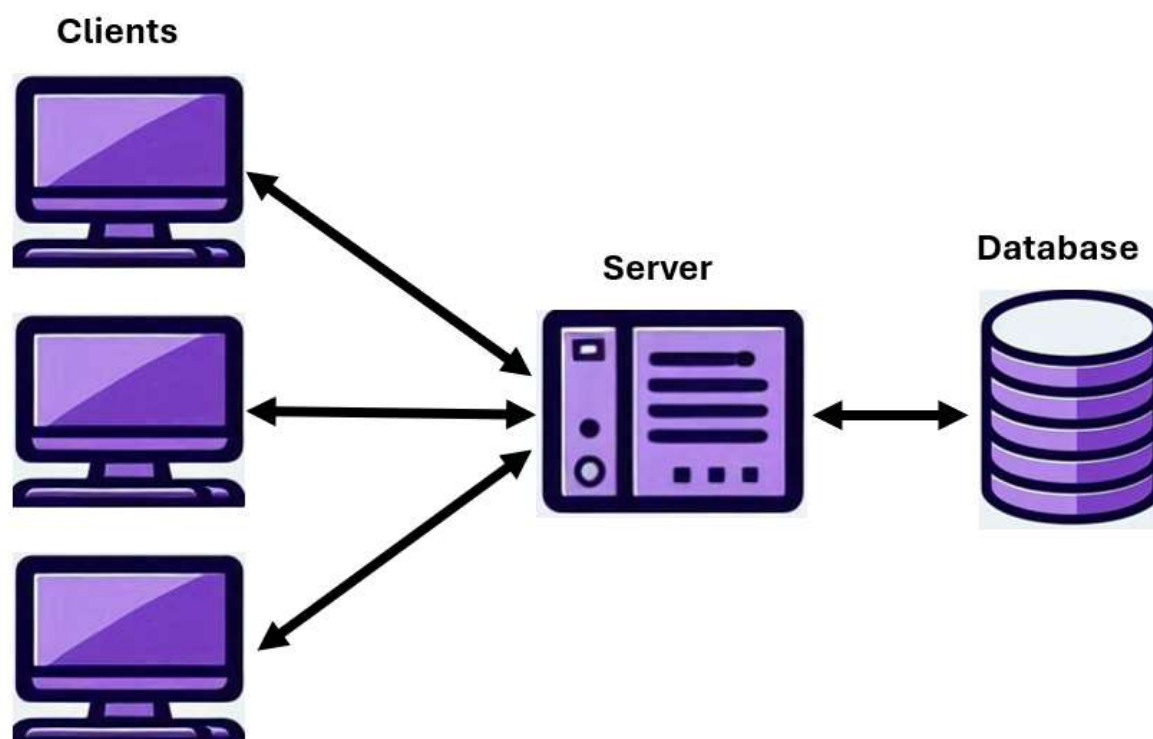
שרת:

השרת הוא מחשב חזק המריץ תוכנת שרת, אשר מאזינה לבקשות מהלקוחות ומטפלת בהן בהתאם. הוא מספק שירותים למחשבים אחרים כמו התחברות למערכת, שליחת הודעות וקבצים, ניהול גישה למסד הנתונים וביצוע הצפנת מידע בין המשתמשים. תוכנת שרת פועלת תמיד, מאזינה לבקשות (request) ומחזירה תגובות (response) בהתאם לפרוטוקול שנקבע מראש.

לקוח:

הלקוח הוא מחשב או התקן קצה שמתחבר לשרת לצורך שליחת וקבלת נתונים. הוא פועל כמשתמש פעיל במערכת ומבצע פעולות בהתאם לצורך. במהלך העבודה עם השרת, הלקוח שולח בקשות שונות, כגון התחברות למערכת, שליחת הודעות והעלאת קבצים, כאשר השרת מעבד את הבקשות ומחזיר לו את המידע הנדרש. בנוסף, הלקוח נדרש להצפין את המידע שהוא שולח כדי להבטיח תקשורת מאובטחת ולמנוע גישה לא מורשית לנתונים רגישים. בעת קבלת מידע מהשרת, הלקוח מציג אותו באמצעות ממשק משתמש נוח. לעומת תוכנת השרת, תוכנת הלקוח פועלת רק כאשר המשתמש מפעיל אותה ומבצע פעולות תקשורת מול השרת.

שרטוט המציג את הקשרים בין רכיבי החומרה



תיאור טכנולוגיה רלוונטית

שפת תכנות:

רוב הקוד בפרויקט נכתב בשפת Python שהיא שפת תכנות דינמית מהנפוצות ביותר.

מערכת הפעלה:

הפרויקט בנוי על מערכת ההפעלה Windows מבית מיקרוסופט.

תקשורת שרת-לקוח:

צורת התקשורת של הפרויקט היא של שרת מרובה משתמשים, עם שימוש ב-socket המתבסס על פרוטוקול TCP.

בסיס נתונים SQL :

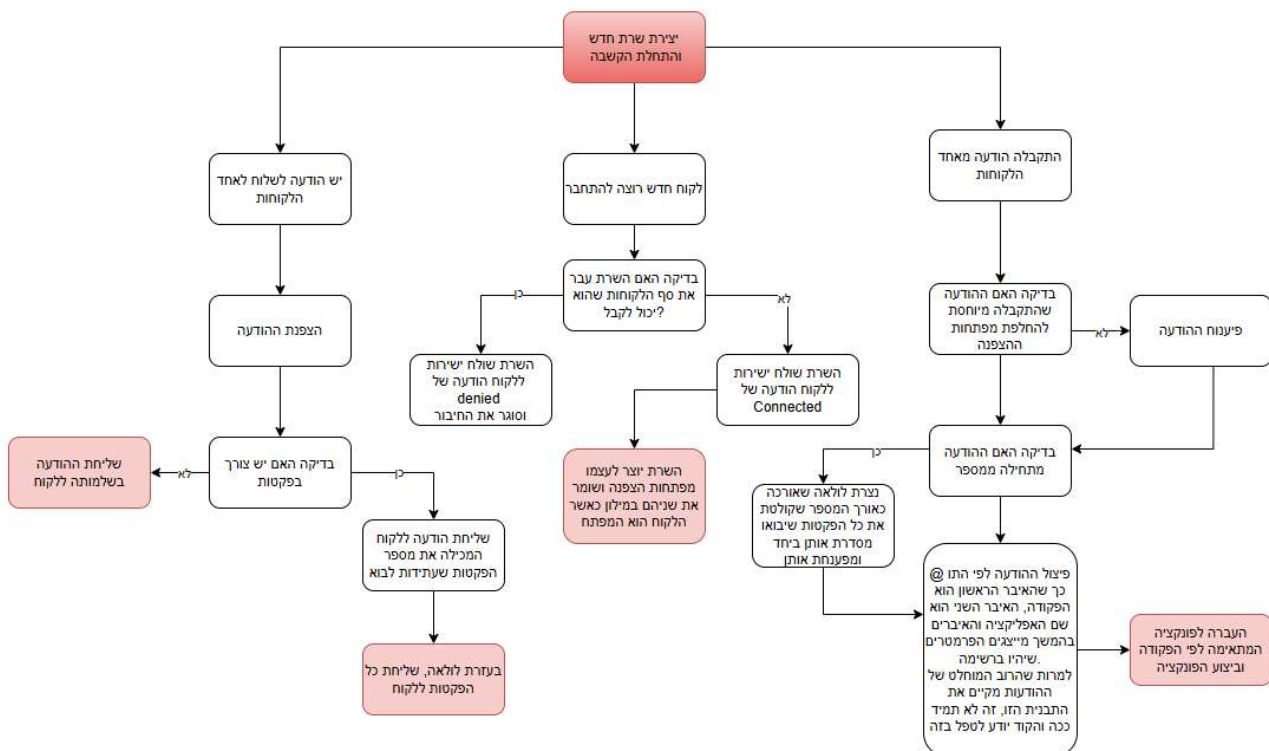
השימוש בבסיס הנתונים מבוסס על SQL, מאפשר שליפה ועדכון נתונים, יצירה ושינוי של מבני טבלאות בהתאם לצרכים השונים של המערכת.

הצפנת RSA :

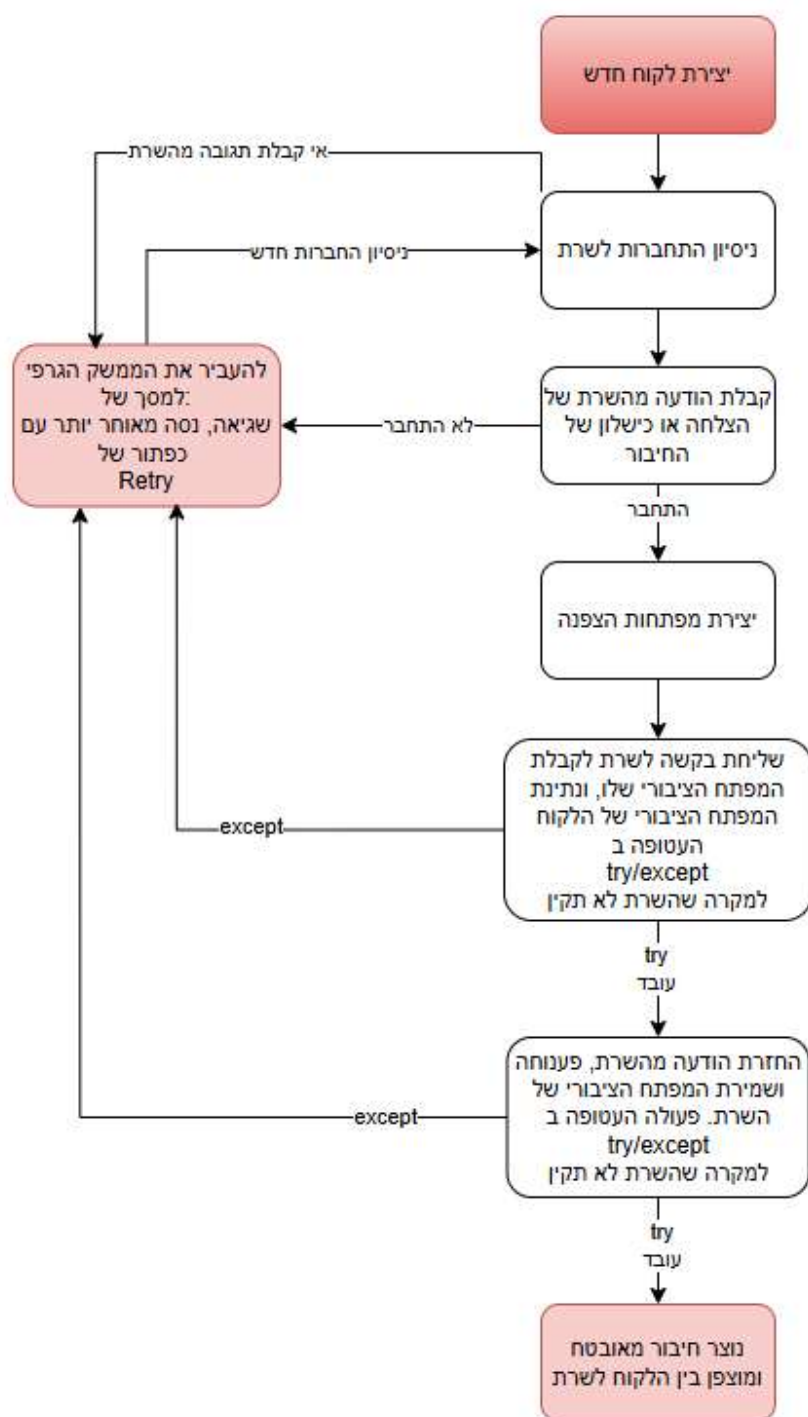
הפרויקט עושה שימוש בהצפנת RSA שהיא מערכת הצפנת מפתח ציבורי מהנפוצות ביותר במערכות אבטחת מידע מודרניות. ב - RSA מפתח ההצפנה אינו סודי אך שונה ממפתח הפענוח שנשמר בסוד, ולכן היא נקראת שיטת הצפנה א-סימטרית.

תיאור זרימת המידע במערכת

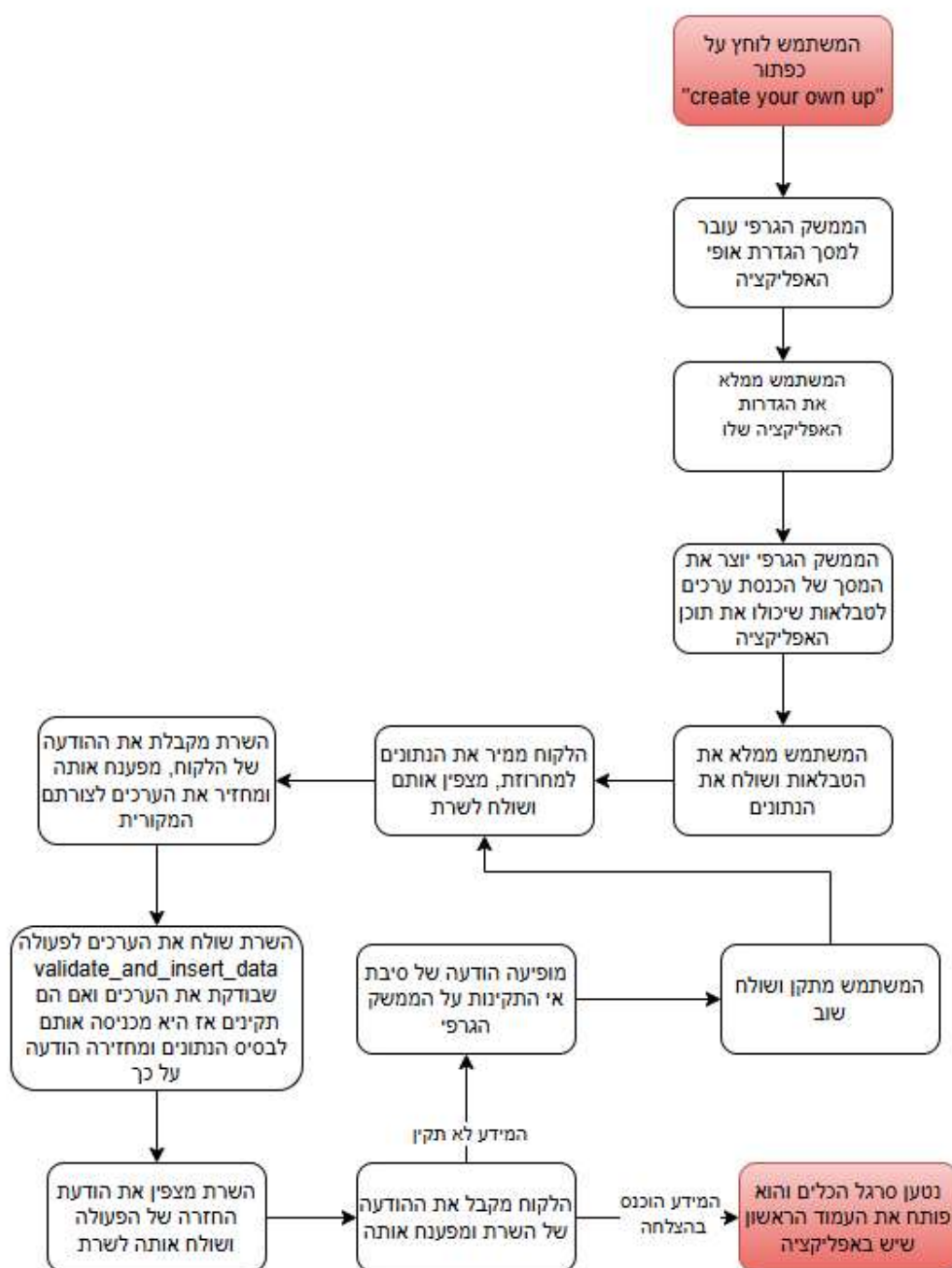
פתיחת השרת ותפקודו:



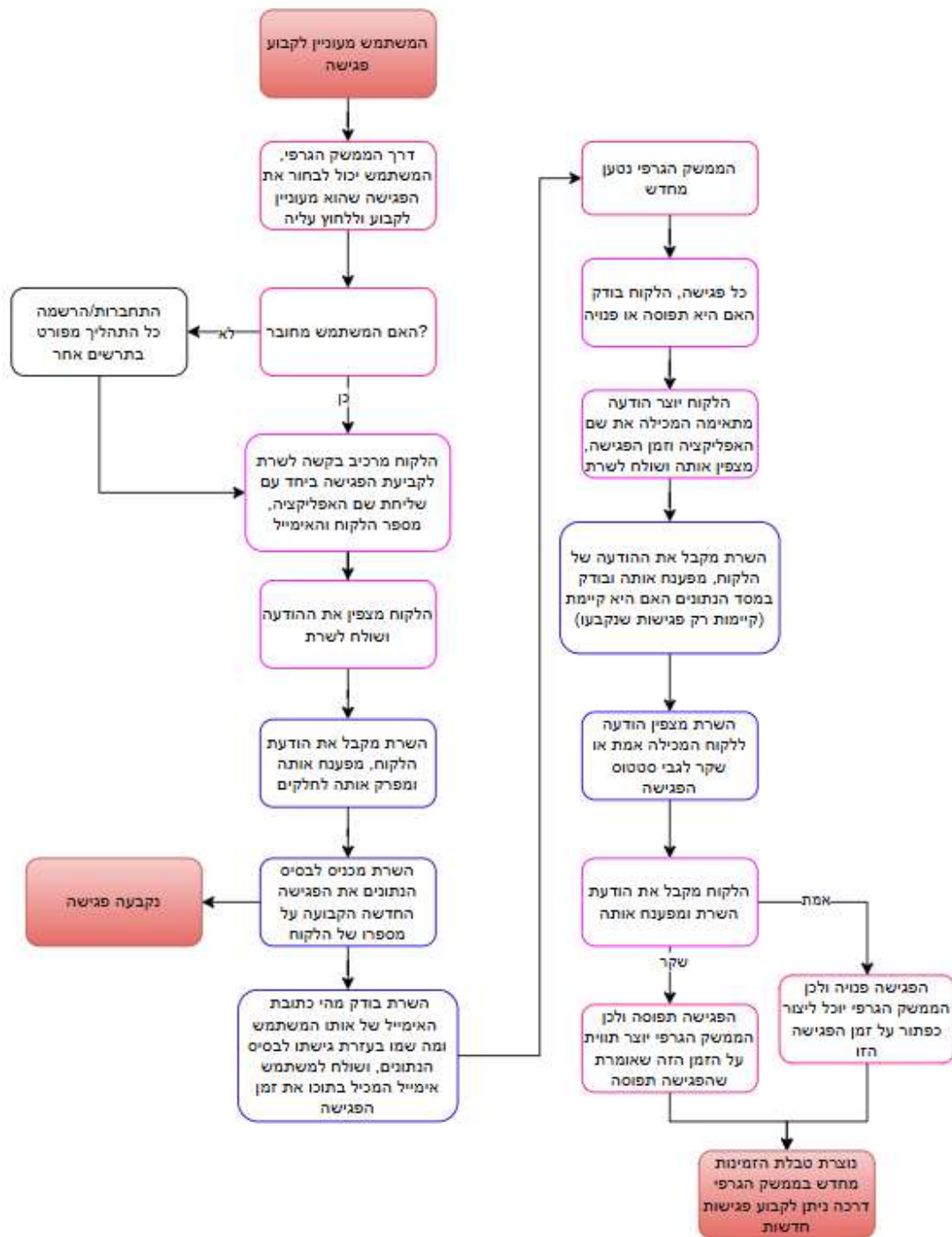
תפקוד הלקוח:



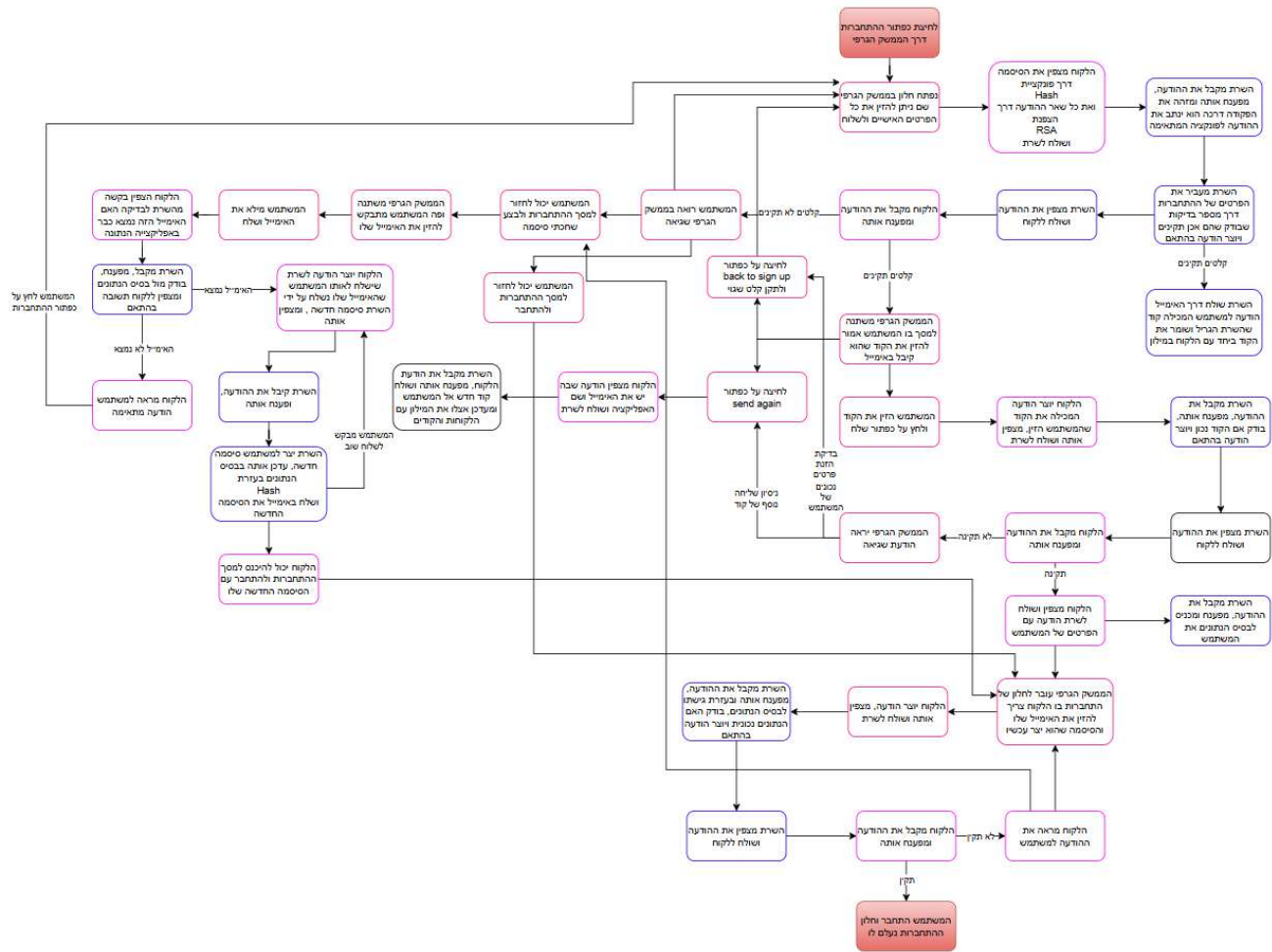
יצירת אפליקציה:



קביעת פגישה:



התחברות למערכת:



תיאור האלגוריתמים מרכזיים בפרויקט

העברת הודעות לא רגילות בתקשורת:

ניסוח וניתוח הבעיה האלגוריתמית:

בעת יצירת התקשורת בין השרת ללקוחות, נדרשתי להתחשב בכמה גורמים מרכזיים. ראשית, מכיוון שהתוכן שהעברתי לא תמיד היה מסוג מחרוזת, היה צורך לוודא תמיכה בסוגי נתונים שונים ולנהל אותם בהתאם. בנוסף, בשל העברת התמונות שהפרויקט שלי מטפל בהן, גודל ההודעות לעיתים חרג מהמגבלה המקסימלית של 10,240 בתים, מה שחייב טיפול מיוחד לניהול ושינוע המידע באופן אופטימלי. פתרון הבעיות הללו היה חיוני להבטחת תקשורת יעילה ותקינה בין השרת ללקוחות.

אלגוריתמים קיימים לפתרון הבעיה:

- שימוש ב - Json כדי להעביר טיפוסים מורכבים, כאשר הוא מעביר אותם לצורה של string ולאחר השליחה מפענח אותם בחזרה.

- המרה בין string ל - bytes ולהפך בעזרת ספריית base64.
- חלוקה לפקטות של תוכן ההודעה ושליחת הפקטות.
- שימוש בספריית ast המאפשרת להמיר מחרוזת המכילה קוד תקף בפיתון לעץ תחבירי שניתן לעיבוד.

הפנייה למקורות רלוונטיים:

שימוש ב - AI וחיפוש באתרים כמו stack overflow.

סקירת הפתרון הנבחר:

כדי שפרוטוקול התקשורת שלי יעבוד לכל הקלטים וההודעות הקיימות בתקשורת, הייתי צריכה להשתמש בגישות שונות לפי הקלטים:

במקרה של מבני נתונים מורכבים כמו רשימות או מילונים, השתמשתי ב - `json.dumps()` כדי להמיר את הנתונים למחרוזת, מה שאפשר שליחתם בצורה פשוטה דרך התקשורת, ולאחר קבלתם השתמשתי ב - `json.loads()` כדי לשחזר את המבנה המקורי.

כאשר לא היה צורך להעביר את כל המחרוזת, או כשהיה צורך לעבד חלקים מסוימים בלבד, השתמשתי ב - `ast.literal_eval()` שאפשר גישה ישירה למשתנים ולמבנים תחביריים בפיתון.

מבחינת ההשוואה בין JSON ל AST, JSON הוא פורמט נתונים טקסטואלי שמאפשר העברת מידע בין מערכות בצורה פשוטה, והוא קומפקטי וקל לשימוש עם פונקציות מובנות להמרה ושחזור. לעומתו AST, מיועד יותר לניתוח מבנה קוד בפיתון, מאפשר גישה ישירה לרכיבים תחביריים, אך דורש עיבוד נוסף ואינו מתאים תמיד להעברת מידע בצורה ישירה כמו JSON. בסופו של דבר JSON הוא הבחירה הטבעית להעברת נתונים ברשת, בעוד AST שימושי בעיקר לניתוח קוד ועיבוד חלקים מסוימים ממנו.

כאשר גודל ההודעה שרציתי להעביר בתקשורת הייתה גדולה מגודל ההודעה המקסימלי שהוגדר (10240 בתים), בחרתי לפצל את תוכן ההודעה לפקטות. בחרתי בשיטה זו מכיוון שפיצול תוכן ההודעה לפקטות מאפשר לשלוח מידע גדול בצורה יעילה מבלי לחרוג מהמגבלות שנקבעו בתקשורת. בשיטה זו, כל הודעה גדולה מחולקת לחלקים קטנים יותר (פקטות), כך שניתן לשלוח אותם בזה אחר זה ולוודא שכל חלק מתקבל בשלמותו.

כאשר רציתי להעביר תמונות בתקשורת, השתמשתי בפונקציה `base64` שהופכת את התוכן הבינארי של התמונה לטקסט. אמנם זה מספיק בשביל לבצע את העברת המסרים, אך מכיוון שתמונות יכולות להיות מאוד גדולות וכך הן עלולות לעכב את המערכת, הוספתי עוד פעולה שמקטינה את גודל התמונה בצורה שלא ממש שמים לב אליה.

הצפנת ההודעות- שימוש ב - RSA:

ניסוח וניתוח הבעיה האלגוריתמית:

RSA הוא אחד האלגוריתמים הנפוצים ביותר להצפנה א-סימטרית, והוא פותר את הבעיה של העברת מידע

מאובטח בין צדדים שלא חלקו מראש מפתח סודי. הבעיה המרכזית היא כיצד לאפשר הצפנה ודיסקרטיות, כך שרק הנמען המיועד יוכל לפענח את ההודעה, מבלי לחשוף מידע רגיש לאף צד אחר.

אלגוריתמים קיימים לפתרון הבעיה:

לפני פיתוח אלגוריתם RSA, השיטות המרכזיות ששימשו להצפנה היו מבוססות על הצפנה סימטרית, כגון AES ו-DES. באלגוריתמים אלו, שני הצדדים המשתתפים בתקשורת היו מחויבים לשתף ביניהם מפתח סודי זהה מראש, שהיה נדרש הן לתהליך ההצפנה והן לפענוח המידע. גישה זו, למרות יעילותה, יצרה אתגר משמעותי בניהול והפצת המפתחות, במיוחד בסביבות עם מספר רב של משתמשים, שכן שמירת המפתח המשותף באופן מאובטח הייתה משימה מורכבת ודורשת משאבים רבים.

סקירת הפתרון הנבחר:

אלגוריתם RSA נבחר בזכות רמת האבטחה הגבוהה שלו, הנובעת מהמורכבות הכרוכה בפירוק מספרים גדולים לגורמים. קושי זה הופך אותו לפתרון יעיל להצפנה מאובטחת והגנה על מידע רגיש. הוא עובד כך:

1. יצירת זוג מפתחות – מפתח ציבורי להצפנה ומפתח פרטי לפענוח.
2. שימוש בחישוב מודולרי ופעולות אריתמטיות פשוטות אך קשות לפירוק.
3. האלגוריתם מבוסס על המורכבות של פירוק מספרים גדולים לגורמים, שהיא בעיה חישובית קשה לפתרון בפרק זמן סביר.

אלגוריתם RSA מצטיין בפשטות המתמטית שלו תוך שמירה על רמת אבטחה גבוהה, מה שהופך אותו לאחד הפתרונות המובילים בתחום ההצפנה. יתרון משמעותי נוסף הוא שאין צורך לשתף מפתח סודי מראש, שכן השיטה מבוססת על שימוש במפתח ציבורי להצפנה ובמפתח פרטי לפענוח.

התחברות מאובטחת, שימוש ב - Hashing, מניעת SQL Injection וביצוע אימות דו שלבי:

ניסוח וניתוח של הבעיה האלגוריתמית:

בעת פיתוח מנגנון התחברות מאובטח למערכת, יש להתמודד עם מספר איומים מרכזיים, כגון גניבת סיסמאות, חשיפת מידע רגיש, והתקפות SQL Injection. שימוש ישיר בערכי משתמש בתוך שאילתות SQL ללא אבטחה מתאימה עלול לאפשר לתוקף להחדיר קוד זדוני, לבצע שינויים לא רצויים במסד הנתונים ואף לגשת למידע חסוי של משתמשים.

אלגוריתמים קיימים לפתרון הבעיה:

כדי להבטיח אבטחה גבוהה, משתמשים בשיטות הבאות:

- Hashing לסיסמאות – הפיכת סיסמאות למחרוזות מוצפנות חד-כיוונית באמצעות אלגוריתמים כמו bcrypt, SHA-256 או Argon2 כך שגם אם הנתונים נחשפים, הסיסמאות אינן ניתנות לשחזור ישיר.
- שימוש ב – Prepared Statements - מנגנון מאובטח לביצוע שאילתות SQL שמונע הזרקות זדוניות על ידי שימוש בפרמטרים בטוחים, במקום חיבור ישיר של קלט המשתמש לשאילתה.
- אימות דו-שלבי - שכבת הגנה נוספת המוודאת שהמשתמש הוא אכן בעל החשבון, באמצעות קוד חד-פעמי או אמצעי אימות ביומטרי.

סקירת הפתרון הנבחר:

בפרויקט שלי, מנגנון האבטחה מבוסס על שילוב של Hashing עבור הסיסמאות, כדי להבטיח שהן אינן נשמרות בפורמט קריא או ניתנות לשחזור ישיר. בנוסף, השימוש ב Prepared Statements - מבטיח שהשאילתות במסד הנתונים אינן חשופות להזרקות SQL, שכן קלט המשתמש מנוהל בפרמטרים מאובטחים במקום כחלק ישיר מהשאילתה.

יתר על כן, בעת התחברות משתמש חדש למערכת, יש שימוש באימות דו שלבי שיוודא כי הפרטים של המשתמש אכן נכונים, ובכך יאפשר מאוחר יותר להחליף את הסיסמה של אותו המשתמש במקרה של תקלה/בעיה או שהמשתמש שכח בעצמו.

תיאור סביבת הפיתוח

פירוט כלי הפיתוח הנדרשים לפיתוח:

1. Python 3.12
2. Pycharm Community Edition – לצורך כתיבה והרצת הקוד.
3. DB Browser (SQLite) - ליצירת בסיס הנתונים ולצפייה במידע הנמצא בתוך קבצי בסיס הנתונים.

פרוט הסביבה והכלים הנדרשים לבדיקות:

1. שורת הפקודה CMD – הזנת הפקודה ipconfig לצורך מציאת כתובת ה-IP של המחשב שממנו רץ השרת (כדי לאפשר ריצה במקביל בכמה מכשירים).
2. WIRESHARK – תוכנת הסנפה שמאפשרת לוודא שכל הפקטות עוברות כראוי בין צדדי התקשורת השונים, ולזהות איזה מידע רגיש נותר חשוף בעת הסנפה.
3. DB Browser (SQLite) – לצפייה במידע במסד הנתונים ולוודא שהוא נכנס ונשלף כמו שצריך.

תיאור פרוטוקול התקשורת

תיאור מילולי של פרוטוקול התקשורת:

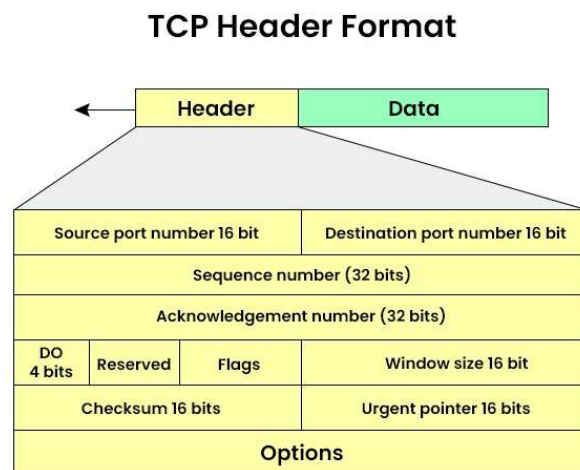
פרוטוקול התקשורת של המערכת מבוסס על TCP (Transmission Control Protocol) שהוא פרוטוקול אמין ויציב להעברת מידע ברשתות מחשבים. TCP פועל בשכבת התעבורה (השכבה הרביעית במודל ה-OSI) ומבטיח העברה מסודרת, בטוחה וללא שגיאות של נתונים בין תחנות ברשת באמצעות מנגנון חיבור מקושר.

בעת יצירת חיבור בין שני הצדדים, TCP מוסיף לכל חבילת מידע Header בגודל של 20 עד 60 בתים, המכיל נתונים חיוניים לניהול התקשורת, כולל מספרי רצף, אמצעי תיקון שגיאות, וסימוני שלמות המידע. מנגנון זה מאפשר שמירה על סדר הנתונים והבטחת העברה תקינה גם כאשר התווך בין התחנות אינו יציב.

בנוסף, הפרוטוקול מאפשר מעבר נתונים דרך כתובות IP תוך שיוך כל תקשורת לפורט ייעודי השייך לאפליקציה מסוימת – חלק אינטגרלי מחבילת הפרוטוקולים TCP/IP. בפרויקט שלי, הפרוטוקול משולב ומיושם בתוך מחלקת ה-Socket של Python, מה שמאפשר תקשורת יעילה עם לקוחות מרובים באופן מבוקר ומאובטח.

פרוטוקול TCP בו אני משתמשת לצורך התקשורת בין השרת לבין כל אחד ממשתמשי הקצה, מחייב את מקור ההודעה בתהליך שנקרא "לחיצת ידיים משולשת" (Three Way Handshake), תהליך הכולל שלושה שלבים: **שליחת SYN** – הלקוח שולח בקשה לשרת להתחלת חיבור. **קבלת SYN-ACK** – השרת מגיב ומאשר את הבקשה. **שליחת ACK** – הלקוח שולח אישור סופי, והחיבור הופך למקושר.

מבנה פקטת TCP:



פירוט כלל ההודעות הזורמות במערכת:

תיאור ההודעות שנשלחות:

בפרויקט שלי אורכה של כל הודעה שמתקבלת לא יעלה על 10240 בתים (פי 10 מהסטנדרט). זאת מכיוון שאין

בעיה למערכת שלי לשלוח כמות כזו של בתים בפעם אחת, וכך המערכת חוסכת לי להשתמש בפקטות להודעות יחסית קצרות.

○ הודעה הנשלחת בפקטה אחת מהלקוח:

ההודעה מורכבת ממחרוזת, כאשר כדי להפריד בין הנתונים השונים בה, השתמשתי בתו @ (במקרה של שימוש באימייל, יש התייחסות לכך). לסדר הפריטים במחרוזת יש גם כן השפעה, כך שהאיבר הראשון הוא הפקודה, השני הוא שם האפליקציה והבאים בתור הם הפרמטרים. במקרה שאין צורך בשם האפליקציה או יש חריגה מהתבנית, יש התייחסות לכך בקוד.

○ הודעה הנשלחת עם פקטות מהלקוח:

נשלחת הודעה ראשונה שהיא מכילה את מספר הפקטות שההודעה מכילה (אורך ההודעה אחרי שהיא מוצפנת/מספר הבתים המקסימלי), ונתוני הגדרות, כמו הפקודה, שם האפליקציה ופרמטר רלוונטי קצר. לאחר מכן, בצורה של לולאה, נשלחות כל שאר הפקטות, כאשר הן מכילות את החלקים הארוכים בהודעה, כמו נניח התמונות.

○ הודעה הנשלחת בפקטה אחת מהשרת:

השרת בפרויקט לא שולח הודעות עם בסיס מורכב מראש, מכיוון שאין אצל הלקוח ניתוב הודעות. השרת מחזיר את ההודעה שלו אל אותה פעולה אצל הלקוח ששלחה את ההודעה/בקשה.

○ הודעה הנשלחת עם פקטות מהשרת:

השרת מתחיל מלשלוח רק את מספר הפקטות העתידות לבוא, ולאחר מכן בצורה של לולאה שולח את כל הפקטות.

פירוט ההודעות שנשלחות:

*כולן מוצפנות חוץ מהתחברות הלקוח לשרת ותשובתו, והעברת המפתח של הלקוח לשרת.
**כלל מבני השדות בהודעה שאינם מחרוזת הופכות למחרוזת בעזרת JSON.

1. **שם ההודעה:** התחברות לשרת

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: (self.SERVER_IP, self.SERVER_PORT)

2. **שם ההודעה:** סטטוס חיבור

נשלחת מ: השרת **אל:** הלקוח

מבנה השדות בהודעה: "connected"/"denied"

3. **שם ההודעה:** החלפת מפתחות – get_server_public_key

נשלחת מ: הלקוח **אל:** השרת

- מבנה השדות בהודעה:** `f"GET_SERVER_PUBLIC_KEY@{self.keys[0]}"`
 כאשר `self.keys` מכיל את המפתחות של הלקוח ומיקום 0 מכיל את המפתח הציבורי.
4. **שם ההודעה:** החלפת מפתחות
נשלחת מ: השרת **אל:** הלקוח
מבנה השדות בהודעה: המפח הציבורי של השרת (מספר).
5. **שם ההודעה:** קבלת כל שמות האפליקציות – `get_apps_names`
נשלחת מ: הלקוח **אל:** השרת
מבנה השדות בהודעה: `"GET_APPS_NAMES"`
6. **שם ההודעה:** שליחת כל שמות האפליקציות – `fetch_apps_names`
נשלחת מ: השרת **אל:** הלקוח
מבנה השדות בהודעה: רשימה של כל השמות.
7. **שם ההודעה:** בדיקה האם שם האפליקציה הוא ייחודי – `check_unique_name`
נשלחת מ: הלקוח **אל:** השרת
מבנה השדות בהודעה: `f"CHECK_UNIQUE_NAME@{app_name}"`
8. **שם ההודעה:** החזרת תשובה לגבי ייחוד השם – `check_unique_name`
נשלחת מ: השרת **אל:** הלקוח
מבנה השדות בהודעה: מחרוזת של השם או 0.
9. **שם ההודעה:** לקבל את תוכן עמוד הבית – `get_homepage_content`
נשלחת מ: הלקוח **אל:** השרת
מבנה השדות בהודעה: `f"GET_HOMEPAGE_CONTENT@{app_name}"`
10. **שם ההודעה:** לשלוח את תוכן עמוד הבית – `fetch_homepage_content`
נשלחת מ: השרת **אל:** הלקוח
מבנה השדות בהודעה: מערך דו מימדי המכיל את תוכן כל הטבלה של עמוד הבית. אם נדרשות פקטות, אז תשלח קודם הודעה המכילה את מספר הפקטות ולאחר מכן ישלח המערך כאשר הוא מחולק לפקטות.
11. **שם ההודעה:** לקבל את תוכן הקטלוג – `get_catalog_content`
נשלחת מ: הלקוח **אל:** השרת
מבנה השדות בהודעה: `f"GET_CATALOG_CONTENT@{app_name}"`
12. **שם ההודעה:** לשלוח את תוכן הקטלוג – `fetch_catalog_content`
נשלחת מ: השרת **אל:** הלקוח

מבנה השדות בהודעה: מערך דו מימדי המכיל את תוכן כל הטבלה של הקטלוג. אם נדרשות פקטות, אז תשלח קודם הודעה המכילה את מספר הפקטות ולאחר מכן ישלח המערך כאשר הוא מחולק לפקטות.

13. **שם ההודעה:** התחברות למערכת – login_to_system

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: f"LOG_IN@{app_name}@{email}@{password}"

14. **שם ההודעה:** סטטוס התחברות – handle_login

נשלחת מ: השרת **אל:** הלקוח

מבנה השדות בהודעה: "false" אם ההתחברות לא עברה בהצלחה או את פרטי המשתמש.

15. **שם ההודעה:** בדיקת קוד אימות דו שלבי – check_confirmation

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: f"CHECK_CONFIRMATION@{code}" כאשר הקוד הוא קוד האימות שהמשתמש הזין.

16. **שם ההודעה:** סטטוס אימות

נשלחת מ: השרת **אל:** הלקוח

מבנה השדות בהודעה: "true"/"false"

17. **שם ההודעה:** בקשת שליחת קוד אימות בשנית באימייל – send_confirmation_again

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: f"SEND_CONFIRMATION_AGAIN@{gmail}"

18. **שם ההודעה:** בקשה להכניס משתמש למערכת – insert_user

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: f"INSERT_USER@{app_name}@{json.dumps(user_var)}"

19. **שם ההודעה:** שליחה סיסמה חדשה למשתמש באימייל ובקשה לעדכן אותה במערכת –

send_new_password

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: f"SEND_NEW_PASSWORD@{app_name}@{user_gmail}"

20. **שם ההודעה:** בדיקה האם אימייל מסוים נמצא באפליקציה – `check_gmail_in_app`

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: `f"CHECK_GMAIL_IN_APP@{app_name}@{gmail}"`

21. **שם ההודעה:** סטטוס בדיקת אימייל באפליקציה – `check_gmail_in_app`

נשלחת מ: השרת **אל:** הלקוח

מבנה השדות בהודעה: `"true"/"false"`

22. **שם ההודעה:** בדיקת פניות של פגישה – `check_meeting_availability`

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: `f"CHECK_MEETING_AVAILABILITY@{app_name}@{meeting_time}"`

23. **שם ההודעה:** סטטוס פניות של פגישה – `check_meeting_availability`

נשלחת מ: השרת **אל:** הלקוח

מבנה השדות בהודעה: `"true"/"false"`

24. **שם ההודעה:** קביעת פגישה – `book_meeting`

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: `f"BOOK_MEETING@{app_name}@{meeting_time}@{customer_id}"`

25. **שם ההודעה:** הצלחת קביעת הפגישה – `book_meeting`

נשלחת מ: השרת **אל:** הלקוח

מבנה השדות בהודעה: `"true"/"false"`

26. **שם ההודעה:** בקשה לקבל את כל הפגישות של משתמש מסוים – `get_users_meetings`

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: `f"GET_USERS_MEETINGS@{app_name}@{customer_id}"`

27. **שם ההודעה:** מחזיר את כל הפגישות של המשתמש – `getting_users_meeting`

נשלחת מ: השרת **אל:** הלקוח

מבנה השדות בהודעה: רשימה של כל הפגישות.

28. **שם ההודעה:** בקשה למחיקת פגישה – `delete_meeting`

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: `f"DELETE_MEETING@{app_name}@{user}@{meeting_time[0]}"`

29. **שם ההודעה:** הצלחת מחיקת הפגישה – `delete_meeting`

נשלחת מ: השרת **אל:** הלקוח

מבנה השדות בהודעה: "true"/"false"

30. **שם ההודעה:** הכנסת ערכים לטבלה – insert_into_table

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: הודעה ראשונה - f"{packets_num}@INSERT_INTO_TABLE@{app_name}"

אחריה כל הפקטות האחרות ישלחו בצורת לולאה-

send_values[i * self.MAX_MSG_LENGTH:(i + 1) * self.MAX_MSG_LENGTH]

31. **שם ההודעה:** סטטוס הכנסת הערכים

נשלחת מ: השרת **אל:** הלקוח

מבנה השדות בהודעה: מחרוזת שבה רשום הצלחה/כשל בהכנסת הערכים ואת הסיבה לכשל.

32. **שם ההודעה:** ניקוי בסיס הנתונים של טבלה מסוימת – clear_app_database

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: f"CLEAR_APP_DATABASE@{app_name}"

33. **שם ההודעה:** בקשה לקבלת הערכים של מערכת הפגישות – get_meeting_system_vars

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: f"GET_MEETING_SYSTEM_VARS@{app_name}"

34. **שם ההודעה:** מחזיר את הערכים של מערכת הפגישות - getting_meeting_system_vars

נשלחת מ: השרת **אל:** הלקוח

מבנה השדות בהודעה: רשימה של כל הערכים של מערכת הפגישות.

35. **שם ההודעה:** בקשה לקבל את העיצוב של האפליקציה – get_design

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: f"GET_DESIGN@{app_name}"

36. **שם ההודעה:** מחזיר את העיצוב של האפליקציה – getting_design

נשלחת מ: השרת **אל:** הלקוח

מבנה השדות בהודעה: מחזיר את שם העיצוב.

37. **שם ההודעה:** בקשה לקבל את מספר השורות שנמצאות בטבלה מסוימת – count_rows

נשלחת מ: הלקוח **אל:** השרת

מבנה השדות בהודעה: f"COUNT_ROWS@{app_name}@{table_name}"

38. **שם ההודעה:** סופר את השורות בטבלה ומחזיר את התוצאה – `counting_rows`

נשלחת מ: השרת **אל:** הלקוח

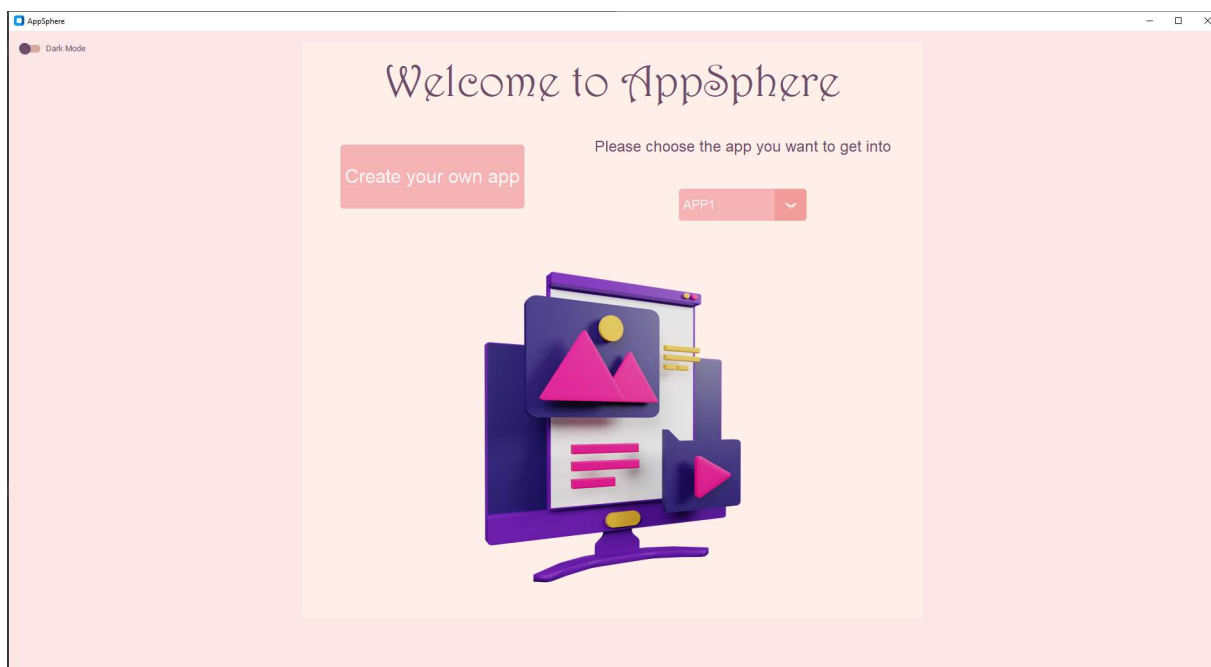
מבנה השדות בהודעה: מחזיר את מספר השורות בטבלה.

תיאור מסכי המערכת

תיאור כל מסך:

1. **מסך פתיחה:**

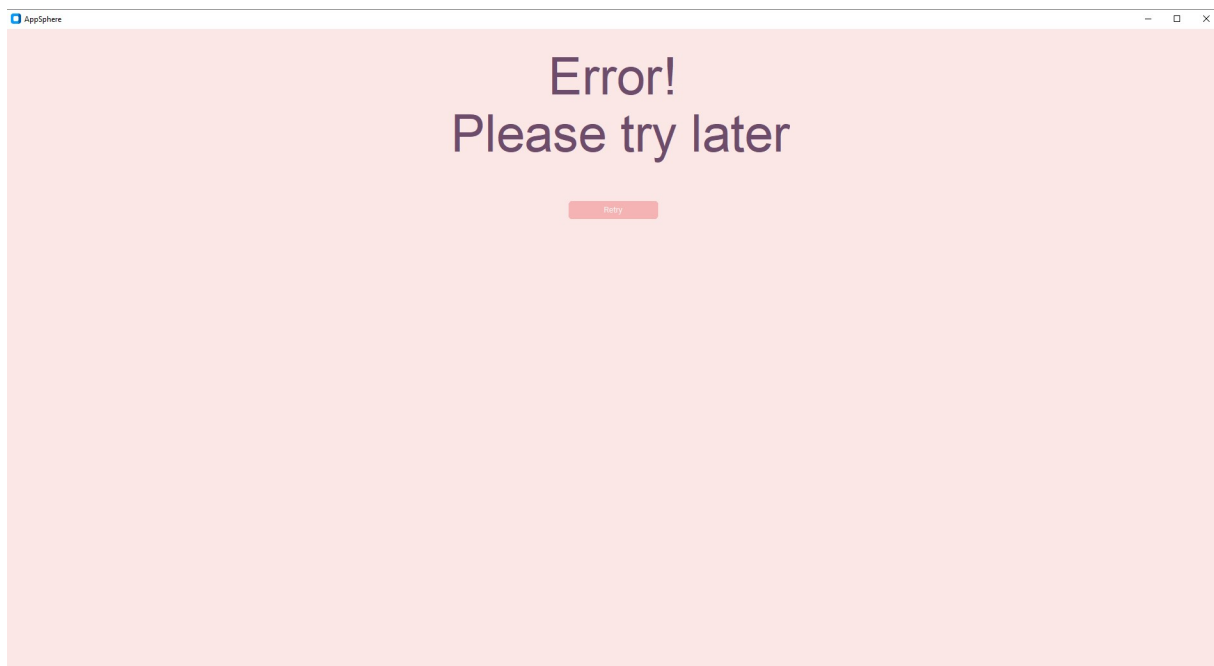
המסך הראשון אותו רואים כאשר מתחברים לאפליקציה. יש בו שתי אופציות: ליצור אפליקציה משלך או להיכנס לאפליקציות קיימות לפי ה - option menu שנמצא שם.



2. **מסך שגיאה:**

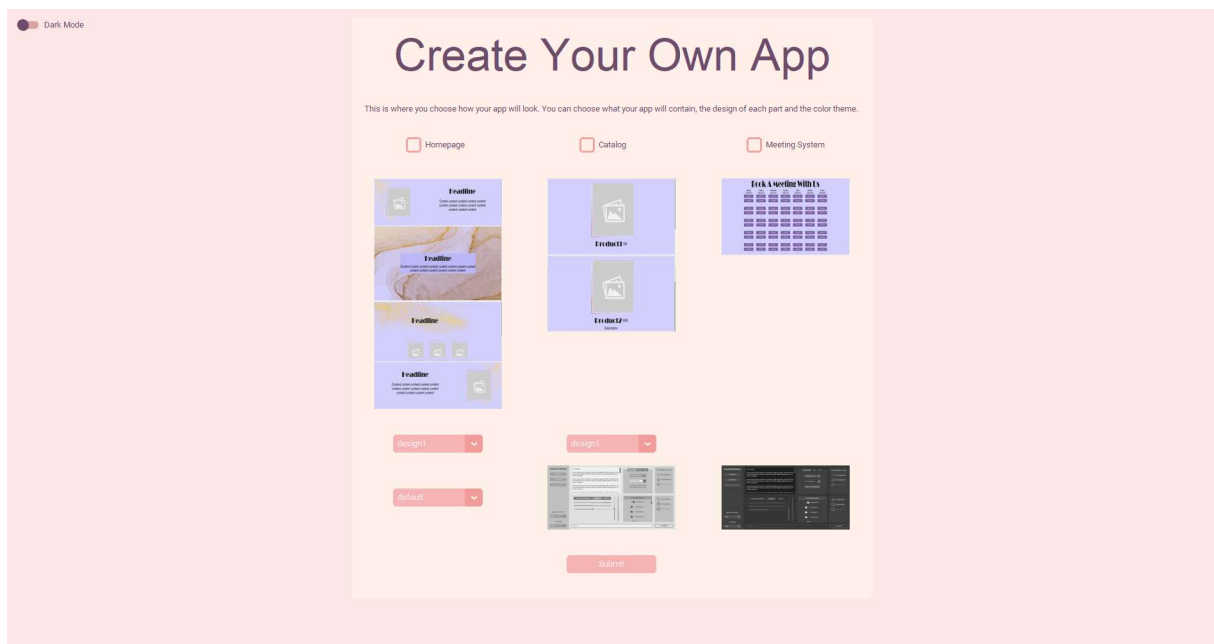
במקרה ומסיבה כלשהיא לא ניתן להתחבר לשרת (אם השרת הגיע לתכולתו המלאה או שהוא לא הופעל או לא רץ כרגע מסיבה כלשהיא), אז המשתמש יראה את המסך הזה, דרכו הוא יכול ללחוץ `retry` ולקוות שהוא

יצליח להתחבר בקרוב.



3. מסך הגדרת תכונות האפליקציה:

אם המשתמש רוצה ליצור לעצמו אפליקציה, אז הוא יגיע אל מסך זה. במסך זה יש אופציות בשביל לעצב את האפליקציה כך שהעיצוב שלה יתאים לרצונות של היוצר עד כמה שאפשר. במסך הזה היוצר יכול לבחור את החלקים שהוא רוצה שיהיו באפליקציה שלו (עמוד בית/קטלוג/מערכת פגישות), הוא יכול לבחור את העיצוב של כל חלק ואת צבע ערכת הנושא של האפליקציה שלו.



4. מסך טבלאות להכנסת ערכים לבסיס הנתונים:

לאחר לחיצה על הכפתור Submit במסך הקודם, יפתח המשך למסך הקודם שבו היוצר יוכל להכניס את הערכים שהוא רוצה שיהיו באפליקציה שלו. כלומר במסך זה היוצר מכניס אל בסיס הנתונים את פרטי האפליקציה שלו.

במסך הבית – היוצר בוחר את סוג ה"בלוק" שהוא רוצה וממלא את המקומות שנפתחו לו, לאחר מכן תפתח לו שורה חדשה בה הוא יוכל להמשיך להוסיף מידע. היוצר חייב למלא את כל השדות שנפתחו לו.

בקטלוג - היוצר ממלא את שם המוצר, מחיר ותמונה, ואם הוא רוצה (לא חובה) גם תיאור. הוא יכול ליצור כמה מוצרים שירצה.

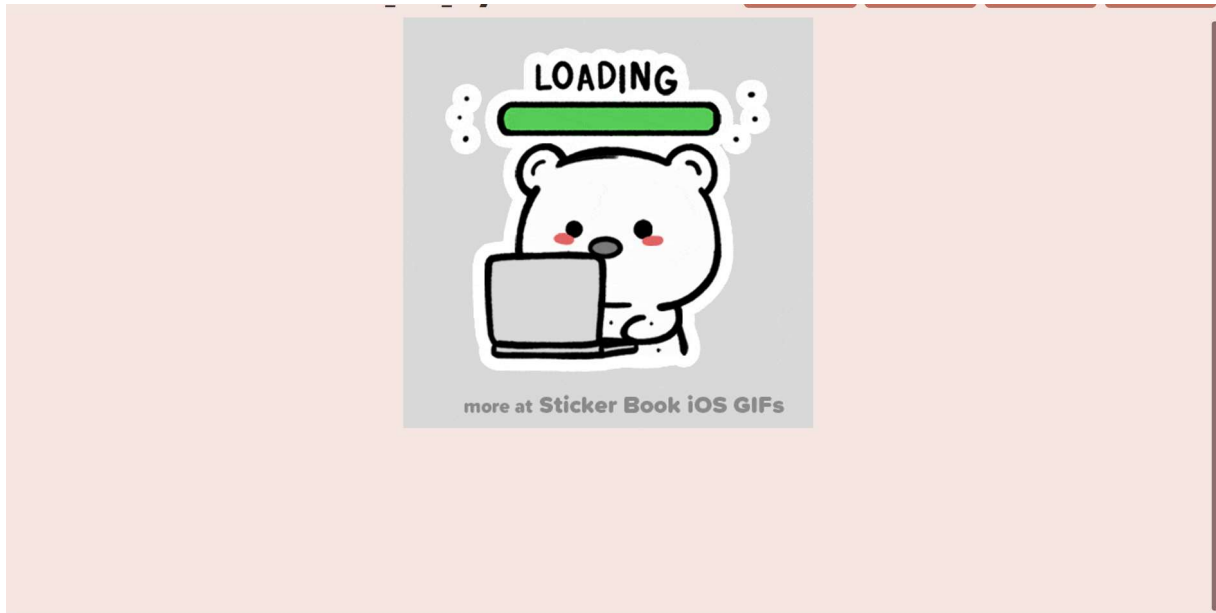
במערכת הפגישות – באפליקציה נוצרת מערכת פגישות אוטומטית, לכן כדי להנגיש אותה ליוצר, ולשימוש, יש פה פרטים שעליו למלא: אורך כל פגישה, אורך הפסקה, כל כמה פגישות יש הפסקה ומהי שעת ההתחלה והסיום של כל יום שלו.

ולבסוף, היוצר בוחר שם לאפליקציה שלו.

את כל הקלטים המערכת בודקת לפני הכנסתם לבסיס הנתונים, במקרה והם לא תקינים, אז היא תראה הודעה של אי תקינות וסיבה לכך, והיוצר יצטרך לתקן (כמו נניח שם אפליקציה שהוא לא ייחודי, או שהושאר שדה ריק שאמור להיות מלא).

5. מסך טעינה:

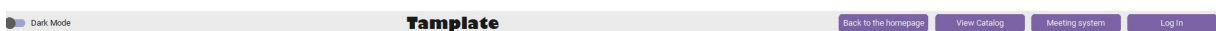
גיף טעינה חמוד שמופיע בכל תהליך שלוקח קצת זמן על מנת שהמשתמש יראה שהמערכת עדיין עובדת.



6. מסך סרגל כלים של האפליקציה:

למסך זה ניתן להגיע דרך שתי אופציות: מסך הנמצא כבר באפליקציה קיימת והוא נפתח לאחר לחיצה על האפליקציה הזו במסך הפתיחה, או לאחר סיום יצירת אפליקציה חדשה. בסרגל הכלים, ניתן לנווט בין הדפים השונים שיש באפליקציה (מעודכן לפי מה שיוצר האפליקציה בחר, זאת אומרת שאם הוא לא בחר בניח קטלוג, אז לא יהיה בסרגל כפתור של קטלוג). בנוסף, דרך הסרגל ניתן להתחבר לאפליקציה בעזרת כפתור ה- log in, וניתן לשנות את האפליקציה מ- Light mode ל- dark mode ולהפך.

באמצע הסרגל מוצג שם האפליקציה.



7. עמוד הבית:

אם קיים עמוד בית, אז הוא העמוד הראשון שיופיע בעת פתיחת המערכת. בעמוד זה ניתן רק לגלול ולראות את התכנים שהיוצר כתב והציג.



8. הקטלוג:

אם לא קיים עמוד בית באפליקציה אז זה העמוד הראשון שיופיע, אחרת ניתן לגשת אליו דרך סרגל הכלים. דרך הקטלוג ניתן לגלול ולהסתכל על המוצרים שיוצר האפליקציה מעוניין להציג.



9. מערכת הפגישות:

אם עמוד הבית והקטלוג לא קיימים באפליקציה, אז זה העמוד הראשון שיופיע, אחרת ניתן להיכנס אליו דרך סרגל הכלים.

במסך זה מופיעה טבלה של כפתורים, כאשר כל כפתור מייצג פגישה, והטבלה מתקדמת לפי הימים שעוברים. אם הפגישה תפוסה אז הכפתור יוחלף במחרוזת שמראה כי הפגישה תפוסה. על מנת לקבוע פגישה, המשתמש נדרש להתחבר וללחוץ על הפגישה שהוא מעוניין לקבוע. כאשר משתמש מחובר, הוא יכול לראות בתחתית הטבלה את הפגישות שלו (רק הרלוונטיות וממוין לפי תאריך), ואם הוא מעוניין למחוק את אחת הפגישות אז הוא יכול ללחוץ על כפתור הפח שנמצא ליד הפגישה.

הטבלה מתעדכנת מיד אחרי כל שינוי.

מערכת הפגישות כאשר המשתמש לא מחובר ולא נקבעו פגישות בכלל:

Book A Meeting With Us						
Monday 2025-05-19	Tuesday 2025-05-20	Wednesday 2025-05-21	Thursday 2025-05-22	Friday 2025-05-23	Saturday 2025-05-24	Sunday 2025-05-25
06:45-07:45	06:45-07:45	06:45-07:45	06:45-07:45	06:45-07:45	06:45-07:45	06:45-07:45
07:45-08:45	07:45-08:45	07:45-08:45	07:45-08:45	07:45-08:45	07:45-08:45	07:45-08:45
----	----	----	----	----	----	----
09:05-10:05	09:05-10:05	09:05-10:05	09:05-10:05	09:05-10:05	09:05-10:05	09:05-10:05
10:05-11:05	10:05-11:05	10:05-11:05	10:05-11:05	10:05-11:05	10:05-11:05	10:05-11:05
----	----	----	----	----	----	----
11:25-12:25	11:25-12:25	11:25-12:25	11:25-12:25	11:25-12:25	11:25-12:25	11:25-12:25
12:25-13:25	12:25-13:25	12:25-13:25	12:25-13:25	12:25-13:25	12:25-13:25	12:25-13:25
----	----	----	----	----	----	----
13:45-14:45	13:45-14:45	13:45-14:45	13:45-14:45	13:45-14:45	13:45-14:45	13:45-14:45
14:45-15:45	14:45-15:45	14:45-15:45	14:45-15:45	14:45-15:45	14:45-15:45	14:45-15:45
----	----	----	----	----	----	----
16:05-17:05	16:05-17:05	16:05-17:05	16:05-17:05	16:05-17:05	16:05-17:05	16:05-17:05
17:05-18:05	17:05-18:05	17:05-18:05	17:05-18:05	17:05-18:05	17:05-18:05	17:05-18:05
----	----	----	----	----	----	----

מערכת הפגישות לאחר התחברות וקביעת כמה פגישות:

Book A Meeting With Us						
Monday 2025-05-19	Tuesday 2025-05-20	Wednesday 2025-05-21	Thursday 2025-05-22	Friday 2025-05-23	Saturday 2025-05-24	Sunday 2025-05-25
06:45-07:45	06:45-07:45	06:45-07:45	06:45-07:45	06:45-07:45	06:45-07:45	06:45-07:45
07:45-08:45	07:45-08:45	07:45-08:45	07:45-08:45	07:45-08:45	07:45-08:45	07:45-08:45
----	----	----	----	----	----	----
09:05-10:05	09:05-10:05 (taken)	09:05-10:05	09:05-10:05	09:05-10:05	09:05-10:05	09:05-10:05
10:05-11:05	10:05-11:05	10:05-11:05	10:05-11:05	10:05-11:05	10:05-11:05	10:05-11:05
----	----	----	----	----	----	----
11:25-12:25	11:25-12:25	11:25-12:25	11:25-12:25	11:25-12:25	11:25-12:25	11:25-12:25
12:25-13:25	12:25-13:25	12:25-13:25	12:25-13:25	12:25-13:25	12:25-13:25	12:25-13:25
----	----	----	----	----	----	----
13:45-14:45	13:45-14:45	13:45-14:45	13:45-14:45 (taken)	13:45-14:45	13:45-14:45	13:45-14:45
14:45-15:45	14:45-15:45	14:45-15:45	14:45-15:45	14:45-15:45	14:45-15:45	14:45-15:45
----	----	----	----	----	----	----
16:05-17:05	16:05-17:05	16:05-17:05	16:05-17:05	16:05-17:05	16:05-17:05	16:05-17:05
17:05-18:05	17:05-18:05	17:05-18:05	17:05-18:05	17:05-18:05	17:05-18:05	17:05-18:05
----	----	----	----	----	----	----

My Meetings

Date & Time

(Tuesday
2025-05-20 09:05)

(Thursday
2025-05-22 13:45)

10. מסך התחברות:

אל מסך זה המשתמש מגיע כאשר הוא לוחץ על כפתור ההתחברות בסרגל הכלים, או כאשר הוא מנסה לקבוע פגישה בלי שהוא מחובר למערכת.

במסך זה יש למשתמש מקום למלא את שם המשתמש שלו והסיסמה שלו, ולשלוח. למקרה ששם המשתמש והסיסמה לא נכונים, אז תופיע הודעה, ואם הם נכונים, אז המסך ייסגר. בנוסף, אם המשתמש מעוניין

להתחבר למערכת או שהוא שכח סיסמה, אז הוא יכול לעשות זאת בעזרת הכפתורים שמופיעים במסך זה.



The screenshot shows a web browser window titled "Log In/Sign Up". The main heading is "Log In". Below the heading, there are two input fields: "Gmail:" and "Password:". The "Password:" field has a small purple star icon to its right. Below the input fields, there are two buttons: a purple "Log In" button and a yellow "Sing Up" button. At the bottom, there is a link that says "Forgot Password".

11. מסך הרשמה:

דרך מסך ההתחברות ניתן להגיע למסך ההרשמה, בו המשתמש רושם את פרטיו. המערכת בודקת את הפרטים בדיקות כמו תקינות של מחרוזת מספר טלפון או של תבנית אימייל ובודקת בנוסף שלא קיים כבר אימייל ומספר טלפון כמו שהמשתמש רשם עכשיו.



The screenshot shows a web browser window titled "Log In/Sign Up". The main heading is "Sing Up". Below the heading, there are four input fields: "Name:", "Gmail:", "Phone Number:", and "Password:". Below the input fields, there is a purple "Sing Up" button.

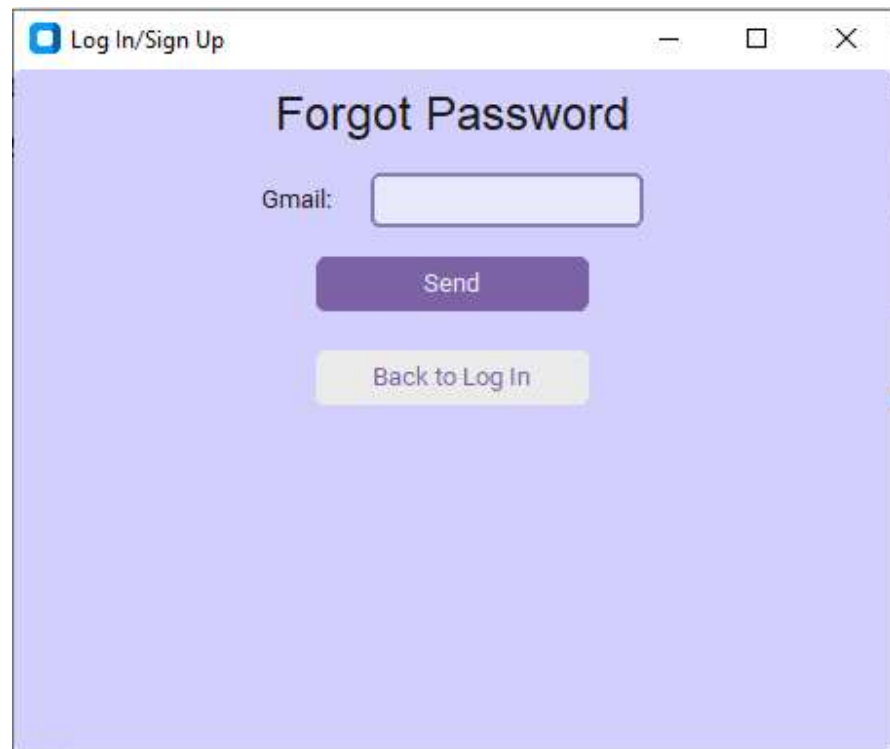
12. מסך אימות:

לאחר הירשמות לאפליקציה, המערכת שולחת למשתמש אימייל אימות המכיל קוד אותו המשתמש צריך להזין ולשלוח לאימות. במסך זה יש מקום למשתמש לרשום את הקוד שלו ולשלוח, ובנוסף אם קרה מצב והאימייל לא הגיע, אז המשתמש יכול ללחוץ על כפתור ה send again, או שהוא יכול לחזור להרשמה ולוודא שכל הפרטים שלו נכונים.

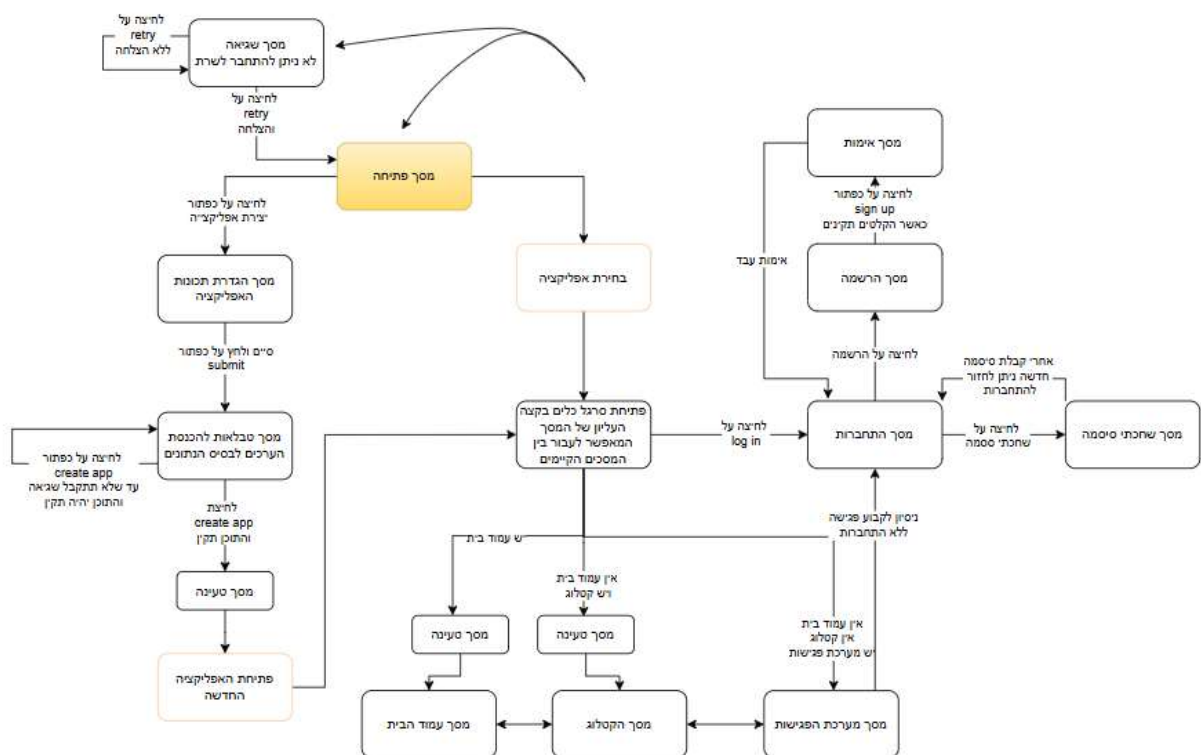
13. מסך שכחתי סיסמה:

אם המשתמש שכח את הסיסמה שלו, הוא יכול לעבור למסך שכחתי סיסמה, בו הוא יקבל סיסמה חדשה. המשתמש רושם בו את האימייל שלו והמערכת שולחת לאימייל הזה את הסיסמה החדשה. בעזרת הכפתור send, המשתמש יכול לשלוח בשנית סיסמה חדשה, אם במקרה האימייל לא הגיע אליו. כאשר האימייל מגיע אל המשתמש, הוא יכול ללחוץ על הכפתור של חזרה להחברות ולהתחבר. **במידה והמשתמש לא התחבר אף פעם לאפליקציה אבל מגיע למסך זה, המערכת אומרת לו לעבור למסך

ההתחברות.



תרשים מסכים:



תיאור מבני הנתונים

פירוט מבני הנתונים:

מסד הנתונים במערכת שלי הוא database.db מסוג SQL. כל התפעול מול מסד הנתונים מתבצע דרך שני הקבצים: my_database2 שמכיל בתוכו את יצירת הטבלאות, הכנסת הנתונים לטבלאות ובדיקת תקינות הנתונים בצורה מסודרת, והקובץ server.py שהוא מכיל את כל קוד השרת, כאשר רק לשרת יש את הגישה לבסיס הנתונים והוא בעזרת sqlite3 יכול לבצע שאילתות על בסיס הנתונים.

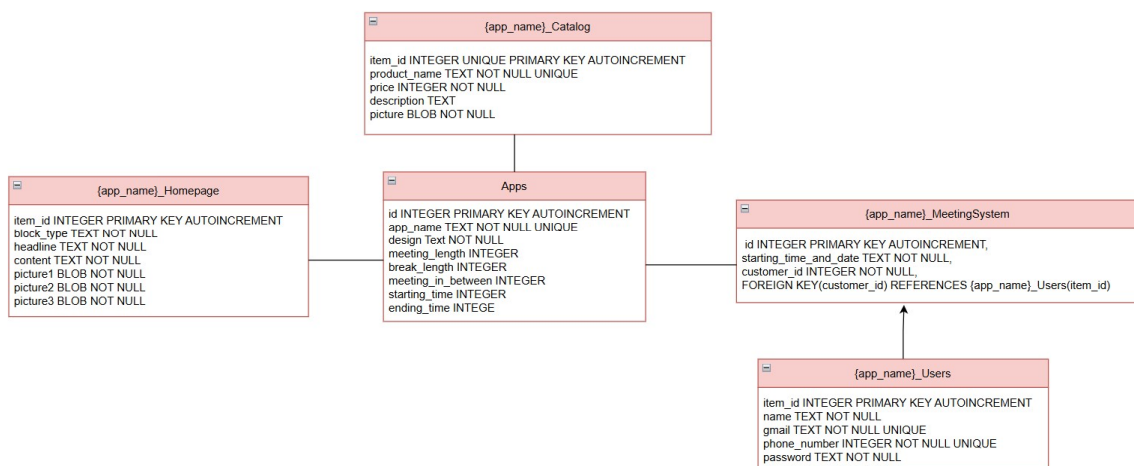
השרת משתמש בקובץ מקומי אחד בשבילו שהוא הקובץ prime2 שמכיל את כל המספרים הראשוניים של המיליון השני. בעזרת הקובץ הזה, השרת יכול ליצור לעצמו מפתחות הצפנה.

מהצד של המשתמש, הוא יכול להגיע לבסיס הנתונים רק דרך הלקוח שמתקשר עם השרת, client.py.

הקבצים המקומיים המשומשים במערכת הם:

- תמונות מוכנות מראש – תמונות המשמשות להצגת ממשק גרפי, אייקונים, או אלמנטים קבועים במערכת.
- קובץ מספרים ראשוניים להצפנה prime2 – קובץ שמכיל מספרים ראשוניים ומשמש ליצירת מפתחות הצפנה עבור RSA.
- קובצי JSON כל ה-designs - משמשים להגדרת תצוגות, ממשקי תקשורת ולניהול מידע מובנה.

פירוט מאגרי המידע של המערכת:



Apps – טבלה המאחסנת את שמות כל האפליקציות עם מידע הרלוונטי אליהן.

****** הטבלה היחידה שיש רק אחת ממנה, כל שאר הטבלאות משוכפלות כל פעם שאפליקציה חדשה נוצרת.

שם השדה	תיאור	טיפוס השדה	דוגמאות	מאפיינים
---------	-------	------------	---------	----------

PRIMARY KEY AUTOINCREMENT	1 2	INTEGER	מספר האפליקציה	Id
NOT NULL UNIQUE	"MY_APP" "1APP"	TEXT	שם האפליקציה	App_name
NOT NULL	"cherry" "coffee"	TEXT	שם העיצוב של האפליקציה	Design
-	60 45 "X" – אין מערכת פגישות	INTEGER	אם יש מערכת פגישות – אורך הפגישה בדקות	Meeting_length
-	20 15 "X" – אין מערכת פגישות	INTEGER	אם יש מערכת פגישות – אורך ההפסקה בדקות	Break_length
-	1 2 3 "X" – אין מערכת פגישות	INTEGER	אם יש מערכת פגישות – כמות הפגישות שיש בין כל הפסקה	Meeting_in_between
-	(9:30) 570 (9:00) 540 (10:15) 615 "X" – אין מערכת פגישות	INTEGER	אם יש מערכת פגישות – שעת התחלה, מומרת לדקות	Starting_time
-	(18:30) 1110 (22:00) 1320 (20:00) 1200 "X" – אין מערכת פגישות	INTEGER	אם יש מערכת פגישות – שעת סיום, מומרת לדקות	Ending_time

{app_name}_Catalog – טבלה המאחסנת את ערכי הקטלוג של אפליקציה מסוימת.

שם השדה	תיאור	טיפוס השדה	דוגמאות	מאפיינים
---------	-------	------------	---------	----------

UNIQUE PRIMARY KEY AUTOINCREMENT	1 2 3	INTEGER	מספר המוצר	Item_id
NOT NULL UNIQUE	"lamp" "cake"	TEXT	שם המוצר	Product_name
NOT NULL	99 19	INTEGER	מחיר המוצר	Price
-	"very pretty" "size ()"	TEXT	תיאור	description
NOT NULL	כל תמונה שהיא	BLOB	תמונה	picture

{app_name}_Homepage – טבלה המאחסנת את עמוד הבית של אפליקציה מסוימת.

שם השדה	תיאור	טיפוס השדה	דוגמאות	מאפיינים
Item_id	מספר ה"בלוק" של עמוד הבית	INTEGER	1 2	PRIMARY KEY AUTOINCREMENT
Block_type	סוג ה"בלוק"	TEXT	"only_content" "left_picture_and_content"	NOT NULL
Headline	כותרת הקטע	TEXT	"About us" "?What do we do"	NOT NULL
content	תוכן	TEXT	"to contact us use..." "א" – אם אין בבלוק זה תוכן	NOT NULL
Picture1	תמונה ראשונה	BLOB	כל תמונה "א" – אם אין בבלוק זה תמונה	NOT NULL
Picture2	תמונה שנייה	BLOB	כל תמונה "א" – אם אין בבלוק זה תמונה	NOT NULL
Picture3	תמונה שלישית	BLOB	כל תמונה "א" – אם אין בבלוק זה תמונה	NOT NULL

{app_name}_Users – טבלה המאחסנת את פרטי המשתמשים שהתחברו לאפליקציה.

שם השדה	תיאור	טיפוס השדה	דוגמאות	מאפיינים
Item_id	מספר היוזר	INTEGER	1	PRIMARY KEY
			2	AUTOINCREMENT
Name	שם היוזר	TEXT	"Tamar"	NOT NULL
Gmail	אימייל	TEXT	"gmaill@gmail.com"	NOT NULL
				UNIQUE
Phone_number	מספר הטלפון	INTEGER	0555458655	NOT NULL
				UNIQUE
password	הסיסמה של היוזר – מאוכסנת בצורה של HASH	TEXT	ba9731b32be7196a36"fcade21ed628704509da9a62"360ca12c08c8508296650c	NOT NULL

{app_name}_MeetingSystem – טבלה המאחסנת את כל הפגישות שנקבעו דרך האפליקציה.

שם השדה	תיאור	טיפוס השדה	דוגמאות	מאפיינים
Id	מספר הפגישה	INTEGER	1	PRIMARY KEY
			2	AUTOINCREMENT
starting_time_and_date	שעת ותאריך התחלה	TEXT	Saturday"2025-04-26"16:50Friday"2025-05-09"14:50	NOT NULL
customer_id	מספר המשתמש שהזמין את הפגישה	INTEGER	1	NOT NULL
			2	FOREIGN KEY(customer_id)
			3	REFERENCES {app_name}_Users(item_id)

סקירת חולשות והאיומים

שכבת האפליקציה:

- **SQL Injection**: מניעת התקפות בעזרת שאילתות מוכנות (Parameterized Queries), כפי שמיושם בקוד.
(cursor.execute(f"SELECT * FROM {app_name}_Users WHERE gmail = ? AND password = ?", (email, password)))
- **תהליך ה-Login**: אימות דו-שלבי בעת התחברות לאימות תקינות האימייל, והאפשרות להחלפת סיסמה במקרה הצורך, מה שמוסיף שכבת אבטחה נוספת.
- **MITM (Man-in-the-Middle Attack)**: השימוש בהצפנת RSA מבטיח שהנתונים המועברים בין הלקוח לשרת מוגנים מפני האזנות.
- **DDoS/DOS**: הגבלת מספר המשתמשים המתחברים מבטיחה שלא יהיו עודף של לקוחות על השרת.
- **שמירת סיסמאות ומידע רגיש**: כל המידע הרגיש במערכת עובר כבר מהשלב של הלקוח בצורה מוצפנת של hash.

שכבת התעבורה:

1. **פרוטוקול TCP - לחיצת יד משולשת**: בעקבות שימוש ב-TCP Socket הפרוטוקול יטפל אוטומטית בכך.

הצפנה:

- **RSA**: משמש להצפנת מידע רגיש ולהגנה על תקשורת מאובטחת.
- **Hash-256**: משמש לאחסון סיסמאות בצורה מוצפנת שלא ניתן לשחזר את הערך המקורי שלהן.

הפעולות מאחורי Hash-256:

256 – HASH היא פונקציה קריפטוגרפית שממירה קלט באורך משתנה למחרוזת קבועה של 256 ביטים. המתמטיקה שמאחוריו מבוססת על פעולות בינאריות, שינויים ביטים, וחישובים מודולריים כדי להבטיח אבטחה גבוהה ועמידות בפני התנגשויות.

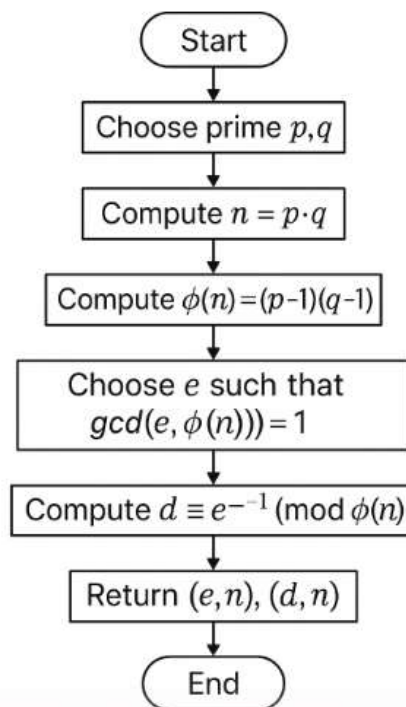
האלגוריתם מחלק את הקלט לבלוקים של 512 ביטים, ומעבד כל בלוק ב-64 סבבים של חישובים. בכל סבב, נעשה שימוש בפונקציות לוגיות כמו AND, XOR, OR וכן הזזות ביטיות כדי לערבב את הנתונים וליצור אפקט "מפולת" שבו שינוי קטן בקלט גורם לשינוי משמעותי בתוצאה. בנוסף, נעשה שימוש בקבועים שמקורם בשורשים השלישיים של המספרים הראשוניים הראשונים, מה שמוסיף אקראיות לחישוב.

הפעולות מאחורי RSA:

1. יצירת מפתחות:

- בוחרים שני מספרים ראשוניים גדולים: p ו- q .
- בודקים שהמספרים p ו- q ראשוניים בעזרת הפונקציה $\text{is_prime}(p/q)$.
- מחשבים את המכפלה שלהם: $n = p \cdot q$ הנקראת מודולוס והוא משמש גם להצפנה וגם לפיענוח.
- מחשבים את פונקציית אוילר: $\phi(n) = (p-1)(q-1)$. זהו מספר שמייצג כמה מספרים קטנים מ- n אינם מתחלקים בו.
- בוחרים מספר e כלשהו $1 < e < \phi(n)$ כאשר $\text{gcd}(e, \phi(n)) = 1$, אין לו מחלק משותף איתו, כלומר, $\text{gcd}(e, \phi(n)) = 1$.
- מחשבים את המפתח הפרטי d , שהוא ההופכי של e מודולו $\phi(n)$ - $d = e^{-1} \bmod \phi(n)$ בעזרת הפונקציה $\text{multiplicative_inverse}(e, \phi)$.

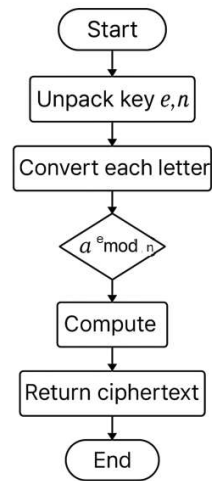
המפתח הציבורי (e, n) והמפתח הפרטי (d, n) .



2. הצפנה:

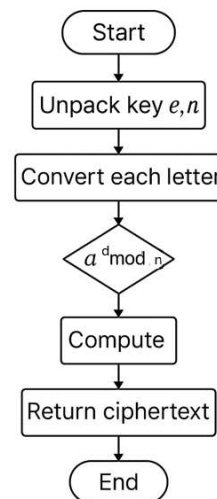
- המרת ההודעה לטקסט מוצפן – הופכים אותה לערכי ASCII.
- כל תו m בהודעה מועלה בחזקת e מודולו n כדי ליצור את הטקסט המוצפן c - $c = m^e \bmod n$

- התוצאה היא רשימת מספרים שמייצגת את ההודעה המוצפנת.



3. פענוח:

- כעת משתמשים במפתח הפרטי (d, n) על מנת לפענח את המסר.
- כל מספר בטקסט המוצפן מועלה בחזקת d מודולו n כדי לשחזר את המסר המקורי - $m = c^d \bmod n$.
- כל מספר מופענח בחזרה לתו באמצעות הפונקציה $\text{chr}()$ ומתקבלת ההודעה המקורית.

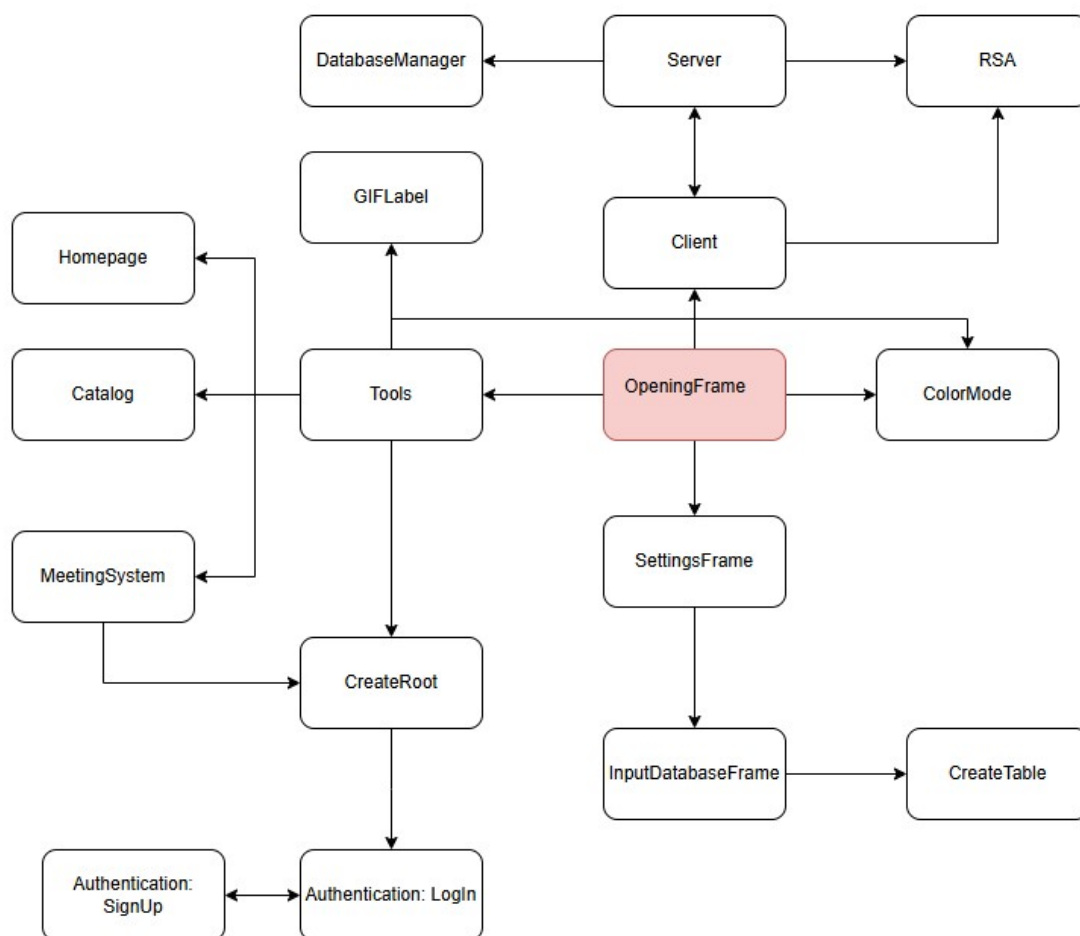


מימוש הפרויקט

סקירת מודולים/ מחלקות מיובאים

- **Custom tkinter** - משמש לבניית הממשק הגרפי של התוכנית.
- **Json** - מאפשר גישה לעיצובים גרפיים ושמירת נתונים בפורמט טקסט לשימוש ברשת.
- **Threading** – מאפשר הרצת תהליכים במקביל לשיפור יעילות הקוד.
- **Random** – יוצר מספרים אקראיים, בעיקר לצורכי הצפנה.
- **Sqlite3** – מנהל מסד נתונים ומאפשר ביצוע שאילתות.
- **Datetime** – מסייע בגישה לזמן הנוכחי ולניהול פגישות.
- **Partial מתוך functools** – יוצר פונקציות עם ערכים קבועים מראש, גם אם הם משתנים אחר כך.
- **Image מתוך PIL** – מאפשר עבודה נוחה עם תמונות בממשק הגרפי.
- **Socket** – אחראי על התקשורת בין הלקוח לשרת.
- **Base64** – ממיר נתונים בינאריים למחרוזות ולהפך, שימושי לשליחת תמונות ברשת.
- **Ast** – ממיר מחרוזות למודולים של פייתון מחדש.
- **Smtplib** – משמש לשליחת אימיילים.
- **EmailMessage מתוך email.message** – משמש לטיפול במבנה הודעות האימייל.
- **Hashlib** – מצפין מידע רגיש למחרוזת שלא ניתן לפענח.
- **String** – מסייעת ביצירת סיסמאות חזקות.
- **Re** – בודקת התאמת הנתונים למבנה הרגולרי שלו שהוגדר מראש.
- **io** – מאפשרת עבודה עם נתונים בינאריים כקובץ.
- **Time** – כדי לסנכרן תהליכים ולשמור על סדר בביצועם.
- **Font מתוך customtkinter** – מאפשר להשתמש באובייקט Font ליצירת גופנים מותאמים אישית בתוך ממשק הגרפי.

מודולים/מחלקות שפיתחתי



OpeningFrame ○

- **תפקיד ושימוש:** מחלקה היורשת מ CTkFrame ומייצגת את המסך הראשון שהמשתמש רואה, דרכו הוא יוכל לבחור להכין לעצמו אפליקציה חדשה או להיכנס לאפליקציה קיימת.

- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
Root	החלון עליו אובייקט נמצא.
Background_frame	החלון עליו אובייקט המחלקה נמצא שמאפשרת לו להיות Scrollable.
Client	אובייקט הלקוח.

- **פעולות במחלקה:**

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	החלון עליו האובייקט המחלקה פועל (root), את frameen שהאובייקט עליו (background_frame) ואת אובייקט הלקוח.	מאתחלת את כל תכונות המחלקה עם ערכי ברירת מחדל, ומזמנת את הפונקציה שמרכיבה את הממשק הגרפי שלה (create_frame).
Create_frame		יוצרת את כל הממשק הגרפי של אובייקט המחלקה.
Go_to_app	שם האפליקציה שנבחרה (var)	מעבירה את המשתמש לאפליקציה שבחר.
Go_to_vreator_frame		מעבירה את המשתמש ל-frame חדש בו הוא יוכל ליצור לעצמו אפליקציה משלו.

ColorMode ○

- **תפקיד ושימוש:** משמשת כדי לשנות את מוד הגרפיקה (dark mode/light mode). היא בעצם מתג שמשמש לאורך הקוד שלי.
- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
mode	המוד בו המסך נמצא
Mode_switch	מתג המשנה את המוד

- **פעולות במחלקה:**

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	מסך הרקע (background_frame)	מאתחלת את תכונות המחלקה.
Choose_mode	את המוד שנבחר (var)	משנה את מוד המסך ומעדכנת את התכונה self.mode.

SettingFrame ○

- **תפקיד ושימוש:** מחלקה היורשת CTKFrameM ומייצגת את מסך בחירת אופי ומראה האפליקציה.
- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
Root	החלון עליו אובייקט המחלקה יהיה
Client	אובייקט הלקוח
Background_frame	Scrollable frame עליה אובייקט המחלקה יהיה
Check_boxs_var	הערכים ההתחלתיים של ה check boxes שמייצגים את החלקים של אפליקציה שהיוצר רוצה שיכללו באפליקציה שלו.
Chosen_design	מערך המייצג את העיצובים שנבחרו לחלקי האפליקציה.
color	צבע ערכת הנושא ההתחלתי
Pictures	מערך דו מימדי המייצג את התמונות שיוצגו לפי עיצוב של כל חלק באפליקציה.
colors	מערך של כל הצבעים של ערכות נושא
pic	מערך דו מימדי של שתי תמונות (במצב dark ו light) של כל צבע ערכת נושא
Database_frame	אובייקט של CTKFrame המייצג את frameM הבא בתור שיגיע.

- **פעולות במחלקה:**

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	החלון עליו האובייקט המחלקה פועל (root), את frameM שהאובייקט עליו (background_frame) ואת אובייקט הלקוח.	מאתחלת את כל תכונות המחלקה עם ערכי ברירת מחדל, ומזמנת את הפונקציה שמרכיבה את הממשק הגרפי שלה (create_sttings_frame).
Create_settings_frame		יוצרת את כל הממשק הגרפי
Print_pic		מדפיסה את כל התמונות של העיצובים, כאשר העיצוב הוא 1

Choose_design	העיצוב שנבחר (design) והאינדקס של סוג העיצוב (לעמוד בית/קטלוג/מערכת פגישות) (index)	משנה את self.chossen_designs ומחליף את התמונה שהשתמש רואה של אותו העיצוב לעיצוב הנבחר
Choose_color	צבע ערכת הנושא הנבחר (var)	מעדכן את self.color ומעביר את הקוד לפעולה שתשנה את התמונות שהשתמש רואה שיתאימו לערכת הנושא שבחר.
Print_color_pic	מיקום ערכת הנושא הנבחרת יחסית למערך self.pic	מראה את התמונות של העיצוב לפי האינדקס במערך כך שיתאימו לבחירת המשתמש.
Get_database_table		לאחר בדיקה שהגדרות הגיוניות (נניח שהשתמש בחר לפחות עמוד אחד לאפליקציה שלו), הפעולה הזו מעבירה את הקוד לframe הבא.

InputDatabaseFrame ○

- **תפקיד ושימוש:** מחלקה היורשת מ - CTkFrame ומייצגת את מסך הטבלאות של הכנסת ערכי המשתמש למסד הנתונים. המחלקה הזו מעבירה את הנתונים אל השרת שבודק אצלו האם הם תקינים ומחזיר הודעה בהתאם, ואם הם תקינים הנתונים מוכנסים לבסיס הנתונים. את המסך הזה המשתמש לא יכול לעבור עד שהוא מכניס נתונים תקינים.

• תכונות המחלקה:

שם התכונה	תפקיד ושימוש
Settings_frame	תכונה המייצגת את האובייקט של המחלקה הקודמת, דרכו ניתן לגשת לבחירות העיצובים שהמשתמש ביצע במסך הקודם.
client	אובייקט הלקוח
Next_row	תכונה המייצגת את השורה הבאה בתור. מתחיל מ - 0.

מערך דו מימדי המכיל את טבלת הערכים שהמשתמש הזין של עמוד הבית.	Homepage_values
מערך דו מימדי המכיל את טבלת הערכים שהמשתמש הזין של הקטלוג.	Catalog_values
מערך המכיל את הנתונים של מערכת הפגישות שהמשתמש הזין.	Meeting_system_values
CTkFrame שמשומשת כדי להראות את הכשל בהכנסת הנתונים למסד הנתונים (כמו נניח שהמשתמש השאיר חלק ריק שמוגדר כ - not null).	input_update_frame
Ctk_entry שבו השתמש מזין את שם האפליקציה שלו.	Name_entry

• פעולות במחלקה:

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	Settings_frame my_client	מאתחלת את כל ערכי התכונות שלה עם ערכי ברירת מחדל.
Create_tables		בודק איזה חלקים מהאפליקציה סומנו במסך הקודם (בעזרת self.settings_frame.check_boxs_var ומנתב את הקוד כך שיצור את הטבלאות המתאימות. בנוסף הוא יוצר את כפתור ה-Create App.
Homepage_table		יוצר את הטבלה והנתונים לגבי עמוד הבית.
Catalog_table		יוצר את הטבלה והנתונים לגבי הקטלוג.
Meeting_system_table		בגלל שהנתונים של מערכת הפגישות לא נכנסים בצורה של טבלה כמו האחרים, הפעולה הזו יוצרת את הממשק הגרפי של הכנסת נתוני עמוד הבית.

מקבלת אובייקט המייצג איזה option menu המשתמש שינה (i) ומקבלת את הזמן אליו שינה המשתמש (time)	Change_hours	מעדכנת את המערך self.meeting_system_values כדי שיתאים לערך החדש.
מקבלת אובייקט המייצג איזה option menu המשתמש שינה (i) ומקבלת את הזמן אליו שינה המשתמש (time)	Change_minuts	מעדכנת את המערך self.meeting_system_values כדי שיתאים לערך החדש.
	Insert_arr_to_database	היא לוקחת את כל הערכים שהתמלאו בטבלאות ושולחת אותם אל השרת ומחכה להודעה ממנו. אם הכל בסדר אז הפעולה תמשיך הלאה ותפתח את האפליקציה שנוצרה ואם לא אז הפעולה תציג את הבעיה למשתמש על מנת שיתקן.

CreateTable ○

- **תפקיד ושימוש:** יוצר טבלה לפי הדרישות של העמודות (נניח אם צריך להוסיף תמונות או פסקאות ולא רק ערכים קטנים).
- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
Next_row	משתנה המייצג את השורה הבאה בתור.
columns	מספר העמודות של הטבלה.
Picture_places	מערך המייצג את המיקומים של העמודות שהם של תמונות.
Text_box_places	מערך המייצג את המיקומים של העמודות שבהם יש text box.
Var_arr	מערך בסיס שיכיל בתוכו את הערכים של הטבלאות שנוצרות (כמו homepage_values)
headlines	מערך של כותרות התורים של הטבלה.

Plus_button	CTkButton המייצג פלוס על מנת ליצור שורה נוספת בטבלה.
arr	מערך חד מימדי המכיל שורה אחת של הטבלה. לאחר שהשורה הזו נבנית, המערך מתווסף למערך הדו מימדי (var_arr).
Coose_block	CTkOprionmenu שמכיל את סוגי ה"בלוקים" של עמוד הבית שהמשתמש יכול לבחור מהם.

• פעולות במחלקה:

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	Backgroundf_frame varr_arr headlines	מאתחלת את כל ערכי התכונות ומעבירה את המחלקה אל פעולות שיצרו את הממשק הגרפי שלה.
Print_column_headlines		מציגה בממשק הגרפי את כל כותרות הטבלה, ומעבירה את התוכנה לפעולה של יצירת שורה.
Create_new_row		יוצרת שורה חדשה בתוך self.arr ושולחת את התוכנה לפעולת הדפסת השורה.
Print_row		מציגה בממשק הגרפי את השורה שנוצרה קודם לכן ומוסיפה את השורה הזו למערך הדו מימדי של כל הטבלה. בנוסף היא מוסיפה כפתור פלוס על מנת שיהיה ניתן להוסיף עוד שורות לטבלה.
blocks	מקבל את השורה בו ה-option menu הזה נמצא ואת הערך הנבחר.	לפי סוג הבלוק, הפעולה מכניסה ערכים לself.arr

ומציגה את הערך נבחר בlabel במקום את option menu.		
הפעולה גורמת לסייר קבצים של המחשב להיפתח, דרכו המשתמש יוכל לבחור קבצים של תמונות בלבד, ולבסוף הפעולה מחליפה את הכפתור בשם הקובץ שנבחר.	מקבל את המערך בו הכפתור שמזמן את הפעולה הזו נמצא (arr), את השורה הבאה שיש בזמן זה (self.next_row) ואת מספר העמודה שהכפתור נמצא בה.	Select_file

DatabaseManager ○

- **תפקיד ושימוש:** מחלקה יוצרת את מסד הנתונים, ומקצה פעולות שאפשר להפעיל עליו.
- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
Db_name	שם מסד הנתונים.

- **פעולות במחלקה:**

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	Db_name	מאתחלת את שם מסד הנתונים וקוראת לפונקציה הראשונה שיוצרת אותה (create_main_tables).
Create_main_tables	App_name – שם האפליקציה עליה עובדים.	יוצר את טבלאות בסיסי הנתונים על מנת שיתאימו לאפליקציה החדשה.
Insert_data	App_name – שם האפליקציה עליה מכניסים מידע. table_name – שם הטבלה אליה נכניס נתונים חדשים. new_data – המידע החדש אותו מכניסים לטבלה.	מכניסה לטבלה שורה חדשה.

מוחקת את כל הערכים של האפליקציה המתקבלת, זאת אומרת את כל הטבלאות שנוצרו לאפליקציה הזו.	App_name	Clear_database
מקבל את שם האפליקציה וטבלה אליה רוצים להכניס מידע חדש. הפעולה מבצעת הרבה מאוד בדיקות על מנת לבדוק שהתוכן תקין. במידה והוא תקין אז הפעולה מעבירה את data אל הפעולה insert_data ואז המידע החדש מוכנס לבסיס הנתונים. הפעולה תמיד מחזירה מחרוזת שאומרת על הצלחת/כישלון הכנסת הנתונים ולמה.	App_name table_name New_data	Validate_and_insert_data

Tools ○

- **תפקיד ושימוש:** המחלקה יורשת מ CTKFrame ומייצגת את סרגל הכלים שיש בחלק העליון של האפליקציה, דרכו ניתן לנתב בין חלקים שונים באפליקציה. בנוסף, המחלקה הזו היא המחלקה הראשונה שנוצרת מתוך האפליקציה והיא זו שפותחת את האפליקציה ומציגה אותה בצורה תקינה.
- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
Name	שם האפליקציה.
Root	החלון עליו אובייקט נמצא.
client	אובייקט הלקוח.
design	שם העיצוב של האפליקציה.
j	אובייקט מסוג Json שעוזר לי לגשת לקובץ Json של העיצוב.
Background_frame	CTKScrollableFrame עליה יהיה את תוכן האפליקציה.

user	מספר היוזר שמחובר (אם לא מחובר אז מחרוזת ריקה).
column	עמודות בטבלת grid.

• פעולות במחלקה:

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	Name Root client	מתחלת את התכונות ושולחת את התוכנה לפעולה שיוצאת את הממשק הגרפי.
Create_tools		יוצר את כל סרגל הכלים ומעביר את התוכנה אל הפונקציה הרלוונטית שתיצור את הממשק הגרפי של העמוד הראשון של האפליקציה.
Create_homepage		שולח thread שיצור את עמוד הבית.
Create_catalog		שולח thread שיצור את הקטלוג.
Create_meeting_system		שולח thread שיצור את מערכת הפגישות.
Log_out		כאשר לוחצים על כפתור ה log out בסרגל הכלים מגיעים לפונקציה זו. הפונקציה מאפסת את המשתנה self.user והופכת את הכפתור בברגל הכלים להיות בחזרה log in.
Log_in		כאשר לוחצים על הכפתור log_in מגיעים לפעולה זו ששולחת את המשתמש לחלון של התחברות.
After_log_in	User_name meeting_row	מעדכנת את המשתנה self.user על מנת שיתאים

ליוזר החדש, משנה את הכפתור בסרגל הכלים ל log out, ולמקרה שהמשתמש הגיע לפעולת ההתחברות דרך מערכת הפגישות, אז הפעולה מחזירה אותו בחזרה למערכת הפגישות ומעדכנת אותה לפי הפגישה שהוא קבע.	meeting_column meeting_time meeting_act	
--	--	--

GIFLabel ○

- **תפקיד ושימוש:** מחלקה היורשת מ CTKLabel ויוצרת גיף טעינה בזמן שהקוד תוען את המערכת.
- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
_gif_image	מכיל את הגיף לאחר שנפתח.
duration_	מכילה את זמן התצוגה.
_frames	מערך שמכיל את הפריימים של הגיף.

- **פעולות במחלקה:**

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	Master imag_path **kwargs	מתחלת את התכונות של המחלקה.
animate_		מעבירה בין הפריימים של הגיף ויוצרת כך את האנימציה.

HomepageDesign ○

- **תפקיד ושימוש:** יוצרת את הממשק הגרפי של עמוד הבית.
- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
Name	שם האפליקציה.

Background_frame	Frame הרקע עליה יוצג הקטלוג.
client	אובייקט הלקוח.
rows	מערך דו מימדי של טבלת הקטלוג בבסיס הנתונים.
Gif	מכיל את גיף הטעינה שנמצא לפני שתוכן עמוד הבית נטען.
Design_num	מספר העיצוב.

• פעולות במחלקה:

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	Background_frame name my_client design_num	מאתחלת את התכונות של המחלקה וקוראת לפעולה שיוצרת את הממשק הגרפי.
Create_frames		בעזרת לולאה, הפעולה עוברת על כל השורות של עמוד הבית ומנתבת אותן לפי ה"בלוק" המתאים להן. בנוסף לאחר הצבת הבלוק הראשון, הפונקציה סוגרת את גיף הטעינה.
Create_left_picture_and_content	מספר השורה בrows (i)	יוצרת את הממשק הגרפי לפי עיצוב הבלוק.
Create_right_picture_and_content	מספר השורה בrows (i)	יוצרת את הממשק הגרפי לפי עיצוב הבלוק.
Create_only_content	מספר השורה בrows (i)	יוצרת את הממשק הגרפי לפי עיצוב הבלוק.
Create_pictures_only	מספר השורה בrows (i)	יוצרת את הממשק הגרפי לפי עיצוב הבלוק.
*wrap_label_text_in_frame – לא פעולה מתוך המחלקה אבל היא כן משתמשת בה	Frame – המסך עליו עובדים label – הטקסט אותו מסדרים column_index	הפעולה מתאימה את הטקסט למקום שיש לו, זאת אומרת שהיא מסדרת אותו

כך שירד שורה אם השורה ארוכה מדי.		
הפעולה משנה את גודל התמונה המתקבלת כך שתתאים לרוחב ספציפי שמתקבל בצורה שהתמונה לא תתעוות.	Picture size	*change_picture_to_size – לא פעולה מתוך המחלקה אבל היא כן משתמשת בה.

○ CatalogDesign

- **תפקיד ושימוש:** יוצרת את הממשק הגרפי של הקטלוג.
- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
Name	שם האפליקציה.
Background_frame	Frame הרקע עליה יוצג הקטלוג.
client	אובייקט הלקוח.
rows	מערך דו מימדי של טבלת הקטלוג בבסיס הנתונים.
Design_num	מספר העיצוב.
Gif	מכיל את גיף הטעינה שנמצא לפני טעינת הקטלוג.

- **פעולות במחלקה:**

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	Background_frame name my_client design_num	מאתחלת את התכונות של המחלקה וקוראת לפעולה שיוצרת את הממשק הגרפי.
Create_frames		בעזרת לולאה, הפעולה עוברת שורה שורה על המערך הדו מימדי self.rows ולכל שורה מעביר אותה לפעולה הבאה על מנת שתופיע בממשק הגרפי. סוגר את גיף הטעינה.

יצרת frame לכל מוצר ומציגה אותו.	מספר השורה של הנתונים (i).	Create_product_frame
הפעולה מתאימה את הטקסט למקום שיש לו, זאת אומרת שהיא מסדרת אותו כך שירד שורה אם השורה ארוכה מדי.	Frame – המסך עליו עובדים label – הטקסט אותו מסדרים column_index	*wrap_label_text_in_frame – לא פעולה מתוך המחלקה אבל היא כן משתמשת בה
הפעולה משנה את גודל התמונה המתקבלת כך שתתאים לרוחב ספציפי המוגדר מראש ולא תתעוות.	Picture	*change_picture_to_size – לא פעולה מתוך המחלקה אבל היא כן משתמשת בה.

○ MeetingSystemDesign

- **תפקיד ושימוש:** מחלקה היורשת מ-CTkFrame ומייצגת את כל הממשק הגרפי של מערכת הפגישות ומאפשר לקבוע ולמחוק פגישות.
- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
Name	שם האפליקציה.
Background_frame	Frame הרקע עליה יוצג הקטלוג.
User	מספר המשתמש שמחובר.
client	אובייקט הלקוח.
Design	שם ערכת הנושא.
root	החלון עליו מערכת הפגישות נמצאת.
After_log_in	משתנה השומר את הפעולה הזו (שמקורה ב-Tools).
Input_arr	המערך המייצג את הנתונים של מערכת הפגישות.
Meetings_arr	מערך המכיל את כל הפגישות שניתן לקבוע וניראות על הממשק הגרפי. אם הפגישה לא נקבעה-היא מיוצגת ככפתור, ואם היא נקבעה-היא מיוצגת כתווית שרשום עליה taken.
frame	מסך המראה את הפגישות של משתמש, מופיע רק כשהמשתמש מחובר.

• פעולות במחלקה:

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	Background_frame user input_arr design root after_log_in name my_client	מאתחלת את כל התכונות, שמה את האובייקט על המסך וקוראת לפונקציית שתבנה את הממשק הגרפי (create_frame).
Create_frame		בעזרת חישובים וההגדרות שמשמש עשה, הפונקציה מבצעת חישובים ויוצרת טבלה של פגישות, שמעודכנת לפי התאריך. בטבלה כל פגישה מיוצגת על ידי כפתור ברגע שהיא פנויה ועל ידי תווית כאשר היא תפוסה.
Book_a_meeting	Row – השורה בה הפגישה נמצאת. column – העמודה בה הפגישה נמצאת. Meeting_time New_user – היוזר שקובע את הפגישה.	אם המשתמש לא מחובר כרגע, אה הפעולה תביא אותו למסך ההתחברות מכיוון שבשביל לקבוע פגישה חייבים להיות מחוברים. הפעולה שולחת לשרת את הבקשה לקבוע את הפגישה, השרת שם את הפגישה בטבלה meeting_system ושולח למשתמש אימייל על כך. לאחר מכן הפונקציית הזו קוראת לפונקציה create_frame עוד

פעם, על מנת שהטבלה תתעדכן.		
רק כאשר המשתמש מחובר, הפעולה הזאת נקראת. היא יוצרת frame חדש מתחת למערכת הפגישות בו הלקוח יכול לראות את הפגישות שלו. הוא יראה רק את הפגישות הרלוונטיות לו (לא כאלו שעברו מבחינת זמן) ותראה אותן לפי סדר של קרוב לרחוק. בנוסף, לכל פגישה יש כפתור ליד שימחק אותה.		My_meeting
הפונקציה מוחקת את הפגישה שנוצרה על ידי היוזר הזה בשעה הזו. היא שולחת לשרת בקשה לכך, השרת מוחק את הפגישה מבסיס הנתונים ושולח למשתמש אימייל על כך.	User time	Delete_meeting

CreateRoot ○

- **תפקיד ושימוש:** יוצרת מסך חדש top level לאחד הנוכחי בו יוצג תהליך ההתחברות.

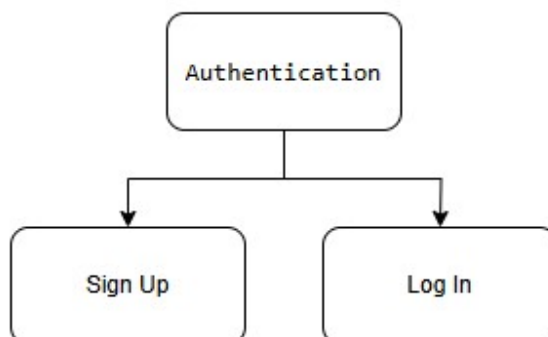
- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
name	שם האפליקציה.
Design	שם ערכת הנושא.
Root	המסך עליו יהיה כל התהליך.
After_log_in	הפעולה שתגיע מיד אחרי שתהליך הזה יגמר, במסך הראשי.
Meeting_row	למקרה שההתחברות קוראת בזמן קביעת פגישה - מספר השורה של הפגישה.

Meeting_column	למקרה שההתחברות קוראת בזמן קביעת פגישה - מספר העמודה של הפגישה.
Meeting_time	למקרה שההתחברות קוראת בזמן קביעת פגישה - זמן הפגישה.
Meeting_act	למקרה שההתחברות קוראת בזמן קביעת פגישה - פעולת הפגישה שתבוא אחרי.
Result	תוצאת ההתחברות-מספר הלקוח המחובר.
Client	אובייקט הלקוח.

• פעולות במחלקה:

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	Design Last_root After_log_in Meeting_row Meeting_column Meeting_time Meeting_act Name client	מאתחלת את כל התכונות, יוצרת חלון חדש ומעבירה את הקוד לפעולה הבאה.
Initialize_frame		יוצרת אובייקט של המחלקה Login ושמה אותו על המסך.
Set_result	Result	מעדכנת את תוצאת הפעולה.
Run		מריצה את ה mainloop של המסך הזה.



Authentication ○

- **תפקיד ושימוש:** יורשת מ־CTkFrame ומשמשת כמחלקת האב של שתי המחלקות הבאות. היא מכילה בתוכה תכונות ופעולות המשותפות לשתי המחלקות.
- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
Name	שם האפליקציה.
Root	החלון של ההתחברות.
Design	שם ערכת הנושא.
client	אובייקט הלקוח.
Callback	מייצג את הפעולה שאליה הפונקציה תלך כשפעולות ההתחברות יגמרו.
Error_label	תווית שיהיו בה את ההחזרות של הקוד למקרה שהכנסת הנתונים של המשתמש לא תהיה תקינה.
Password_entry	CTkEntry שאליה המשתמש צריך להכניס את הסיסמה שלו.
Gmail_entry	CTkEntry שאליה המשתמש צריך להכניס את האימייל שלו.

- **פעולות במחלקה:**

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	Root Design Name My_client	מאתחלת את כל התכונות.
Go_to_sign_up	Frame	עוברת לframe של sign up.
Go_to_log_in	Last_frame	עוברת לframe של login.

○ Login

○ **תפקיד ושימוש:** מחלקה היורשת מ Authentication ומייצגת את מסך ההתחברות.

• **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
Passwars_shown	משתנה בוליאני שאומר האם אפשר לראות את הסיסמה או שהיא מוסדרת בעזת החלפתה בממשק הגרפי ב "*".
Result	תוצאת ההתחברות – מספר המשתמש.

• **פעולות במחלקה:**

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	Root Design Name My_client	מאתחלת את כל התכונות (גם אלו שבמחלקת האב) ומעבירה את התוכנה לפונקציה שיוצרת את הממשק הגרפי.
Create_frame		יוצרת את הממשק הגרפי של ההתחברות.
Toggle_passwars_visibilit		מחליפה את המוד שהסיסמה נראית בממשק הגרפי. בין "*" לבין הסיסמה עצמה.
Forgot_password		הפעולה מעבירה את השתמש למסך חדש בו הוא יכול לרשום

את האימייל שלו והשרת ישלח לו שם סיסמה חדשה.		
הפעולה בודקת שהקלט או תקין (נניח שאין מקום ריק או שאין אימייל לא חוקי), ומעבירה לשרת לבדיקה האם הפרטים נכונים. אם כן אז הפעולה עוברת לפעולה set_result של המחלקה CreateRoot וסוגרת את החלון הזה. אם לא אז תופיע הודעה מתאימה על המסך.		Login_action
הפעולה נקראת דרך הפעולה forgot_password והיא בודקת שהאימייל שהמשתמש הזין הוא תקין ומבקשת מהשרת לשלוח אימייל עם סיסמה חדשה.	Frame Gmail	Send_new_password

○ SignUp

- **תפקיד ושימוש:** מחלקה היורשת מ Authentication ומייצגת את מסך ההרשמה.
- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
Name_entry	CTkEntry שיכל את השם של המשתמש.
Phone_number_entry	CTkEntry שיכל את מספר הטלפון של המשתמש.

- **פעולות במחלקה:**

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	Root Design	מאתחלת את כל התכונות (גם אלו שבמחלקת האב) ומעבירה

את התוכנה לפונקציה שיוצרת את הממשק הגרפי.	Name My_client	
יוצרת את הממשק הגרפי של ההרשמה.		Create_frame
הפעולה בודקת שכל הנתונים של המשתמש תואמים את הכללים שלהם הנמצאים בביטויים הרגולריים. אם לא אז מופיעה הודעה על כך ואם כן אז הנתונים נשלחים לשרת, שם הוא בודק מול מסד הנתונים את נתוני המשתמש ומחזיר תשובה על כך. אם לפי השרת הנתונים תקינים אז המשתמש יעבור למסך של אימות אימייל, ואם לא תופיעה הודעה על כך.		Sign_up_action
הפעולה שולחת לשרת את קוד האימות של הלקוח. אם השרת מחזיר שהוא תקין אז הפעולה מעבירה את הלקוח למסך ההתחברות, ואם השרת מחזיר שהקוד לא תקין אז מופיעה הודעה בהתאם.	Entry – מכילה את קוד האימות. Frame User_vars – כל הפרטים שהזין במסך הקודם.	Check_confirmation

RSA ○

- **תפקיד ושימוש:** עושה את כל מה שקשור להצפנה.

- **פעולות המודול:**

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
-----------	---------------------------------	---

gcd	מקבלת שני מספרים שלמים (a,b)	הפונקציה מחזירה את המחלק המשותף הגדול ביותר של המספרים.
multiplicative_inverse	מקבלת שני מספרים שלמים e ו-phi שבהם e צריך להיות זר לphi.	הפונקציה מחזירה את ההופכי הכפלי של e מודולו phi .
is_prime	מקבלת מספר.	מבצע מספר בדיקות ומחזיר האם המספר הזה ראשוני.
generate_keypair	מקבלת שני מספרים ראשוניים (pi, qi)	הפונקציה מחזירה זוג מפתחות לשיטת ההצפנה RSA.
encrypt	הפונקציה מקבלת מפתח ציבורי pk ומחרוזת plaintext	הפונקציה מחזירה רשימה של מספרים שלמים המייצגים את הטקסט מוצפן.
decrypt	הפונקציה מקבלת מפתח פרטי pk ורשימה של מספרים ciphertext המייצגים טקסט מוצפן.	הפונקציה מחזירה את הטקסט הגלוי לאחר פענוח.

Client ○

- **תפקיד ושימוש:** מחלקה המייצגת לקוח.
- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
IP_SERVER	שמירת ה- IP של השרת.
SERVER_PROT	הפורט של השרת.
MAX_MSG_LENGTH	האורך המקסימלי של הודעה.
Root	החלון עליו הממשק הגרפי רץ.
Retry_act	פעולת ה- retry אליה הלקוח יעביר את המשתמש אם יש בעיה עם השרת.
Lock	מנעול שימנע מהודעות שהלקוח שולח/מקבל להתערבב.
Client_socket	אובייקט מסוג socket שמייצג את הלקוח.

Status	משתנה שמעיד האם החיבור של השרת עם הלקוח עבר בהצלחה.
Keys	מפתחות ההצפנה של הלקוח.
Server_keys	המפתח הציבורי של השרת.

• פעולות במחלקה:

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	מקבלת את החלון (root) ואת הפעולה retry retry_act()	מגדירה את כל תכונות המחלקה, בודקת האם החיבור עם השרת עבר בהצלחה (בעזרת התכונה self.status), ואם כן אז הוא יוצר לעצמו מפתחות הצפנה וקורא לפונקציה שגורמת להחלפת מפתחות.
Send_sata	מקבלת את ההודעה המלאה שצריך לשלוח לשרת	מצפינה ושולחת את ההודעה לשרת. בנוסף בודקת שהשרת תקין בעזרת try/except. אם הוא לא תקין אז אם כרגע רץ ה-thread הלא ראשי אז היא קוטעת את תפקידו. והיא מעבירה את הthread הראשי למסך retry.
Recv_message	מקבלת ערך דיפולטי להחזרה במקרה שהשרת לא תקין.	מחכה להודעה מהשרת ומפענחת אותה. בנוסף בודקת שהשרת תקין בעזרת try/except. אם הוא לא תקין אז אם כרגע רץ ה-thread הלא ראשי אז היא קוטעת את תפקידו. והיא מעבירה את ה-thread הראשי למסך retry ומחזירה לתוכנית ערך דיפולטי.
recv_packets_and_change_to_original	מקבלת את שם הטבלה ששייכים	עושה את כל הקבלה של הפקטות מהשרת ומפענחת אותם בצורה מסודרת. גם היא עובדת בצורה בטוחה ובודקת האם השרת תקין ומתנהגת בהתאם.

אליה הערכים.		
שולחת הודעה ללקוח, בעזרת send_message לפעמים מחכה לתשובה ולפעמים מחזירה למשתמש ערכים בעזרת recv_message/ recv_packets_and_change_to_original	מקבלת את הערכים שנדרשים להישלח ללקוח	כל פעולות שליחת ההודעות – פורטו מאוד לעומק בחלק "תיאור פרוטוקול התקשורת".
משנה את רזולוציה התמונה כך שהיא תהיה קלה יותר ומחזירה את התמונה לאחר השינויים.	מקבלת את התמונה בצורה בינארית	**זו לא פעולה במחלקה אבל היא כן בשימוש פה - reduce_image_binary

Server ○

- **תפקיד ושימוש:** מייצג את השרת.
- **תכונות המחלקה:**

שם התכונה	תפקיד ושימוש
SERVER_IP	ה-IP של השרת.
SERVER_PORT	הפורט של השרת.
MAX_MSG_LENGTH	האורך המקסימלי של הודעה.
Max_clients	מספר הלקוחות המקסימלי שהלקוח יכול לקבל (למניעת דידוס).
Codes	מילון המכיל לקוחות ואת קוד האימות שלהם.

- **פעולות במחלקה:**

שם הפעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__		מאתחלת את התכונות של השרת.
Server		יוצא אובייקט מהמחלקה socket של שרת ופועל בלולאה אינסופית שעושה את כל תפקוד השרת.

send_packets	מקבלת את מספר הפקטות שישלחו, את הסוקט שאליו שולחים ואת המידע שישלח.	הפעולה שולחת ללקוח הודעות בצורת פקטות. *היא קיימת על מנת לחלק את השליחות הגדולות לפקטות כשי שיוכלו להתקיים במקביל.
כל פעולות שליחת וקבלת הודעות - פורטו מאוד לעומק בחלק "תיאור פרוטוקול התקשורת"	מקבל ערכים מהלקוח.	עושה את הפעולה שהלקוח ביקש ואולי מחזיר ללקוח תשובה או ערכים.

קטעי קוד האלגוריתמים המרכזיים

• העברת הודעות לא רגילות בתקשורת:

בקטעי קוד האלו באה לידי ביטוי יכולת העברה של הודעות לא רגילות בתקשורת. בשלב הקודם אנחנו סקרנו את האופציות השונות, מהן נבחר השימוש ב - json, ב - IO וב - base64. קוד זה מראה בדיוק איך השתמשתי בכל אחת מהפונקציות הללו על מנת להעביר את ההודעות הללו בתקשורת.

קבלת הודעות מצד ה - server, יכולות להיות בפקטות או לא:


```

try:
    data = current_socket.recv(self.MAX_MSG_LENGTH).decode()
except ConnectionResetError:
    data = ""
if data == "":
    client_sockets.remove(current_socket)
    current_socket.close()
else:
    if not data.startswith("GET_SERVER_PUBLIC_KEY"):
        data = data.split(",")
        data = [int(n) for n in data]
        data = RSA.decrypt(client_and_keys[current_socket][1], data)

# פיצול המחרוזת לרשימה
data_parts = data.split('@')

# בדוקת אורך הרשימה
if len(data_parts) < 1:
    print("Error: Invalid data format. At least a command and app_name are required.")
    continue

if data_parts[0].isdigit(): # בדוקת העברה בפקטות
    content = ""
    for i in range(int(data_parts[0]) + 1):
        content = content + current_socket.recv(self.MAX_MSG_LENGTH).decode()
    content = content.split(",")
    content = [int(n) for n in content]
    result = RSA.decrypt(client_and_keys[current_socket][1], content)

```

דוגמה של החזרת הודעת ה - string ליצירתה המקורית בצד servern (שימוש גם ב - Json וגם ב - base64):

```

6 usages (5 dynamic) TamarHoffmann
def insert_into_table(self, app_name, table_name, table_data, current_socket):
    table_data = json.loads(table_data)
    # פענוח התמונות מ-base64
    for key in table_data.keys():
        if key.startswith("picture") and table_data.get(key) != "X":
            table_data[key] = base64.b64decode(table_data[key]) # פענוח

```

דוגמה של הכנת הודעת מחרוזת של ה - server (שימוש ב - base64):

```

1 usage  ± TamarHoffmann
def fetch_homepage_content(app_name):
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()
    cursor.execute(f"SELECT * FROM {app_name}_Homepage")
    raw_data = cursor.fetchall()
    conn.close()

    processed_data = []
    for row in raw_data:
        row_list = list(row)
        # המרת עמודות התמונות (אנדקס 4 עד 6)
        for i in range(4, 7):
            if row_list[i] != "X": # רק אם זו תמונה
                row_list[i] = base64.b64encode(row_list[i]).decode('utf-8') # Base64 עידוד
        processed_data.append(row_list)
    return processed_data

```

ולאחר מכן שימוש ב - Json בצד ב - server:

```

elif command == 'GET_HOMEPAGE_CONTENT':
    content = fetch_homepage_content(app_name)
    messages_to_send.append((current_socket, json.dumps(content)))

```

ולבסוף שליחת הודעות ללקוח מצד ה - server כאשר במקרה הצורך יש שימוש בפקטות:

```

if messages_to_send:
    for message in messages_to_send:
        current_socket, data = message
        if current_socket in wlist:
            data = RSA.encrypt(client_and_keys[current_socket][0], data)
            data = ", ".join([str(n) for n in data])

            packets_num_int = len(data.encode('utf-8')) // self.MAX_MSG_LENGTH
            if packets_num_int > 0:
                packets_num = RSA.encrypt(client_and_keys[current_socket][0], str(packets_num_int))
                packets_num = ", ".join([str(n) for n in packets_num])
                current_socket.send(str(packets_num).encode())

                for i in range(packets_num_int):
                    current_socket.send(
                        data[i * self.MAX_MSG_LENGTH:(i + 1) * self.MAX_MSG_LENGTH].encode())

                if len(data) % self.MAX_MSG_LENGTH != 0:
                    current_socket.send(data[packets_num_int * self.MAX_MSG_LENGTH:].encode())
            else:
                current_socket.send(data.encode())
        messages_to_send.remove(message)

```

דוגמה מצד ה - client לשליחת הודעה מורכבת:

```
5 usages (5 dynamic)  TamarHoffmann *
def insert_into_table(self, app_name, table_name, values):
    with self.lock:
        # קידוד התמונות ל-Base64
        for key in values.keys():
            if key.startswith("picture") and values[key] != "X" and values[key] != "Null":
                # בדיקת אם יש תמונה מתחיל ב-"picture" ואינו "X"
                values[key] = reduce_image_binary(values[key])
                values[key] = base64.b64encode(values[key]).decode('utf-8') # קידוד ל-Base64

        send_values = RSA.encrypt(self.server_key, plaintext=f"{table_name}@{json.dumps(values)}")
        send_values = ",".join([str(n) for n in send_values])
        packets_num = len(send_values.encode('utf-8')) // self.MAX_MSG_LENGTH

        self.send_message(f"{packets_num}@INSERT_INTO_TABLE@{app_name}")

        for i in range(packets_num):
            try:
                self.client_socket.send(send_values[i * self.MAX_MSG_LENGTH:(i + 1) * self.MAX_MSG_LENGTH].encode())
            except (socket.error, ConnectionRefusedError, BrokenPipeError, ConnectionResetError):
                self.retry_act(self.root)
                self.client_socket.close()
        if len(send_values) % self.MAX_MSG_LENGTH != 0:
            try:
                self.client_socket.send(send_values[packets_num * self.MAX_MSG_LENGTH:].encode())
            except (socket.error, ConnectionRefusedError, BrokenPipeError, ConnectionResetError):
                self.retry_act(self.root)
                self.client_socket.close()

        return self.recv_message("Error: Server is not connected")
```

הפעולה שהופכת את התמונות ל"קלות יותר" בעזרת IO בצד השרת:

```
def reduce_image_binary(image_bytes, target_size_kb=15):
    """ מקבל תמונה בינארית, מחזיר ל-JPG אם צריך, מקטין את הגודל, ומחזיר תמונה בינארית קטנה יותר """
    image = Image.open(io.BytesIO(image_bytes)) # טעינת התמונה מביןארי

    # בדיקת אם מדובר ב-PNG וחמרה ל-JPG
    if image.format == "PNG":
        image = image.convert("RGB") # הסרת ערוץ שקופות
        format_pic = "JPEG" # JPG-ל פורמט
    else:
        format_pic = image.format # PNG הוא לא פורמט המקורי אם הוא לא

    # קביעת איכות ראשונית
    quality = 95

    while True:
        buffer = io.BytesIO()
        image.save(buffer, format=format_pic, quality=quality, optimize=True)
        size_kb = buffer.tell() / 1024 # מחשבים גודל קובץ

        if size_kb <= target_size_kb or quality <= 10:
            break # עוצרים אם הגודל מתאים או אם איכות התמונה כבר נמוכה מדי

        quality -= 5 # הפחתת איכות בהדרגה כדי להקטין גודל

    return buffer.getvalue() # מחזיר את התמונה בפורמט בינארי
```

דוגמה מצד ה - client לקבלת הודעה מורכבת והחזרתה למקור:

```
2 usages  TamarHoffmann
def recv_packets_and_change_to_original(self, table):
    packets_num = self.recv_message(0)
    content = ""
    if type(packets_num) is int: # מאגד נדיר, שימוש בפקטות,
        for i in range(int(packets_num) + 1):
            time.sleep(0.05)
            try:
                content = content + self.client_socket.recv(self.MAX_MSG_LENGTH).decode()
            except (socket.error, ConnectionRefusedError, BrokenPipeError, ConnectionResetError):
                self.retry_act(self.root)
                self.client_socket.close()
                exit()

        content = content.split(",")
        content = [int(n) for n in content]
        raw_content = json.loads(RSA.decrypt(self.keys[1], content))
    else:
        raw_content = json.loads(packets_num)

    processed_content = []
    for row in raw_content:
        row_list = row
        if table == "Homepage":
            last_index = 7
        else:
            last_index = 5
        for i in range(4, last_index): # טענות טענות הממונות
            if row_list[i] != "X" and row_list[i] != "Null":
                row_list[i] = base64.b64decode(row_list[i]) # Base64 טענות
        processed_content.append(row_list)

    return processed_content
```

• הצפנת ההודעות- שימוש ב - RSA:

כפי שציינתי בפרק הקודם, על מנת לשמור על העברת ערכים מוצפנת ובטוחה במערכת שלי, בחרתי להשתמש בהצפנה RSA. בקטעי קוד אלו אפשר לראות בדיוק איך החלפתי את המפתחות בין השרת ללקוח ואיך יצרתי תקשורת מאובטחת לאורך כל התוכנית.

יצירת מפתחות הצפנה של הלקוח והחלפתן עם השרת:

```

    if self.status == "Connected":
        with open("primes2.txt", "r") as file:
            content = file.read().strip()
            primes = list(map(int, content.split()))

            random_primes = random.sample(primes, k=2)
            self.keys = RSA.generate_keypair(random_primes[0], random_primes[1])
            self.server_key = self.get_server_public_key()

    except ConnectionRefusedError:
        self.status = "Denied"

1 usage  TamarHoffmann
def get_server_public_key(self):
    with self.lock:
        try:
            self.client_socket.send(f"GET_SERVER_PUBLIC_KEY@{self.keys[0]}".encode())
        except (socket.error, ConnectionRefusedError, BrokenPipeError, ConnectionResetError):
            self.retry_act(self.root)
            self.client_socket.close()
            if threading.current_thread() != threading.main_thread():
                exit() # יוצא רק אם התרחש משגשג
    content = self.recv_message(00000000)
    return content

```

יצירת מפתחות הצפנה של השרת והחלפתן עם הלקוח:

```

connection, client_address = server_socket.accept()
connection.send("Connected".encode()) # שליחת הודעת הצלחה
print(f"New client joined: {client_address}")
client_sockets.append(connection)

with open("primes2.txt", "r") as file:
    content = file.read().strip()
    primes = list(map(int, content.split()))

    random_primes = random.sample(primes, k=2)

    keys = RSA.generate_keypair(random_primes[0], random_primes[1])
    client_and_keys[connection] = list(keys)

```

```

if command == "GET_SERVER_PUBLIC_KEY":
    public_key = client_and_keys[current_socket][0] # שמירת הערך הצבבורי של השרת
    client_and_keys[current_socket][0] = ast.literal_eval(app_name) # מייצגת את המפתח של הלקוח
    messages_to_send.append((current_socket, json.dumps(public_key)))

```


דוגמה לשימוש בהצפנה ופיענוח (במקרה זה אצל הלקוח):

```
def get_apps_names(self):
    with self.lock:
        send_data = RSA.encrypt(self.server_key, plaintext=f"GET_APPS_NAMES")
        send_data = ",".join([str(n) for n in send_data])
        self.client_socket.send(send_data.encode())

        response = self.client_socket.recv(self.MAX_MSG_LENGTH).decode()
        response = response.split(",")
        response = [int(n) for n in response]
        content = json.loads(RSA.decrypt(self.keys[1], response))
        return content
```

- התחברות מאובטחת, שימוש ב-Hashing, מניעת SQL Injection וביצוע אימות דו שלבי:

העברת סיסמאות מצד הלקוח, בצורה מוצפנת אחרי שעברו Hash:

```
user_var_dict = {
    'name': self.name_entry.get(), 'gmail': self.gmail_entry.get(),
    'phone_number': self.phone_num_entry.get(),
    'password': hashlib.sha256(self.password_entry.get().encode()).hexdigest()}

result = self.client.login_to_system(self.name, self.gmail_entry.get(),
                                     hashlib.sha256(self.password_entry.get().encode()).hexdigest())
```

קוד דוגמה של שאילתה המונעת SQL injection:

קוד אימות הדו שלבי מצד הלקוח:

```
def handle_login(app_name, *input_data):
    email, password = (input_data[0] + "@" + input_data[1]), input_data[2]
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()
    cursor.execute(_sql f"SELECT * FROM {app_name}_Users WHERE gmail = ? AND password = ?", _parameters: (email, password))
    user = cursor.fetchone()
    conn.close()
    if user is None:
        return "False"
    return user
```

```

else:
    user_var_dict = {
        'name': self.name_entry.get(), 'gmail': self.gmail_entry.get(),
        'phone_number': self.phone_num_entry.get(),
        'password': hashlib.sha256(self.password_entry.get().encode()).hexdigest()}

    # database
    # אמנם משתמש בפונקציית הודו אבל לא באמת נכנסם התכנים
    # אלא התוכן רק עובר את הבדיקות על סמך לראות שהוא אכן תקין
    self.error_label = ctk.CTkLabel(self, text=self.client.insert_into_table(
        self.name, 'Users', user_var_dict), text_color="red", padx=10)

    if self.error_label.cget("text").startswith("Error"):
        self.error_label.grid(row=5, column=1, columnspan=2)
    else:
        self.destroy()

        confirmation_frame = ctk.CTkFrame(self.root)
        confirmation_frame.pack(expand=True, fill="both")
        confirmation_frame.columnconfigure(index=(0, 2), weight=1)
        label = ctk.CTkLabel(confirmation_frame, text="Please enter your confirmation code",
                             font=("Black Ariel", 20))
        label.grid(row=0, column=1, pady=20)
        entry = ctk.CTkEntry(confirmation_frame)
        entry.grid(row=1, column=1, pady=20)
        send_button = ctk.CTkButton(confirmation_frame, text="Send",
                                    command=lambda: self.check_confirmation(entry.get(), confirmation_frame,
                                                                              user_var_dict))
        send_button.grid(row=2, column=1, pady=10)
        send_again_button = ctk.CTkButton(confirmation_frame, text="Send Again", command=lambda:
                                           self.client.send_confirmation_again(user_var_dict["gmail"]))
        send_again_button.grid(row=3, column=1, pady=10)

        j = json.load(open(f"designs/{self.design}_design.json"))
        back_to_sign_up = ctk.CTkButton(confirmation_frame, text="Back to Sing Up",
                                       fg_color=(j["CTK"]["fg_color"]),
                                       text_color=(j["CTKButton"]["fg_color"]), command=lambda:
                                       self.go_to_sign_up(confirmation_frame))
        back_to_sign_up.grid(row=4, column=1, pady=10)

2 usages (1 dynamic)  TamarHoffmann
def check_confirmation(self, entry, frame, user_vars):
    correct = self.client.check_confirmation(entry)
    if correct == "True":
        self.client.insert_user(self.name, user_vars)
        self.go_to_log_in(frame)
    else:
        error_label = ctk.CTkLabel(frame, text="Code is incorrect", text_color="red")
        error_label.grid(row=5, column=1, pady=10)

```

קוד שיפה בעיני – יצירת מערכת הפגישות:

האלגוריתם הזה אחראי על בניית הממשק לקביעת פגישות בצורה מסודרת: הפעולה create_frame מתחילה ביצירת כותרת ראשית (headline) שמוצגת למשתמש ומכריזה על אפשרות לקביעת פגישה. לאחר מכן, היא רצה בלולאה עבור שבעה ימים קדימה, ומוסיפה תוויות

(CTkLabel) לכל יום עם השם והתאריך שלו, כאשר תאריכי הימים מסונכרנים לתאריכים אמיתיים, והטבלה מתקדם עם הימים, זאת אומרת שכל יום הטבלה תיראה שונה.

לכל יום האלגוריתם מחשב זמני פגישות אפשריים בהתבסס על המשתנים שסופקו במערך

`self.input_arr`

- `self.input_arr[3]` מייצג את שעת ההתחלה.
- `self.input_arr[4]` מייצג את שעת הסיום.
- `self.input_arr[0]` הוא משך פגישה אחת בדקות.
- `self.input_arr[1]` הוא משך ההפסקות בין מקטעים של פגישות.
- `self.input_arr[2]` מייצג כמה פגישות יש לפני כל הפסקה.

בתוך הלולאה, לכל יום נוצרת רשימת פגישות (`arr`), ונתונים אלו משמשים לקביעת הזמנים שבהם ניתן לקבוע פגישה. לאחר קביעת זמני ההתחלה והסיום של כל פגישה, האלגוריתם בודק אם השעה פנויה על ידי קריאה לפונקציה `self.client.check_meeting_availability(self.name, meeting_time)`. אם הפגישה תפוסה, מוצגת תווית עם הזמן וכתוב לידו "(taken)". אם הפגישה פנויה, נוצר כפתור (CTkButton) שמאפשר למשתמש לבחור את הפגישה. לכל כפתור מחובר `command` המפעיל את הפונקציה `book_a_meeting` עם הפרטים המתאימים.

במקרה שהפגישה עוברת את שעת הסיום שהוגדרה, האלגוריתם מדלג על יצירת הפגישה. אם עדיין ניתן לקבוע פגישות נוספות אחרי סיום מקטע מסוים, הוא מוסיף תווית עם "-----" כדי להפריד בין מקטעים ויוצר הפסקה על ידי הוספת זמן הפסקה (`self.input_arr[1]`) לזמן ההתחלה. לבסוף, כל המידע נשמר במערך `self.meetings_arr`, ואם `self.user` מכיל ערך (כלומר, המשתמש מחובר), האלגוריתם מעלים את החלונית הקודמת (`self.frame.pack_forget()`) ומציג את רשימת הפגישות של המשתמש.


```

def create_frame(self):
    headline = ctk.CTkLabel(self, text="Book A Meeting With Us", font=("Broadway", 80))
    headline.grid(row=0, column=1, columnspan=7)

    for i in range(7):
        day = datetime.datetime.today() + datetime.timedelta(days=i)
        day_name = day.strftime('%A')
        label = ctk.CTkLabel(self, text=f"{day_name}\n {day.strftime('%Y-%m-%d')}", font=("Black Ariel", 18))
        label.grid(row=1, column=1 + i)
        arr = []
        start_time = self.input_arr[3]
        end_time = start_time + int(self.input_arr[0])
        row = 2

        while start_time < int(self.input_arr[4]):
            for j in range(int(self.input_arr[2])):
                if end_time > int(self.input_arr[4]):
                    pass
                else:
                    meeting_time = (f"{day_name}\n {day.strftime('%Y-%m-%d')}"
                                     f" {int(start_time / 60):02}:{start_time % 60:02}")
                    meeting_available = self.client.check_meeting_availability(self.name, meeting_time)

                    if meeting_available == "False":
                        arr.append(ctk.CTkLabel(self, text=f"{int(start_time / 60):02}:{start_time % 60:02}-"
                                                            f"{int(end_time / 60):02}:{end_time % 60:02} (taken)"))
                        arr[-1].grid(row=row, column=i + 1, padx=20, pady=5)
                    else:
                        arr.append(ctk.CTkButton(self, text=f"{int(start_time / 60):02}:{start_time % 60:02}-"
                                                            f"{int(end_time / 60):02}:{end_time % 60:02}",
                                                  command=partial(self.book_a_meeting, *args: len(arr), i, meeting_time, self.user)))
                        arr[-1].grid(row=row, column=i + 1, padx=20, pady=5)

                row += 1
                start_time = end_time
                end_time = start_time + int(self.input_arr[0])

            if start_time < int(self.input_arr[4]):
                label = ctk.CTkLabel(self, text="-----")
                label.grid(row=row, column=i + 1)
                row += 1
                start_time += int(self.input_arr[1])
                end_time = start_time + int(self.input_arr[0])

        self.meetings_arr.append(arr)
    if self.user != "":
        self.frame.pack_forget()
        self.my_meetings()

```

מסמך בדיקות מלא

- **מטרה:** האם המידע עובר נכון משני צדדי התקשורת?
- בוצעה:** כפי שתוארה - כדי לגלות אני הדפסתי את המידע בכל אחד מהצדדים ווידאתי שהמידע זהה בשני

הצדדים.

תוצאות: בכל התקדמות משמעותית בביצוע הפרויקט, אני ביצתי את הבדיקה הזו. התוצאה הנפוצה יותר והתוצאה הסופית, היא שהמידע בשני הצדדים זהה.

קרו לי בעיות שראיתי שלא כל המידע עובר/הוא לא עובר בסדר הנכון. את הבעיות האלו פתרתי בעזרת זה שעברתי בצורה קפדנית על הקוד וחיפשתי שורות לא תקינות, או שראיתי שההודעה גדולה מדי והוספתי פקטות, או שראיתי שיש בלגן עם שליחת ההודעות מהלקוח ופתרתי את זה עם lock.

○ **מטרה:** תקינות מנגנון ההצפנה והפיענוח.

בוצעה: כפי שתוארה - הדפסתי את המידע לפני ההצפנה ואחרי ההצפנה ווידאתי שהוא זהה.

תוצאות: בחלק זה לא נתקלתי בבעיות לכן התוצאה שהתקבלה לפני ואחרי ההצפנה הייתה זהה.

○ **מטרה:** תפקוד בסיס הנתונים – האם הוא מתעדכן לפי פעולות המערכת והאם המערכת פועלת בעקבות המידע שנמצא בבסיס הנתונים?

בוצעה: כפי שתוארה - נעזרתי בשימוש ב - db browser כדי לראות שהערכים נכנסים בצורה תקינה, ואת בדיקת הוצעת הערכים ביצתי באמצעות הדפסות והשוואת תוצאת ההדפסה למראה ב - db browser.

תוצאות: המידע הוכנס בצורה טובה ומסודרת לבסיס הנתונים.

○ **בדיקה:** בדיקת הצפנה ה - Hash .

בוצעה: כפי שתוארה - בעזרת הדפסת הסיסמאות לפני שהן נשלחות ובדיקת תוכן הסיסמה בתוך הטבלאות של בסיס הנתונים בעזרת db browser, יכולתי לראות שפונקציית ה - hash עובדת.

תוצאות: הצפנה זו עבדה בצורה טובה ללא בעיות.

○ **מטרה:** האם האימיילים אכן נשלחים ועם תוכן תקין?

בדיקה: כפי שתוארה - בדקתי בכך ששלחתי את המיילים למייל שלי ובחנתי את התוצאות.

תוצאות: לקח לי קצת זמן להבין את המבנה של האימייל, נעזרתי בבדיקה הזו כדי לבדוק אותו, אז בסופו של דבר האימיילים נשלחים בצורה טובה ומסודרת עם תוכן תקין.

○ **מטרה:** ווידוא יצירת אפליקציה תקינה.

בדיקה: כפי שתוארה - להריץ את התוכנה ולראות שהאפליקציה שנוצרה זהה לזו שהיוצר התכוון ליצור.

תוצאה: בתחום זה לא נתקלתי בבעיות, כן השתמשתי בבדיקה זו הרבה על מנת לעצב את התבניות של האפליקציות השונות בצורה שתהיה טובה לכל קלט.

○ **מטרה:** וידוא העלאה של אפליקציות מספריית האפליקציות בצורה תקינה.

בדיקה: כפי שתוכננה - יצרתי כמה אפליקציות שונות וניסיתי להיכנס אליהן. בתור יוצרת האפליקציות, יכולתי לראות שהן זהות למבוקש, ובנוסף, נעזרת במידע השמור בבסיס הנתונים בעזרת הסתכלות עליו מ - db browser.

תוצאה: עבד ופתח את כל האפליקציות שרציתי.

○ **מטרה:** בדיקת מקרי קצה.

בדיקה: תוך כדי פיתוח הקוד לנסות למצוא את מקרי הקצה בעזרת פעולות מסוימות או פלטים מסוימים

ולבדוק שתוצאת המערכת זהה למצופה.

תוצאות: נמצאו לא מעט מקרי קצה אותם תיקנתי ועכשיו הקוד עובד בצורה טובה לכל סוג מקרה קצה שהוא.

○ **מטרה:** בדיקת הגנה מ - SQL injection .

בדיקה: כפי שתוכננה - בעזרת הזנת מחרוזות עם הנתאים המתאימים ל - SQL injection ובדיקה שהמערכת

מגנה מזה.

תוצאה: המערכת חסינה לסוג התקפה זה.

○ **מטרה:** בדיקת תקינות השרת לכמה משתמשים במקביל.

בדיקה: כפי שתוכננה - פתיחת קוד הלקוח על שני מחשבים אחרים ותפעולם מול השרת.

תוצאות: עבד והשרת הצליח להתמודד עם כמה לקוחות במקביל.

○ **מטרה:** בדיקת הגנה מדידוס.

בדיקה: כפי שתוכננה - הקטנת כמות הלקוחות המקסימלית ופתיחת מספר לקוחות הגדול מאחד מהמספר

המקסימלי. לראות שהשרת יודע לקבל רק עד הכמות המקסימלית ושלא מתפנה (אחרי סגירת אחד

מהלקוחות), אז השרת יודע לקבל לקוח חדש.

תוצאות: מערכת ההגנה מדידוס עובדת מעולה.

○ **מטרה:** בדיקת שרת מרובה משתמשים.

בדיקה: כפי שתוכננה - פתיחת השרת במחשב אחד וכמה משתמשים במחשבים אחרים ולראות שתפעולו

עובד.

תוצאות: בהתחלה זה לא עבד בצורה מושלמת, הייתי צריכה לבצע מעט שינויים בקוד ולהשתמש

בtime.sleep על מנת להעביר פקטות בין מחשבים, ובסופו של דבר הבדיקה עבדה.

○ **מטרה:** בדיקת טיפול הלקוח כאשר השרת הפסיק לפעול.

בדיקה: כפי שתוכננה - הפעלת השרת וחיבור הלקוח אליו, הפסקת הרצה השרת ולראות שהלקוח מתמודד

עם זה.

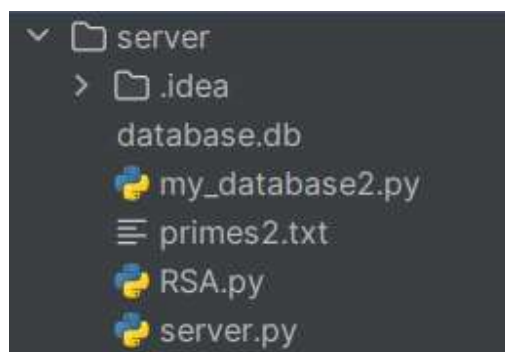
תוצאה: בהתחלה ראיתי שהלקוח לא מצליח להתמודד עם זה והוא נופל גם, אך פתרתי את זה וכרגע

מופיעה הודעה ללקוח על בעיות בשרת.

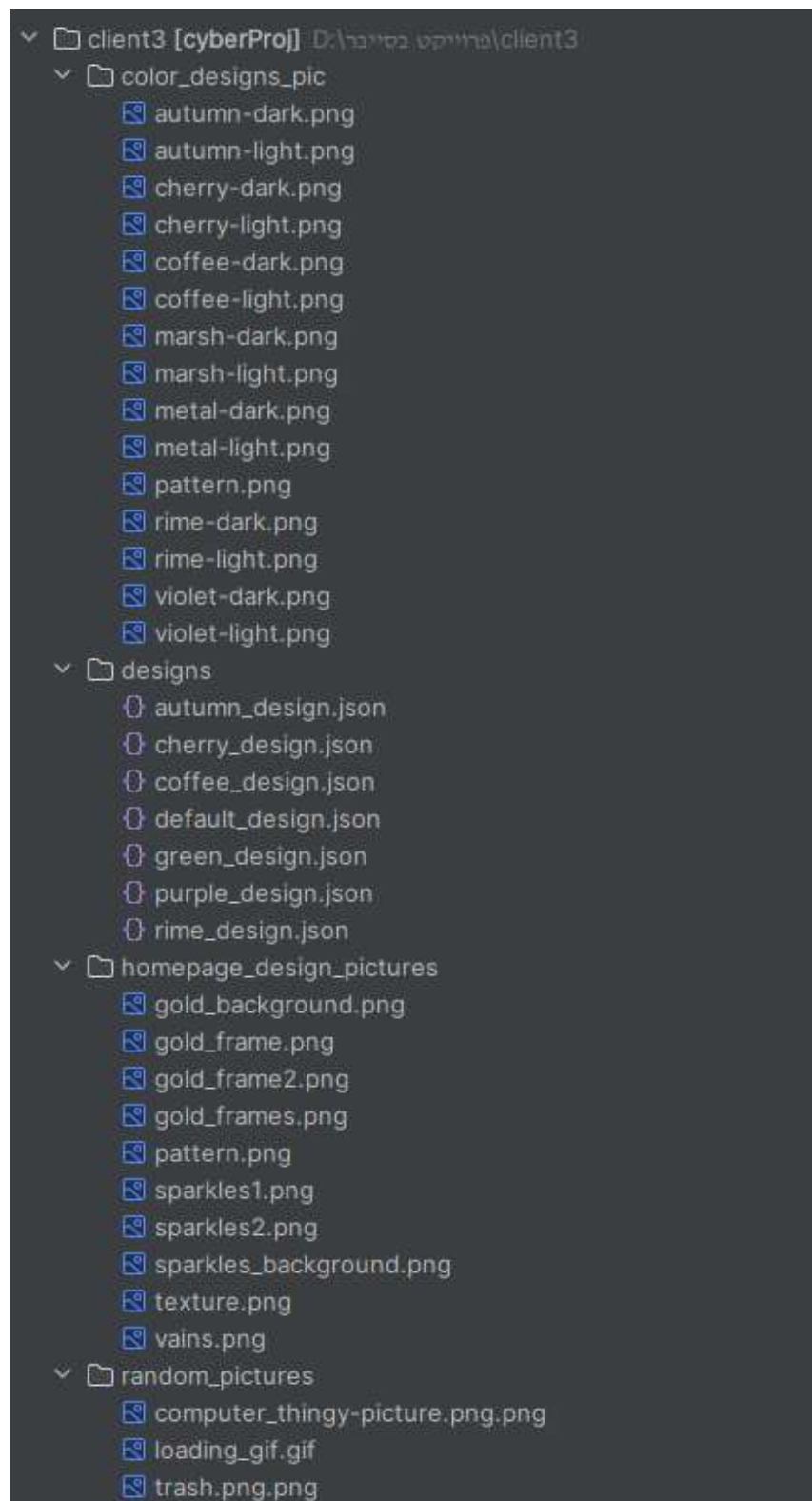
מדריך למשתמש

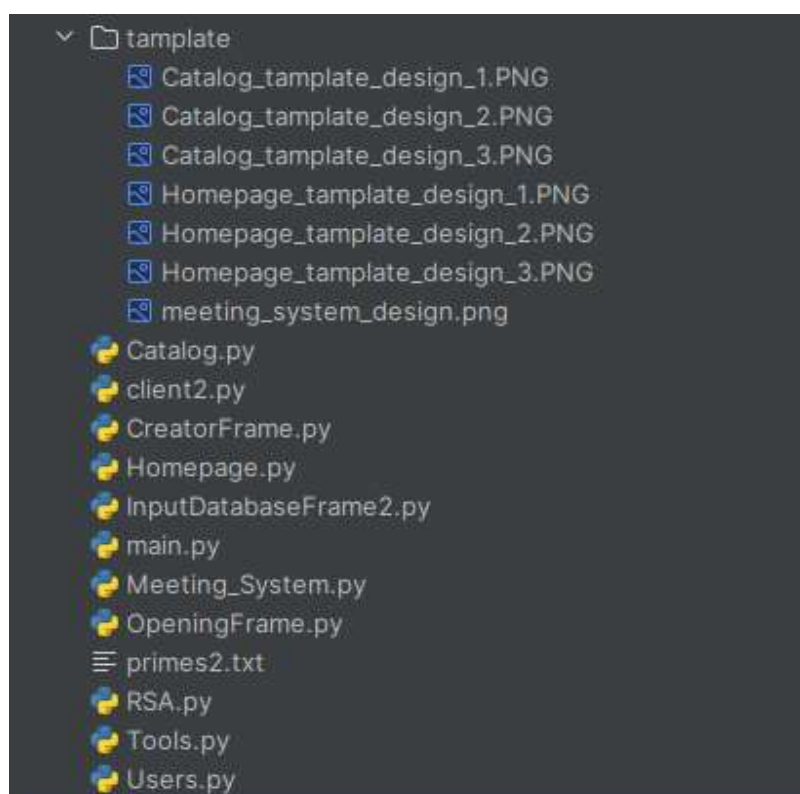
עץ קבצי המערכת:

קבצי שרת:



קבצי לקוח:





התקנת המערכת

סביבת העבודה:

על מנת לעבוד עם הקוד ולהריצו דרוש פייתון של 3.11 ומעלה, ודרושים כל ספריות והמודלים המיובאים.

הכלים הנדרשים:

על מנת לעבוד עם הקוד או להריץ אותו, מומלץ להשתמש ב - PYCHARM.

על מנת לקרוא את המידע במסד הנתונים, מומלץ להשתמש ב - SQLite Browser DB.

על מנת להתקין את כל המודלים והספריות המיובאים, צריך תחילה להתקין את PIP (מנהל הספריות) ולהוריד את כל המודולים שצוינו.

את ההתקנה ניתן לבצע דרך פתיחת ה - cmd והרצת הפקודה: `pip install {module name}`

נתונים התחלתיים:

כל הנתונים ההתחלתיים כמו אימייל המערכת שמורים כמשתני סביבה במחשב השרת.

נדרש רק לשנות את תכונת ה - IP כך שתתאים למחשב השרת.

רשת:

על מנת שהפרויקט יעבוד, המחשבים צריכים להיות מחוברים לאותה רשת.

בנוסף, יש לוודא שהמחשב או המחשבים שמהם מריצים את קוד המערכת מחוברים לאינטרנט, כדי שקודי האימות ושאר ההודעות יישלחו לכתובות הדואר האלקטרוני.

******אין לשנות את מיקומם/שם של הקבצים הנמצאים בתוך התיקייה הראשית של הפרויקט.

מדריך להפעלת השרת:

כדי להריץ את הפרויקט, תחילה יש להריץ את הקובץ `server.py` ממחשב השרת. זאת על ידי לחיצה על הכפתור עם המשולש הירוק או לחיצה על מקש ימני בעכבר ואז על `"run 'server'"`. בזאת מסתיימת עבודתו של מנהל המערכת. יש לשים לב שכל משתני הסביבה מוגדרים לפני ההרצה ושכתובת ה-IP תקינה.

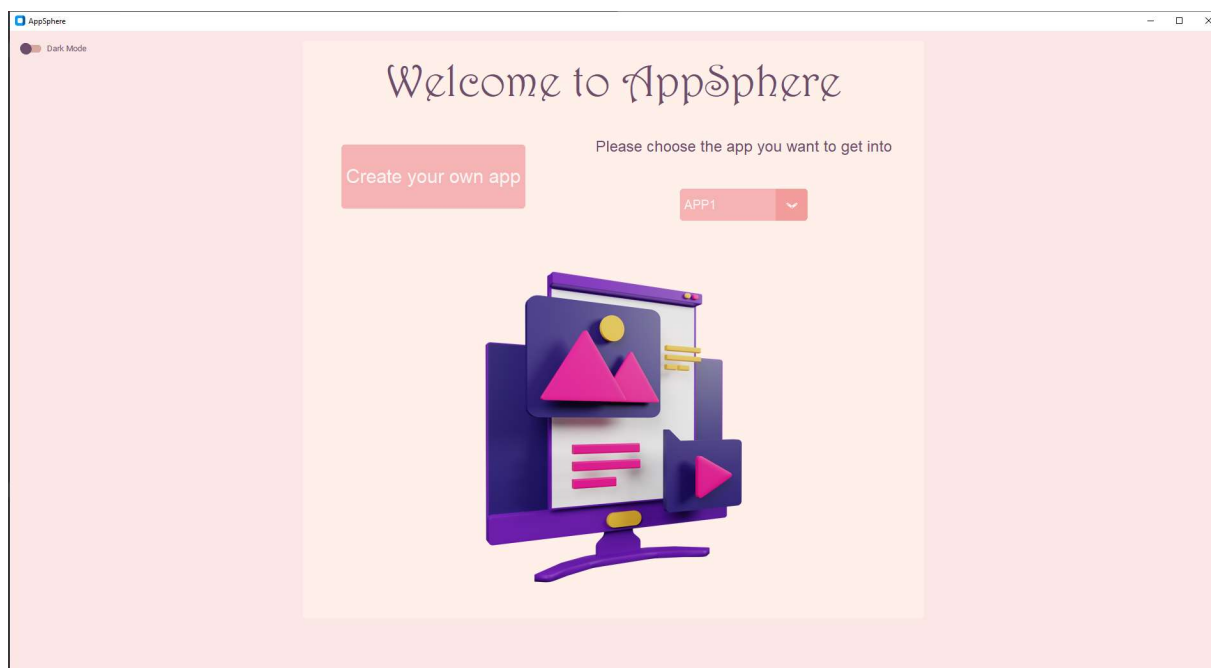
מדריך למשתמש AppSphere:

לפני הרצת הפרויקט:

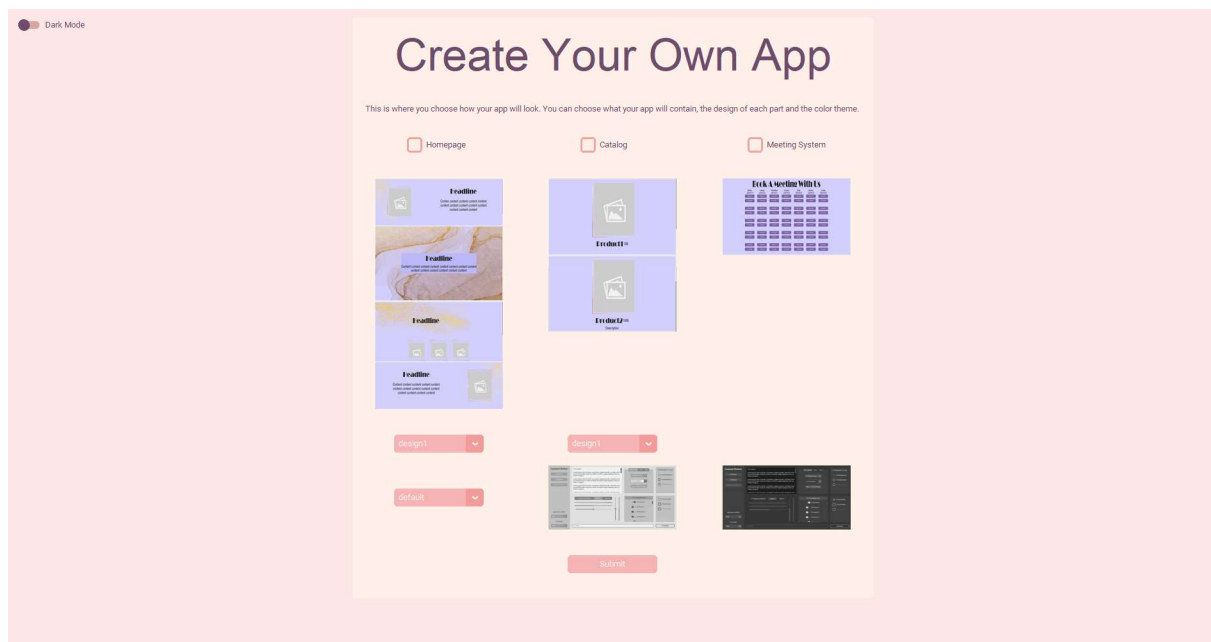
יש לשנות את IP הלקוח בהתאם ל-IP של המחשב שבו מופעל השרת, אחר כך על המשתמש לפתוח ב-PyCharm את הקובץ ה-`main` ולהריץ אותו. זאת על ידי לחיצה על הכפתור עם המשולש הירוק או לחיצה על מקש ימני בעכבר ואז על `"run client"`. מכאן כל העבודה היא מול ממשק המשתמש.

מדריך מול הממשק הגרפי:

בעת הכניסה הראשונית למערכת, המשתמש יפגוש את **מסך הפתיחה**, שמציע שתי אפשרויות: יצירת אפליקציה חדשה או כניסה לאחת האפליקציות הקיימות דרך תפריט הבחירה.



אם המשתמש יבחר ליצור אפליקציה משלו, הוא יגיע למסך הגדרת תכונות האפליקציה, שבו ניתן להתאים את האפליקציה לרצונות האישיים, לבחור את הדפים שייכללו (עמוד בית, קטלוג או מערכת פגישות), לקבוע את סגנון העיצוב, ואת צבע ערכת הנושא.



לאחר קביעת מאפייני האפליקציה, המסך הבא הוא מסך הזנת הנתונים, שבו המשתמש מכניס את הפרטים שירצה שיופיעו באפליקציה שלו. לכל חלק שנבחר במסך הקודם, יהיה שדה מתאים שבו ניתן להזין מידע. כך, אם נוספה מערכת פגישות, יש למלא נתונים כמו משך פגישה, הפסקות בין פגישות, ושעות פעילות יומיות. בקטלוג,

המשתמש יוסיף מוצרים עם שם, מחיר, תמונה ותיאור אופציונלי. חשוב לדעת שהמערכת בודקת את תקינות הנתונים לפני שמירתם, ואם משהו לא תקין (למשל שם אפליקציה שכבר קיים או שדה ריק), המשתמש יקבל הודעה עם הסבר על השגיאה ויידרש לתקן אותה.

בסיום הכנסת הנתונים, המשתמש מועבר האפליקציה שיצר שמכילה את **סרגל הכלים**, שמאפשר ניווט קל בין הדפים שנבחרו באפליקציה. בנוסף, אם במסך הפתיחה המשתמש בחר להיכנס לאפליקציה קיימת, הוא יועבר למסך ותהליך זה. אם המשתמש לא הגדיר קטלוג, למשל, הכפתור שלו לא יופיע בסרגל. דרך סרגל הכלים ניתן גם להתחבר לחשבון באמצעות כפתור Log In, וכן להחליף בין dark mode - light mode. במרכז המסך יוצג שם האפליקציה, בהתאם למה שנבחר בשלבים הקודמים.



כאשר המשתמש נמצא בתוך האפליקציה, הוא יוכל לצפות בתוכן של **עמוד הבית**, אם זה הוגדר מראש, ולגלול בין האלמנטים שהוזנו (מוצגת דוגמה של עמוד בית לאתר תפירה).

Stitch and Style — Your Sewing Haven



Welcome to Stitch and Style, where creativity meets craftsmanship. Whether you are a seasoned tailor or just starting your sewing journey, our shop is dedicated to providing you with the finest materials, tools, and inspiration.

Explore our collection of high-quality fabrics, threads, and accessories, carefully selected to bring your designs to life. With expert guidance and a passion for sewing, we are here to help you turn every stitch into a

masterpiece.

Let your creativity shine with Stitch and Style.

Contact Us

our products or need advice for your next project, we are happy to help.

You can reach us by phone during our business hours or send us an email anytime. Visit our store to explore our collection in person and get expert guidance from our team.

We look forward to hearing from you and being part of your creative journey.

Schedule a Meeting

We offer personalized consultations to help bring your sewing projects to life. Whether you need guidance on fabric selection, design ideas, or professional sewing techniques, we are here to assist you.

Book a one-on-one appointment with our experts to get tailored advice and hands-on support. Meetings can be scheduled in advance to ensure dedicated time for your needs.

Let us know how we can help, and we will

find the perfect time for you!



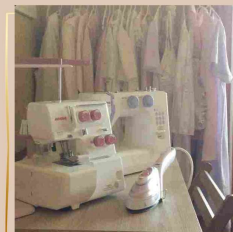
Our Product Catalog



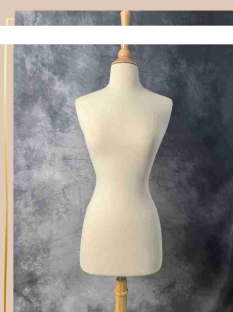
carefully chosen to enhance your creative projects. From premium fabrics and colorful threads to specialized sewing equipment, our catalog offers everything you need to bring your designs to life.

Browse through our collection and find the perfect supplies for every stitching style. Whether you are working on a professional piece or a fun DIY project, we have the right products for you.

אם לא קיים עמוד בית, המסך הראשון שיופיע יהיה **קטלוג המוצרים**, שבו ניתן לעיין בפריטים שהוגדרו ולראות את המידע הקיים עבור כל מוצר (מוצגת דוגמה של קטלוג לאתר תפירה).



sewing machine^{129\$}



sewing manikin^{59\$}





אם עמוד בית וקטלוג אינם קיימים, המסך הראשון שיופיע יהיה **מערכת הפגישות**, שמציגה טבלה מסודרת של זמני פגישות אפשריים. הטבלה מתעדכנת לפי ימים קדימה, וכל פגישה מופיעה ככפתור. אם הפגישה תפוסה, הכפתור יוחלף בתווית המציינת כי הזמן הזה אינו זמין. משתמשים יכולים לבחור פגישה על ידי לחיצה על הכפתור המתאים, אך רק לאחר התחברות למערכת.

Book A Meeting With Us						
Monday 2025-05-19	Tuesday 2025-05-20	Wednesday 2025-05-21	Thursday 2025-05-22	Friday 2025-05-23	Saturday 2025-05-24	Sunday 2025-05-25
09:00-10:40	09:00-10:40	09:00-10:40	09:00-10:40	09:00-10:40	09:00-10:40	09:00-10:40
11:00-12:40	11:00-12:40	11:00-12:40	11:00-12:40	11:00-12:40	11:00-12:40	11:00-12:40
13:00-14:40	13:00-14:40	13:00-14:40	13:00-14:40	13:00-14:40	13:00-14:40	13:00-14:40
15:00-16:40	15:00-16:40	15:00-16:40	15:00-16:40	15:00-16:40	15:00-16:40	15:00-16:40

אם משתמש אינו מחובר ומנסה לקבוע פגישה, הוא יגיע ל**מסך ההתחברות**, שבו יש להזין שם משתמש וסיסמה. במקרה של טעות, תופיע הודעה שתסביר את הבעיה.

Log In/Sign Up

Log In

Gmail:

Password: *

Log In

Sing Up

Forgot Password

משתמשים שאינם רשומים יכולים להגיע מכאן אל **מסך ההרשמה**, שבו הם ממלאים פרטים שונים, כגון שם, אימייל, מספר טלפון וסיסמה.

Log In/Sign Up

Sing Up

Name:

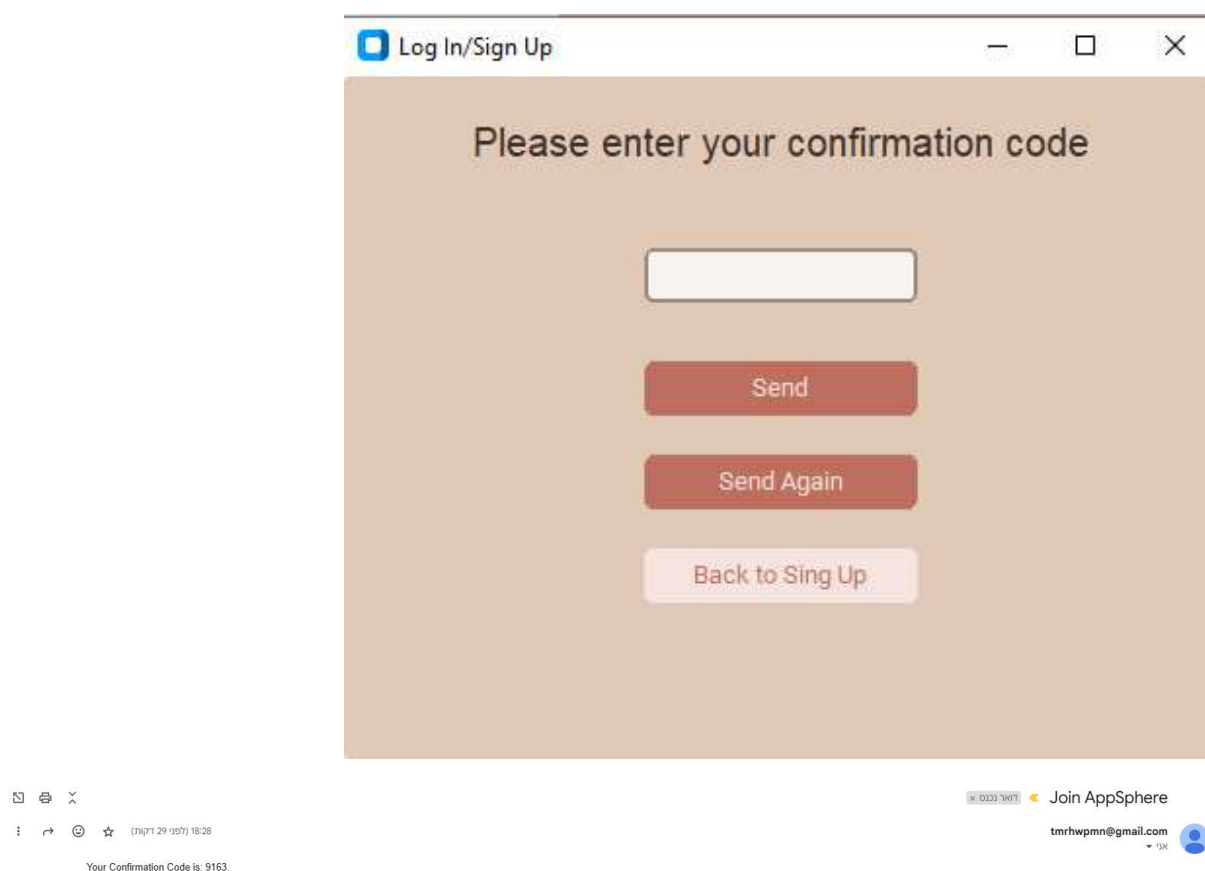
Gmail:

Phone Number:

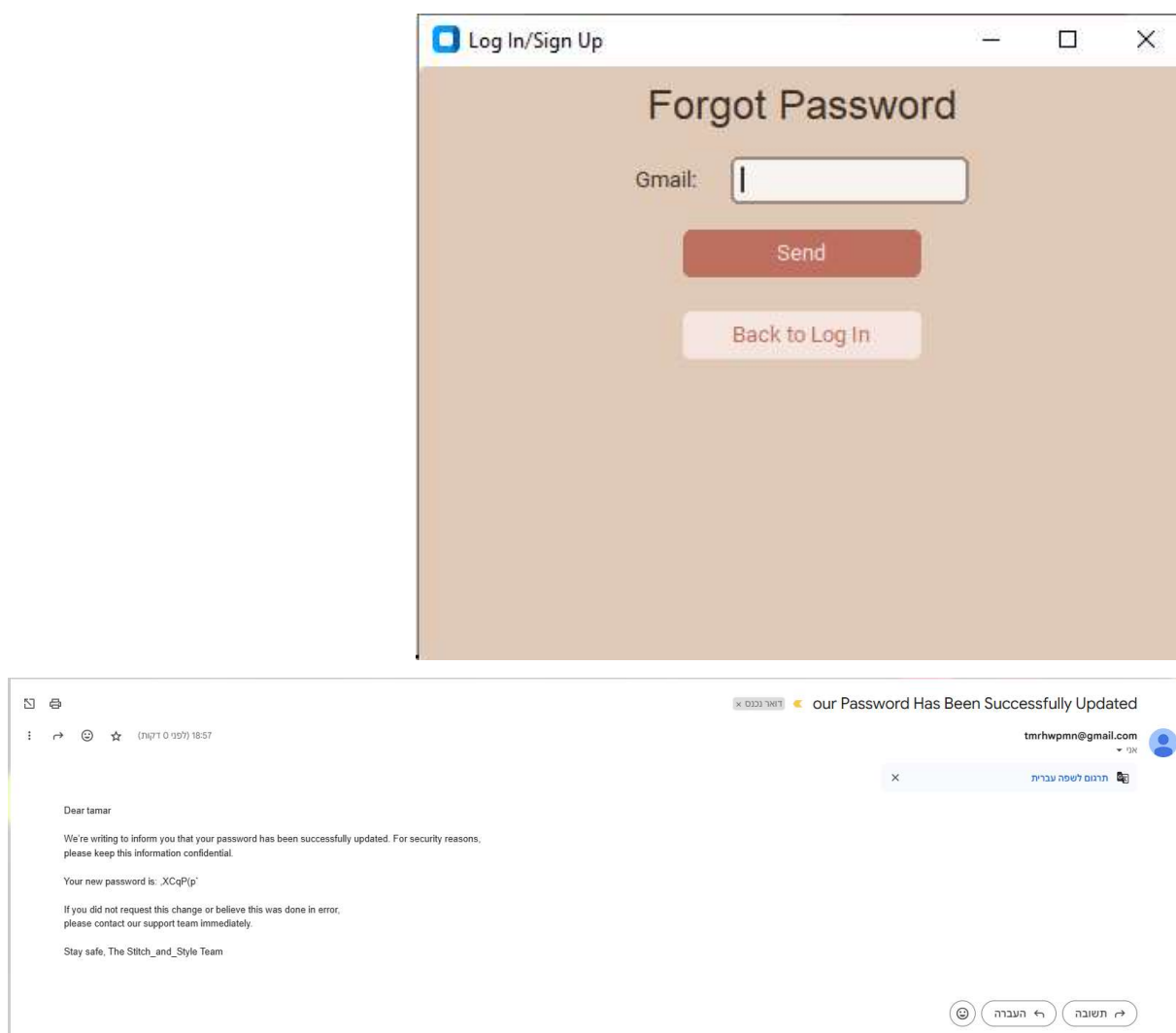
Password:

Sing Up

הנתונים נבדקים באופן אוטומטי כדי להבטיח תקינות (למשל תבנית תקינה של כתובת דוא"ל). לאחר ההרשמה, המשתמש יועבר אל **מסך האימות**, שם יהיה עליו להזין את הקוד שנשלח אליו במייל כדי לאשר את החשבון. אם האימייל לא הגיע, ניתן ללחוץ Send Again כדי לנסות לשלוח שוב את קוד האימות.



במידה והמשתמש שכח את הסיסמה, הוא יכול לעבור למסך שכחתי סיסמה, שבו הוא מזין את כתובת המייל שלו, ומקבל לשם סיסמה חדשה. אם האימייל לא הגיע, אפשר ללחוץ שוב על Send כדי לשלוח סיסמה חדשה. לאחר קבלת הסיסמה, ניתן לחזור למסך ההתחברות ולהזין אותה כדי להיכנס לאפליקציה.



לאורך כל השימוש באפליקציה, המערכת דואגת לעדכן את הנתונים כך שיישארו סינכרוניים ומדויקים. לדוגמה, אם משתמש מחובר וקובע פגישות, הוא יכול לראות אותן בתחתית הטבלה ולמחוק פגישות אם ירצה בכך באמצעות כפתור הפח שנמצא ליד כל פגישה. כל שינוי במערכת מתעדכן מיידית כדי להבטיח חוויית שימוש נוחה ויעילה, ומעדכן את המשתמש על כך דרך האימייל.



📧 📧 ✕
⋮ ↩️ 😊 ☆ (לפני 28 דקות) 18:30

Dear Tamar,

Thank you for scheduling an appointment through Tamplate! We're happy to confirm that your appointment is set for Tuesday 2025-05-20 09:05.

If you have any questions or special requests before the meeting, please feel free to reach out. We're here to help!

Best regards!

דואר (כנס) ✕ Appointment Confirmation

tmrhwpmn@gmail.com



✕ תרגום לשפה עברית 🗣️

📧 📧
⋮ ↩️ 😊 ☆ (לפני 29 דקות) 18:30

Dear Tamar,

We regret to inform you that your appointment scheduled through Tamplate for Saturday 2025-05-24 10:05 has been canceled.

If you would like to reschedule or need any further assistance, please don't hesitate to reach out. We apologize for any inconvenience and appreciate your understanding.

Best regards!

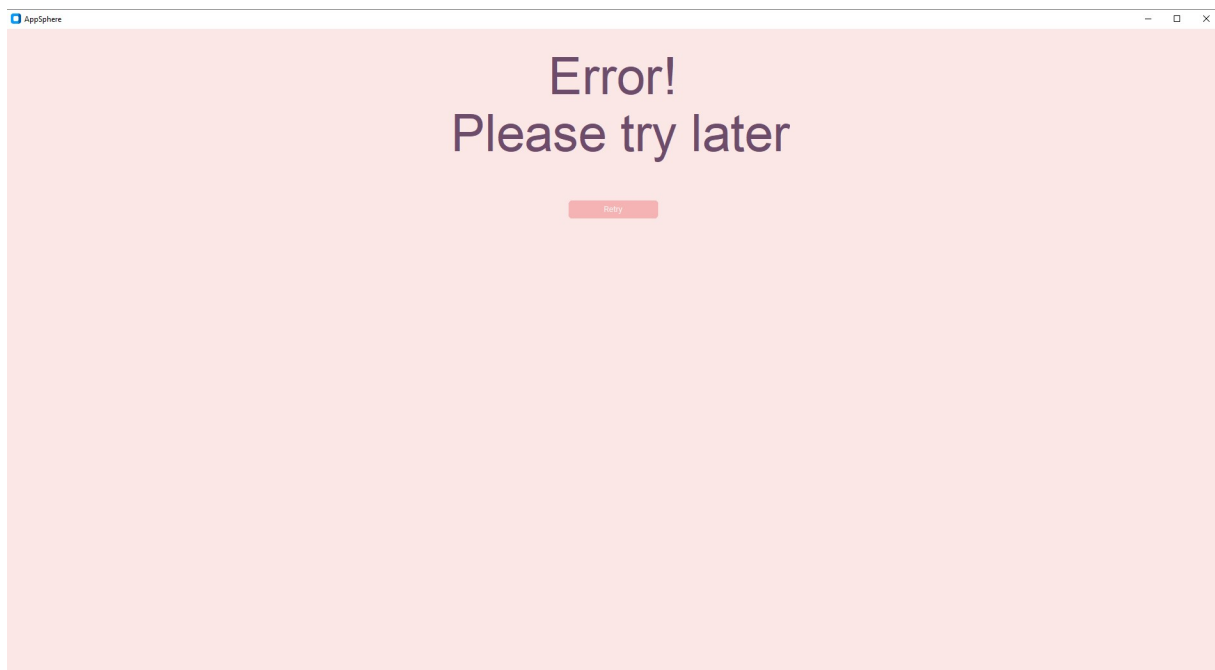
דואר (כנס) ✕ Appointment Cancellation

tmrhwpmn@gmail.com



✕ תרגום לשפה עברית 🗣️

****אם במקום המסך הראשון, מופיע המסך הזה, אל דאגה, על המשתמש לנסות להתחבר למערכת מאוחר יותר או לנסות לטעון מחדש את המערכת בעזרת לחיצה על הכפתור Retry.**



רפלקציה/ סיכום אישי

העבודה על פרויקט זה הייתה אינטנסיבית ולא פשוטה אך מעניינת וכן נהייתי מרובה. עבדתי על הפרויקט הזה מספר חודשים לא קטן, בהם מידת האינטנסיביות עלתה כל יום. בזמן הכנת הפרויקט הפתעתי את עצמי עם כמות השעות שאני יכולה ברצף לעבוד עליו ואת תחושת התגמול כל פעם שסיימתי חלק חשוב.

בזמן הכנת הפרויקט נתקלתי בלא מעט בעיות שנדרשתי לשבת עליהן לכמה שעות רצוף עד שהכול יעבוד. את רובן הצלחתי לפתור בעצמי לבד, אך היו כמה ספציפיות שנעזרתי במורה וישבנו על הבעיה לשעות ביחד עד שהיא נפתרה. אני מרגישה שעם כל הטעויות ובעיות והפתרונות שלהן, ראיתי עד כמה חשוב לבודד את הבעיה, בין אם זה בעזרת print-ים במרווחים שקטנים כל פעם או בין פתיחת קובץ חדש, בו נוכל לצמצם את הנוכחי עד מציאת הבעיה. רק לאחר בידוד הבעיה אפשר באמת להיפטר מכל התפל ולראות בדיוק מה צריך לשנות או לשפר, ומכאן העבודה בדרך כלל פחות קשה.

בזמן עשיית הפרויקט, אני מרגישה שלמדתי כל כך הרבה דברים חדשים בכל כך הרבה תחומים שונים. לדוגמה, אני זוכרת שלפני שהתחלתי את הפרויקט, מה שהיה נראה לי מאיים זה איך הכל מתחבר בעצם? איך התקשורת והממשק הגרפי וההגנות ומסד הנתונים נמצאים באותם כמה קבצי פייתון. אומנם זו שאלה שגיליתי את תשובתה מהר יותר מאחרות, כי כבר בזמן תכנון הפרויקט, הכל התחיל להתחבר, אך אני מרגישה שככל שהמשכתי להתקדם, למדתי עוד ועוד על הנושא הזה.

כל נושא שהוספתי לפרויקט, היה מלווה בחקירה מעמיקה עליו כדי שיפעל בדיוק איך שאני צריכה, גם אם זה נושא שלמדנו בכיתה, ראיתי שאני צריכה להתעמק הרבה יותר בשביל להבין מה בדיוק אני צריכה ולצרף אותו לפרויקט שלי. החקירה על הפונקציות השונות לימדה אותי הרבה על מודולים שונים ואופן תפקוד הפייתון. בנוסף, אני מרגישה שכל נושא שנתקעתי עליו, יצאתי בסוף עם ידע הרבה יותר מעמיק לעומת אם הכל היה עובד מיד מההתחלה.

אני מרגישה שרק בזמן העבודה על הפרויקט אני הכפלתי לפחות פי 5 את כמות הידע שהיה לי בתחום הסייבר והמחשבים, ואני מרגישה שאני מבינה הרבה יותר טוב עכשיו את הנושא ואת צורות הלמידה העצמית שנוחות לי. בזמן כתיבת הפרויקט יצא לי הרבה ללמוד עם חברים, לא רק בזמן השיעור, אלא בעיקר אחרי. אנחנו דרבנו אחד את השני לשבת ולעבוד בכך שקבענו לעבוד ביחד, וגם במקרה של בעיה, נעזרנו אחד בשני. זה באמת היה אחד החלקים הכי נחמדים בעבודה על הפרויקט.

בראייה לאחור, הייתי משנה הרבה דברים בפרויקט ובמהלך עשייתו: דבר ראשון, הייתי מתחילה לעבוד בצורה יותר רצינית על הפרויקט בשלב מוקדם יותר, כך העבודה הייתה יותר מאוזנת והייתי יכולה אולי להוסיף עוד דברים ולשפר עוד את הפרויקט שלי. אמנם אני חושבת שהתוצר שיצא לי הוא מאוד מרשים, והכי חשוב עונה על דרישות הפרויקט, אך ברעיון המקורי שלי היו עוד הרבה מאוד פיצ'רים

שלא הספקתי להכניס בפרויקט.

דבר שני, אני התחלתי לעבוד על הפרויקט בזמן שלא היה לי עדיין את כל הרעיון והתכנון מוכן. אני חשבתי שכן, ושהכל מאורגן, אך ככל שהתקדמתי הגעתי לעוד צמתים של החלטות על אופי הפרויקט. בסופו של דבר, מבחינה תכנונית וגרפית, הפרויקט שלי השתנה הרבה מאוד: הוא התחיל ממערכת שיוצרת אפליקציה אחת ומורידה למשתמש את האפליקציה שיצר. ואז איפה התקשורת? איך הוא יוכל להשתמש בה? אחר כך זו הייתה אפליקציה שיוצרת אפליקציה ומשאירה אותה פתוחה על המחשב של היוצר והוא יכול להשתמש בה. אבל איך לקוח יכול להיכנס? ולבסוף הגעתי לרעיונות הכללי של ספריית האפליקציות.

דבר שלישי ואחרון, במבט לאחור אני התמקדתי בדברים תפלים להרבה זמן ובהסתכלות אחורה, עם כל האינטנסיביות של כתיבת הפרויקט בכמה השבועות האחרונים, זה נראה לי פשוט חבל; זאת אומרת שאני מרגישה שהתמקדתי הרבה מאוד בממשק הגרפי, ואז כל נושאי התקשורת וההצפנה היו יותר לחוצים.

אני לא מרגישה שהיו חסרים לי משאבים ליצירת הפרויקט, רק זמן. גם הזמן הזה באמת לא היה לי מספיק אף פעם כי אני רואה שאני כל הזמן חושבת על עוד ועוד רעיונות להמשיך ולפתח את הפרויקט שלי. הדברים שהייתי רוצה להוסיף אם היה לי יותר זמן הם:

- הרשאות עריכה ליוצר האפליקציה. המצב שקורה כרגע הוא שיוצר האפליקציה יוצר את האפליקציה שלו ועכשיו יש לו את אותן הרשאות כמו למשתמש רגיל. אני חושבת שההוספה של דמות היוצר תוסיף הרבה מאוד ותשדרג ממש את האפליקציה שלי.
- בהמשך לנקודה הקודמת – אולי חיפוש בספריית האפליקציות לפי יוצר.
- לעשות סינונים ומיונים של מוצרי הקטלוג לפי בקשת המשתמש.
- להוסיף עוד עיצובים של כל חלק מהאפליקציה בשביל שהיא תהפוך להיות עוד יותר מותאמת אישית.

תודות:

אני רוצה להודות לכל מורי המגמה – עידן, רום, עודי וקובי, שליוו אותי לאורך כל הפרויקט. תודה ספציפית לעידן פלדברג, שעזר לי לתכנן את הזמנים ביצירת הפרויקט ושיש איתי שעות ארוכות על מנת לפתוח בעיה בקוד. ותודה נוספת לרום רפסון, שעזר לי לפתח את הרעיון של הפרויקט ולהתחיל ליישם אותו, וייעץ לי על פתרונות לפתירת בעיות או ייעול הקוד.

בנוסף, תודה רבה לחברים שלי במגמה – ליה, דנה, תמר ועמית, שבזכותם העבודה על הפרויקט הייתה מהנה הרבה יותר. לאורך כל הדרך הם תמכו, סייעו והציעו עזרה הדדית, מה שהפך את העבודה לנחמדה הרבה יותר.

ביבליוגרפיה

Stackoverflow ○

<https://stackoverflow.com/questions>

Python.org ○

<https://discuss.python.org/>

GitHub ○

<https://github.com/giampaolo/psutil/issues>

Copilot ○

<https://copilot.microsoft.com/chats/bY4FWbVZhyiuTXpFMpAXR>

נספחים – קוד הפרויקט

מצד השרת:

:server

```

import socket
import time
import select
import sqlite3
import json
import base64
import my_database2
import random
import RSA
import ast
import smtplib
from email.message import EmailMessage
import hashlib
import string
import threading

class Server:
    def __init__(self):
        self.SERVER_IP = '127.0.0.1'
        self.SERVER_PORT = 1729
        self.MAX_MSG_LENGTH = 10240
        self.codes = {}
        self.max_clients = 20

        self.server()

    def server(self):
        # השרת הפעיל
        server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        server_socket.bind((self.SERVER_IP, self.SERVER_PORT))
        server_socket.listen()

        print("Server is running...")
        client_and_keys = {}
        client_sockets = []
        messages_to_send = []

        while True:
            rlist, wlist, xlist = select.select([server_socket] +
client_sockets, client_sockets, [])

            for sock in xlist: # באריגות טיפול
                print(f"Exception detected on socket: {sock}")
                client_sockets.remove(sock)
                sock.close()

            for current_socket in rlist:

```

```

        if current_socket is server_socket:
            if len(client_sockets) < self.max_clients:
                connection, client_address = server_socket.accept()
                connection.send("Connected".encode()) # שליחת
הצלחה הודעת

                print(f"New client joined: {client_address}")
                client_sockets.append(connection)

                with open("primes2.txt", "r") as file:
                    content = file.read().strip()
                    primes = list(map(int, content.split()))

                    random_primes = random.sample(primes, 2)

                    keys = RSA.generate_keypair(random_primes[0],
random_primes[1])

                    client_and_keys[connection] = list(keys)
            else:
                connection, client_address = server_socket.accept()
                connection.send("Denied".encode()) # הודעת שלילית
כישלון

                connection.close()
        else:
            try:
                data =
current_socket.recv(self.MAX_MSG_LENGTH).decode()
            except ConnectionResetError:
                data = ""
            if data == "":
                client_sockets.remove(current_socket)
                current_socket.close()
            else:
                if not data.startswith("GET_SERVER_PUBLIC_KEY"):
                    data = data.split(",")
                    data = [int(n) for n in data]
                    data =
RSA.decrypt(client_and_keys[current_socket][1], data)

                # לרשימה המחזורית פיצול
                data_parts = data.split('@')

                # הרשימה אורך בדיקת
                if len(data_parts) < 1:
                    print("Error: Invalid data format. At least a
command and app_name are required.")
                    continue

                if data_parts[0].isdigit(): # בדיקת העברה
                    content = ""
                    for i in range(int(data_parts[0]) + 1):
                        time.sleep(0.05)
                        content = content +
current_socket.recv(self.MAX_MSG_LENGTH).decode()
                    content = content.split(",")
                    content = [int(n) for n in content]
                    result =
RSA.decrypt(client_and_keys[current_socket][1], content)

```

```

        command = data_parts[1]
        app_name = data_parts[2]
        params = result.split("@")

    else:
        command = data_parts[0] # הפקודה
        try:
            app_name = data_parts[1] # האפליקציה שם
            params = data_parts[2:] # הפרמטרים שאר
        except IndexError:
            try:
                app_name = data_parts[1] # האפליקציה שם
                params = []
            except IndexError:
                app_name = ""
                params = []

    # לפקודה בהתאם ניתוב
    if command == "GET_SERVER_PUBLIC_KEY":
        public_key = client_and_keys[current_socket][0]
        # השרת של הציבורי הערך שמירת
        client_and_keys[current_socket][0] =
ast.literal_eval(app_name) # הלקוח של המפתח את מייצגת
        messages_to_send.append((current_socket,
json.dumps(public_key)))
    elif command == "GET_APPS_NAMES":
        content = fetch_apps_names()
        messages_to_send.append((current_socket,
json.dumps(content)))
    elif command == "CHECK_UNIQUE_NAME":
        content = check_unique_name(app_name)
        messages_to_send.append((current_socket,
json.dumps(content)))
    elif command == "GET_HOMEPAGE_CONTENT":
        content = fetch_homepage_content(app_name)
        messages_to_send.append((current_socket,
json.dumps(content)))
    elif command == "GET_CATALOG_CONTENT":
        content = fetch_catalog_content(app_name)
        messages_to_send.append((current_socket,
json.dumps(content)))
    elif command == "LOG_IN":
        response = handle_login(app_name, *params)
        messages_to_send.append((current_socket,
json.dumps(response)))
    elif command == "CHECK_CONFIRMATION":
        if self.codes[current_socket] == app_name: #
        הראשון הפרמטר זהו כי ככה מיוצג
            messages_to_send.append((current_socket,
"True"))
        else:
            messages_to_send.append((current_socket,
"False"))
    elif command == "SEND_CONFIRMATION_AGAIN":
        code = random.randrange(1000, 10000)
        self.codes[current_socket] = str(code) # עדכון

```

```

לחילון
        message = f"""\
            Your Confirmation Code is: {code}."""
        send_gmail("Join AppSphere", message,
app_name+"@"+params[0]) # אימייל כי יפוצל
        elif command == "INSERT_USER":
            insert_user(app_name, params[0] + "@" +
params[1])
        elif command == "SEND_NEW_PASSWORD":
            forgot_password(app_name,
params[0]+"@"+params[1])
        elif command == "CHECK_GMAIL_IN_APP":
            response = check_gmail_in_app(app_name,
params[0]+"@"+params[1])
            messages_to_send.append((current_socket,
json.dumps(response)))
        elif command == "CHECK_MEETING_AVAILABILITY":
            response = check_meeting_availability(app_name,
*params)
            messages_to_send.append((current_socket,
json.dumps(response)))
        elif command == "BOOK_MEETING":
            response = book_meeting(app_name, *params)
            messages_to_send.append((current_socket,
response))
        elif command == "GET_USERS_MEETINGS":
            content = getting_users_meeting(app_name,
params[0])
            messages_to_send.append((current_socket,
json.dumps(content)))
        elif command == "DELETE_MEETING":
            response = delete_meeting(app_name, *params)
            messages_to_send.append((current_socket,
response))
        elif command == "INSERT_INTO_TABLE":
            table_name = params[0] # הטבלה שם
            if table_name == "Users":
                values_json = params[1] + "@" + params[2] #
לחילון JSON בפורמט החילון
            else:
                values_json = params[1] # JSON בפורמט החילון
                response = self.insert_into_table(app_name,
table_name, values_json, current_socket)
                messages_to_send.append((current_socket,
json.dumps(response)))
        elif command == "CLEAR_APP_DATABASE":
            clear_app_database(app_name)
        elif command == "GET_MEETING_SYSTEM_VARS":
            content = getting_meeting_system_vars(app_name)
            messages_to_send.append((current_socket,
json.dumps(content)))
        elif command == "COUNT_ROWS":
            content = counting_rows(app_name, params[0])
            messages_to_send.append((current_socket,
content))
        elif command == "GET_DESIGN":
            content = getting_design(app_name)

```



```

        messages_to_send.append((current_socket,
content))

        else:
            print(f"Unknown command: {command}")

        if messages_to_send:
            for message in messages_to_send:
                current_socket, data = message
                if current_socket in wlist:
                    data =
RSA.encrypt(client_and_keys[current_socket][0], data)
                    data = ",".join([str(n) for n in data])

                    packets_num_int = len(data.encode('utf-8')) //
self.MAX_MSG_LENGTH

                    if packets_num_int > 0:
                        packets_num =
RSA.encrypt(client_and_keys[current_socket][0], str(packets_num_int))
                        packets_num = ",".join([str(n) for n in
packets_num])

                        current_socket.send(str(packets_num).encode())

                        thread =
threading.Thread(target=self.send_packets(packets_num_int, current_socket,
data))

                        thread.start()
                        # thread.join() # לפני חסתיים Thread-שה מוודא

הקוד המטמ

        else:
            current_socket.send(data.encode())
            messages_to_send.remove(message)

        def send_packets(self, packets_num_int, current_socket, data):
            for i in range(packets_num_int):
                current_socket.send(
                    data[i * self.MAX_MSG_LENGTH:(i + 1) *
self.MAX_MSG_LENGTH].encode())

            if len(data) % self.MAX_MSG_LENGTH != 0:
                current_socket.send(data[packets_num_int *
self.MAX_MSG_LENGTH:].encode())

        def insert_into_table(self, app_name, table_name, table_data,
current_socket):
            table_data = json.loads(table_data)
            # התחנות פטנות n-Base64
            for key in table_data.keys():
                if key.startswith("picture") and table_data.get(key) != "X" and
table_data.get(key) != "Null":
                    table_data[key] = base64.b64decode(table_data[key]) # פטנות

            db_manager = my_database2.DatabaseManager("database.db", app_name)
            result = db_manager.validate_and_insert_data(app_name, table_name,
table_data)

            if table_name == "Users" and not result.startswith("Error"):

```

```

        code = random.randrange(1000, 10000)
        self.codes[current_socket] = str(code) # לחילון עדכון
        message = f"""\
            Your Confirmation Code is: {code}."""
        send_gmail("Join AppSphere", message, table_data["gmail"])

# במסד לטבלה הנתונים הכנסת
return result

# הקיימות האפליקציות שמות לשליפת פונקציה
def fetch_apps_names():
    try:
        conn = sqlite3.connect('database.db')
        cursor = conn.cursor()
        cursor.execute(f"SELECT app_name FROM Apps")
        new_data = cursor.fetchall()
        conn.close()
        return new_data
    except sqlite3.OperationalError:
        return ""

def check_unique_name(app_name):
    try:
        conn = sqlite3.connect('database.db')
        cursor = conn.cursor()
        app_name = app_name.lower() # לפני קטנות לאותיות המחרוזת את הופך
        cursor.execute("SELECT COUNT(*) FROM Apps WHERE LOWER(app_name) = ?",
            (app_name,))
        name_taken = cursor.fetchone()[0]
        conn.close()
        return name_taken
    except sqlite3.OperationalError:
        return 0

def fetch_homepage_content(app_name):
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()
    cursor.execute(f"SELECT * FROM {app_name}_Homepage")
    raw_data = cursor.fetchall()
    conn.close()

    processed_data = []
    for row in raw_data:
        row_list = list(row)
        # עד 4 אינדקסים) התמונות עמודות המחרת (6
        for i in range(4, 7):
            if row_list[i] != "X": # תמונה זו אם רק
                row_list[i] = base64.b64encode(row_list[i]).decode('utf-8')
# קידוד Base64
        processed_data.append(row_list)
    return processed_data

```

```

# Catalog נתוני לשליפת פונקציה
def fetch_catalog_content(app_name):
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()
    cursor.execute(f"SELECT * FROM {app_name}_Catalog")
    raw_data = cursor.fetchall()
    conn.close()

    processed_data = []
    for row in raw_data:
        row_list = list(row)
        # 6 עד 4 אינדקסים) התמונות עמודות המרת
        for i in range(4, 5):
            row_list[i] = base64.b64encode(row_list[i]).decode('utf-8') #
            # Base64 קידוד
        processed_data.append(row_list)
    return processed_data

# משתמשים בהתחברות לטיפול פונקציה
def handle_login(app_name, *input_data):
    email, password = (input_data[0] + "@" + input_data[1]), input_data[2]
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()
    cursor.execute(f"SELECT * FROM {app_name}_Users WHERE gmail = ? AND
password = ?", (email, password))
    user = cursor.fetchone()
    conn.close()
    if user is None:
        return "False"
    return user

def insert_user(app_name, user_var_dict):
    user_vars = json.loads(user_var_dict)
    db_manager = my_database2.DatabaseManager("database.db", app_name)
    db_manager.insert_data(app_name, "Users", user_vars) # שהערכים מכיון
    # הבדיקה את קודם עברו כבר

def forgot_password(app_name, gmail):
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()
    client_name = cursor.execute(f"SELECT name FROM {app_name}_Users WHERE
gmail = ?", (gmail,)).fetchone()[0]
    # הדרישות לפי רנדומלית חזקה סטמה יצירת
    password = "".join(random.sample(string.ascii_uppercase + string.digits +
string.punctuation, 3)) + \
        "".join(random.sample(string.ascii_letters + string.digits +
string.punctuation, 5))
    query = f"UPDATE {app_name}_Users SET password = ? WHERE gmail = ?"
    cursor.execute(query, (hashlib.sha256(password.encode()).hexdigest(),
gmail))
    message = f"""\
        Dear {client_name}

        We're writing to inform you that your password has been

```

```

successfully updated. For security reasons,
    please keep this information confidential.

    Your new password is: {password}

    If you did not request this change or believe this was done
in error,
    please contact our support team immediately.

    Stay safe, The {app_name} Team """

conn.commit()
conn.close()
send_gmail("our Password Has Been Successfully Updated", message, gmail)

def check_gmail_in_app(app_name, gmail):
    conn = sqlite3.connect("database.db")
    cursor = conn.cursor()

    cursor.execute(f"SELECT EXISTS(SELECT 1 FROM {app_name}_Users WHERE gmail
= ?)", (gmail,))

    exists = cursor.fetchone()[0] # קיים אם 1 מחזיר, 0 לא אם
    conn.close()

    return "True" if exists else "False"

# פונקציה לבדיקת פונקציה
def check_meeting_availability(app_name, book_meeting_time):
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()
    cursor.execute(f"SELECT * FROM {app_name}_MeetingSystem WHERE
starting_time_and_date = ?", (book_meeting_time,))
    meeting = cursor.fetchone()
    conn.close()
    if meeting is None:
        return "True"
    return "False"

# פונקציה לקביעת פונקציה
def book_meeting(app_name, *input_data):
    meeting_time1, customer_id = input_data[0], input_data[1]
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()
    cursor.execute(f"INSERT INTO {app_name}_MeetingSystem
(starting_time_and_date, customer_id) VALUES (?, ?)",
    (meeting_time1, customer_id,))
    address = cursor.execute(f"SELECT gmail FROM {app_name}_Users WHERE
item_id = ?", (customer_id,)).fetchone()[0]
    client_name = cursor.execute(f"SELECT name FROM {app_name}_Users WHERE
item_id = ?", (customer_id,)).fetchone()[0]

    message = f"""\
    Dear {client_name},

```

```

        Thank you for scheduling an appointment through {app_name}! We're
happy to confirm that your appointment is
        set for {meeting_time1}.

        If you have any questions or special requests before the meeting,
please feel free to reach out.
        We're here to help!

        Best regards! ""

    conn.commit()
    conn.close()
    send_gmail("Appointment Confirmation", message, address)
    return "True"

def getting_users_meeting(app_name, customer_id):
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()
    cursor.execute(f"SELECT starting_time_and_date FROM
{app_name}_MeetingSystem WHERE customer_id = ?", (customer_id,))
    meetings = cursor.fetchall()
    conn.close()
    return meetings

def delete_meeting(app_name, *input_data):
    user, meeting_time = input_data[0], input_data[1]
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()
    cursor.execute(f"DELETE FROM {app_name}_MeetingSystem WHERE
starting_time_and_date = ? AND customer_id = ?",
                    (meeting_time, user))
    address = cursor.execute(f"SELECT gmail FROM {app_name}_Users WHERE
item_id = ?", (user,)).fetchone()[0]
    client_name = cursor.execute(f"SELECT name FROM {app_name}_Users WHERE
item_id = ?", (user,)).fetchone()[0]
    message = f"""\
        Dear {client_name},

        We regret to inform you that your appointment scheduled
through {app_name} for {meeting_time}
        has been canceled.

        If you would like to reschedule or need any further
assistance, please don't hesitate to reach out.
        We apologize for any inconvenience and appreciate your
understanding.

        Best regards! ""

    conn.commit()
    conn.close()
    send_gmail("Appointment Cancellation", message, address)
    return "True"

```

```
def clear_app_database(app_name):
    db_manager = my_database2.DatabaseManager("database.db", app_name)
    db_manager.clear_database(app_name)
    # הנתונים למסד התחברות
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()

    # מהטבלה הערך למחיקת שאילתה
    cursor.execute("DELETE FROM Apps WHERE app_name = ?", (app_name,))
    conn.commit() # הנתונים במסד השינוי ביצוע
    conn.close()

def getting_meeting_system_vars(app_name):
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()
    # האפליקציה שם לפי הערכים לשליפת שאילתה
    cursor.execute("""SELECT meeting_length, break_length,
meeting_in_between, starting_time, ending_time
FROM Apps WHERE app_name = ?""", (app_name,))
    meeting_system_values = cursor.fetchone()
    conn.close()
    return meeting_system_values

def getting_design(app_name):
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()
    cursor.execute("SELECT design FROM Apps WHERE app_name = ?", (app_name,))
    design = cursor.fetchone()[0]
    conn.close()
    return design

def counting_rows(app_name, table_name):
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()
    cursor.execute(f"SELECT COUNT(*) FROM {app_name}_{table_name}")
    count = cursor.fetchone()
    row_count = count[0] if count else 0
    conn.close()
    return str(row_count)

def send_gmail(subject, message, address):
    try:
        msg = EmailMessage()
        msg.set_content(message)
        msg['Subject'] = subject
        msg['To'] = address

        s = smtplib.SMTP_SSL('smtp.gmail.com', 465)
        s.login("tmrhwpnn@gmail.com", "uhqu dimi aijl vtcb")
        s.send_message(msg)
        s.quit()
```

```

except Exception as e: # אינטרנט חיבור שאין למקרה
    print(e)

if __name__ == '__main__':
    server = Server()

```

:RSA

```

import random
# Euclid's algorithm for determining the greatest common divisor
# Use iteration to make it faster for larger integers

def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

# Euclid's extended algorithm for finding the multiplicative inverse of two
numbers

# Python program to demonstrate working of extended
# Euclidean Algorithm

# function for extended Euclidean Algorithm
def multiplicative_inverse(e, phi):
    d = 0
    x1 = 0
    x2 = 1
    y1 = 1
    temp_phi = phi

    while e > 0:
        temp1 = temp_phi // e
        temp2 = temp_phi - temp1 * e
        temp_phi = e
        e = temp2

        x = x2 - temp1 * x1
        y = d - temp1 * y1

        x2 = x1
        x1 = x
        d = y1
        y1 = y

    if temp_phi == 1:
        return d + phi

# Tests to see if a number is prime.
def is_prime(num):
    if num == 2:
        return True
    if num < 2 or num % 2 == 0:

```

```

        return False
    for n in range(3, int(num*0.5)+2, 2):
        if num % n == 0:
            return False
    return True

def generate_keypair(p1, q1):
    if not (is_prime(p1) and is_prime(q1)):
        raise ValueError('Both numbers must be prime.')
    elif p1 == q1:
        raise ValueError('p and q cannot be equal')
    # n = pq
    n = p1 * q1

    # Phi is the totient of n
    phi = (p1-1) * (q1-1)

    # Choose an integer e such that e and phi(n) are coprime
    e = random.randrange(2, phi)

    # Use Euclid's Algorithm to verify that e and phi(n) are coprime
    g = gcd(e, phi)
    while g != 1:
        e = random.randrange(2, phi)
        g = gcd(e, phi)
    # Use Extended Euclid's Algorithm to generate the private key
    d = multiplicative_inverse(e, phi)

    # Return public and private keypair
    # Public key is (e, n) and private key is (d, n)
    return (e, n), (d, n)

def encrypt(pk, plaintext):
    key, n = pk

    # Unpack the key into it's components
    # Convert each letter in the plaintext to numbers based on the character
    # using a^b mod m
    cipher = [pow(ord(char), key, n) for char in plaintext]
    # cipher = [(ord(char) ** key) % n for char in plaintext]
    # Return the array of bytes
    return cipher

def decrypt(pk, ciphertext):
    # Unpack the key into its components
    key, n = pk
    # Generate the plaintext based on the ciphertext and key using a^b mod m
    plain = [chr(pow(char, key, n)) for char in ciphertext]
    # plain = [chr((char ** key) % n) for char in ciphertext]
    # Return the array of bytes as a string
    return ''.join(plain)

```


:My_database2

```

import sqlite3

class DatabaseManager:
    def __init__(self, db_name, app_name):
        self.db_name = db_name
        self.create_main_tables(app_name)

    def create_main_tables(self, app_name):
        """הראשית האפליקציות טבלת יצירת"""
        con = sqlite3.connect(self.db_name)
        cur = con.cursor()

        # לאפליקציות ראשית טבלה
        cur.execute('''CREATE TABLE IF NOT EXISTS Apps (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        app_name TEXT NOT NULL UNIQUE,
                        design Text NOT NULL,
                        meeting_length INTEGER,
                        break_length INTEGER,
                        meeting_in_between INTEGER,
                        starting_time INTEGER,
                        ending_time INTEGER
                    )''')

        # האפליקציה שם של prefix עם טבלאות יצירת
        app_prefix = f"{app_name}_"

        cur.execute(f'''CREATE TABLE IF NOT EXISTS {app_prefix}Catalog (
                        item_id INTEGER UNIQUE PRIMARY KEY AUTOINCREMENT,
                        product_name TEXT NOT NULL UNIQUE,
                        price INTEGER NOT NULL,
                        description TEXT,
                        picture BLOB NOT NULL
                    )''')

        cur.execute(f'''CREATE TABLE IF NOT EXISTS {app_prefix}Homepage (
                        item_id INTEGER PRIMARY KEY AUTOINCREMENT,
                        block_type TEXT NOT NULL,
                        headline TEXT NOT NULL,
                        content TEXT NOT NULL,
                        picture1 BLOB NOT NULL,
                        picture2 BLOB NOT NULL,
                        picture3 BLOB NOT NULL
                    )''')

        cur.execute(f'''CREATE TABLE IF NOT EXISTS {app_prefix}Users (
                        item_id INTEGER PRIMARY KEY AUTOINCREMENT,
                        name TEXT NOT NULL,
                        gmail TEXT NOT NULL UNIQUE,
                        phone_number INTEGER NOT NULL UNIQUE,
                        password TEXT NOT NULL
                    )''')

        cur.execute(f'''CREATE TABLE IF NOT EXISTS {app_prefix}MeetingSystem

```

```

(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    starting_time_and_date TEXT NOT NULL,
    customer_id INTEGER NOT NULL,
    FOREIGN KEY(customer_id) REFERENCES
{app_prefix}Users(item_id)
)'''

con.commit()
con.close()

def insert_data(self, app_name, table_name, new_data):
    """לאפליקציה מותאמת לטבלה נתונים הוספת"""
    con = sqlite3.connect(self.db_name)
    cur = con.cursor()

    app_prefix = f"{app_name}_"
    placeholders = ', '.join(['?'] * len(new_data))
    columns = ', '.join(new_data.keys())
    values = tuple(new_data.values())

    if table_name != "Apps":
        cur.execute(f'INSERT INTO {app_prefix}{table_name} ({columns})
VALUES ({placeholders})', values)
    else:
        cur.execute(f'INSERT INTO {table_name} ({columns}) VALUES
({placeholders})', values)

    con.commit()
    con.close()

def clear_database(self, app_name):
    con = sqlite3.connect(self.db_name)
    cur = con.cursor()

    # האפליקציה שם של prefix יצירת
    app_prefix = f"{app_name}_"

    # לאפליקציה הקשורות הטבלאות שמות שליופת
    cur.execute(f"SELECT name FROM sqlite_master WHERE type='table' AND
name LIKE '{app_prefix}%'")
    tables = cur.fetchall()

    # שנמצאו הטבלאות כל מחיקת
    for table in tables:
        cur.execute(f"DROP TABLE {table[0]}") # לחלוטין הטבלה מחיקת

    con.commit()
    con.close()

def validate_and_insert_data(self, app_name, table_name, new_data):
    required_columns = {
        'Catalog': {'product_name': [str, " NOT NULL", "UNIQUE"],
'price': [int, " NOT NULL"],
'description': [str], 'picture': [bytes, " NOT
NULL"]},
        'Homepage': {'block_type': [str, " NOT NULL"], 'headline': [str,

```

```

" NOT NULL"],
        'content': [str, " NOT NULL"],
        'picture1': [bytes, " NOT NULL"], 'picture2':
[bytes, " NOT NULL"],
        'picture3': [bytes, " NOT NULL"]},
    'Users': {'name': [str, " NOT NULL"], 'gmail': [str, " NOT NULL",
"UNIQUE"],
        'phone_number': [int, " NOT NULL", "UNIQUE"],
'password': [str, " NOT NULL"]},
    'MeetingSystem': {'starting_time_and_date': [str, " NOT NULL"],
'customer_id': [int, " NOT NULL"]},
    'Apps': {'app_name': [str, 'NOT NULL', 'UNIQUE'], 'design': [str,
'NOT NULL'], 'meeting_length': [int],
        'break_length': [int], 'meeting_in_between': [int],
'starting_time': [int], 'ending_time': [int]}
    }

    if table_name in required_columns:
        if table_name != "Apps":
            app_prefix = f"{app_name}_"
            full_table_name = f"{app_prefix}{table_name}"
        else:
            full_table_name = f"{table_name}"
        columns_info = required_columns[table_name]
        missing_columns = [col for col in columns_info if col not in
new_data]
        empty_values = all(value == '' or value == 'Null' or value == 'X'
or
                            value is None for value in new_data.values())

        if empty_values:
            return "Error: All values are empty. No data inserted."

        if missing_columns:
            return f"Error: Missing columns {missing_columns} for table
{full_table_name}."

        for col, expected_type in columns_info.items():
            if col in new_data:
                # Validate NOT NULL constraint
                if " NOT NULL" in columns_info[col] and (new_data[col] ==
'' or new_data[col] is None or
                                                            new_data[col] ==
"Null"):
                    return f"Error: Column '{col}' in table
{full_table_name} cannot be NULL."

                # Validate UNIQUE constraint
                if "UNIQUE" in columns_info[col]:
                    con = sqlite3.connect(self.db_name)
                    cur = con.cursor()
                    cur.execute(f'SELECT {col} FROM {full_table_name}
WHERE {col} = ?', (new_data[col],))
                    if cur.fetchone():
                        con.close()
                        return (f"Error: Duplicate value
'{new_data[col]}' found for unique column ")

```

```

        f"'{col}' in table {full_table_name}."
    con.close()

    # Validate type
    if expected_type[0] == int:
        try:
            new_data[col] = int(new_data[col])
        except ValueError:
            pass
        if not isinstance(new_data[col], expected_type[0]) and
new_data[col] != "X":
            return (f"Error: Column '{col}' in table
{full_table_name} should be of type "
                    f"{expected_type[0].__name__}.")

    con = sqlite3.connect(self.db_name)
    cur = con.cursor()

    # Check for uniqueness constraints
    cur.execute(f'PRAGMA table_info({full_table_name})')
    table_info = cur.fetchall()
    unique_columns = [info[1] for info in table_info if info[5]] #
Column names where 'unique' is true

    for col in unique_columns:
        if col in new_data:
            cur.execute(f'SELECT {col} FROM {full_table_name} WHERE
{col} = ?', (new_data[col],))
            if cur.fetchone():
                con.close()
                return (f"Error: Duplicate value '{new_data[col]}'
found for unique column '{col}' "
                        f"in table {full_table_name}.")

    con.close()
    if table_name != "Users": # אימות אזורי יותר מאוחר בשלב יוֹכנס
        self.insert_data(app_name, table_name, new_data) # שימוש
המעודכנת insert_data בורסת

    return f"Data successfully inserted into {full_table_name}!"

    else:
        return f"Error: Table {table_name} does not exist for app
'{app_name}'."

```

מצד הלקוח:

:Main

```

import customtkinter as ctk
import OpeningFrame
import json
import threading
import client2

```

```

def before_retry(root):
    root.after(0, lambda: retry(root)) # הראשי Thread-ל קריאה

def retry(root):
    for widget in root.winfo_children():
        try:
            widget.destroy()
        except AttributeError:
            pass
    create_gui(root)

def main():
    root = ctk.CTk() # open window
    root.title("AppSphere")
    root.geometry("%dx%d+0+0" % (root.winfo_screenwidth()-20,
root.winfo_screenheight()-70)) # size: fullscreen
    create_gui(root)

def create_gui(root):
    ctk.set_appearance_mode("light") # Modes: system (default), light,
dark
    design = "cherry_design"
    ctk.set_default_color_theme(f"designs/{design}.json")
    j = json.load(open(f"designs/{design}.json"))
    frame = ctk.CTkScrollableFrame(root, fg_color=(j["CTk"]["fg_color"]))
    my_client = client2.Client(root, before_retry)
    if my_client.status == "Connected":
        t = threading.Thread(target=OpeningFrame.OpeningFrame, args=(root,
frame, my_client, ))
        t.start()
    else:
        error_label = ctk.CTkLabel(root, text="Error!\n Please try later",
font=("Black Ariel", 80))
        error_label.pack(pady=30)
        retry_button = ctk.CTkButton(root, text="Retry", command=lambda:
retry(root))
        retry_button.pack(pady=30)

    try:
        root.mainloop()
    except KeyboardInterrupt:
        if my_client.status == "Connected":
            my_client.client_socket.close()
            print("connection closed")

if __name__ == '__main__':
    main()

```

:Client2

```

import socket
import json
import base64

```

```

import RSA
import random
import threading
from PIL import Image
import io
import time

class Client:
    def __init__(self, root, retry):
        self.SERVER_IP = '127.0.0.1'
        self.SERVER_PORT = 1729
        self.MAX_MSG_LENGTH = 10240

        self.root = root
        self.retry_act = retry
        self.lock = threading.Lock() # הרשת קריאות לכלל משותף מנעול

        self.client_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        try:
            self.client_socket.connect((self.SERVER_IP, self.SERVER_PORT))

            self.status =
self.client_socket.recv(self.MAX_MSG_LENGTH).decode()

            if self.status == "Connected":
                with open("primes2.txt", "r") as file:
                    content = file.read().strip()
                    primes = list(map(int, content.split()))

                    random_primes = random.sample(primes, 2)
                    self.keys = RSA.generate_keypair(random_primes[0],
random_primes[1])
                    self.server_key = self.get_server_public_key()

            except ConnectionRefusedError:
                self.status = "Denied"

        def get_server_public_key(self):
            with self.lock:
                try:
self.client_socket.send(f"GET_SERVER_PUBLIC_KYE@{self.keys[0]}".encode())
                except (socket.error, ConnectionRefusedError, BrokenPipeError,
ConnectionResetError):
                    self.retry_act(self.root)
                    self.client_socket.close()
                    if threading.current_thread() != threading.main_thread():
                        exit() # משני החרד אם רק יוצא
                    content = self.recv_message(0000000)
                    return content

        def send_message(self, message):
            try:
                send_data = RSA.encrypt(self.server_key, message)
                send_data = ",".join([str(n) for n in send_data])

```

```

        self.client_socket.send(send_data.encode())
    except (socket.error, ConnectionRefusedError, BrokenPipeError,
ConnectionResetError):
        self.retry_act(self.root)
        self.client_socket.close()
        if threading.current_thread() != threading.main_thread():
            exit() # משני התרד אם רק יוצא

def recv_message(self, default):
    try:
        response = self.client_socket.recv(self.MAX_MSG_LENGTH).decode()
        response = response.split(",")
        response = [int(n) for n in response]
        try:
            content = json.loads(RSA.decrypt(self.keys[1], response))
        except json.JSONDecodeError:
            content = RSA.decrypt(self.keys[1], response)
        return content
    except (socket.error, ConnectionRefusedError, BrokenPipeError,
ConnectionResetError):
        self.retry_act(self.root)
        self.client_socket.close()
        if threading.current_thread() != threading.main_thread():
            exit() # משני התרד אם רק יוצא
    else:
        return default

def recv_packets_and_change_to_original(self, table):
    packets_num = self.recv_message(0)
    content = ""
    if type(packets_num) is int: # בפקטות בשימוש צורך יהיה שלא למקרה,
נדיר מאוד
        for i in range(int(packets_num) + 1):
            time.sleep(0.05)
            try:
                content = content +
self.client_socket.recv(self.MAX_MSG_LENGTH).decode()
            except (socket.error, ConnectionRefusedError,
BrokenPipeError, ConnectionResetError):
                self.retry_act(self.root)
                self.client_socket.close()
                exit()

            content = content.split(",")
            content = [int(n) for n in content]
            raw_content = json.loads(RSA.decrypt(self.keys[1], content))
        else:
            raw_content = json.loads(packets_num)

    processed_content = []
    for row in raw_content:
        row_list = row
        if table == "Homepage":
            last_index = 7
        else:
            last_index = 5
        for i in range(4, last_index): # התמונות עמודות פיענוח

```

```

        if row_list[i] != "X" and row_list[i] != "Null":
            row_list[i] = base64.b64decode(row_list[i]) # פטנטון
Base64
        processed_content.append(row_list)

    return processed_content

def get_apps_names(self):
    with self.lock:
        self.send_message(f"GET_APPS_NAMES")
        return self.recv_message([])

def check_unique_name(self, app_name):
    with self.lock:
        self.send_message(f"CHECK_UNIQUE_NAME@{app_name}")
        return self.recv_message(1)

def get_homepage_content(self, app_name):
    with self.lock:
        self.send_message(f"GET_HOMEPAGE_CONTENT@{app_name}")
        return self.recv_packets_and_change_to_original("Homepage")

def get_catalog_content(self, app_name):
    with self.lock:
        self.send_message(f"GET_CATALOG_CONTENT@{app_name}")
        return self.recv_packets_and_change_to_original("Catalog")

def login_to_system(self, app_name, email, password):
    with self.lock:
        self.send_message(f"LOG_IN@{app_name}@{email}@{password}")
        return self.recv_message("False")

def check_confirmation(self, code):
    with self.lock:
        self.send_message(f"CHECK_CONFIRMATION@{code}")
        return self.recv_message("False")

def send_confirmation_again(self, gmail):
    with self.lock:
        self.send_message(f"SEND_CONFIRMATION_AGAIN@{gmail}")

def insert_user(self, app_name, user_var):
    with self.lock:
self.send_message(f"INSERT_USER@{app_name}@{json.dumps(user_var)}")

    def send_new_password(self, app_name, user_gmail):
        with self.lock:
            self.send_message(f"SEND_NEW_PASSWORD@{app_name}@{user_gmail}")

    def check_gmail_in_app(self, app_name, gmail):
        with self.lock:
            self.send_message(f"CHECK_GMAIL_IN_APP@{app_name}@{gmail}")
            return self.recv_message("False")

    def check_meeting_availability(self, app_name, meeting_time):
        with self.lock:

```



```

self.send_message(f"CHECK_MEETING_AVAILABILITY@{app_name}@{meeting_time}")
return self.recv_message("False")

def book_meeting(self, app_name, meeting_time, customer_id):
    with self.lock:
self.send_message(f"BOOK_MEETING@{app_name}@{meeting_time}@{customer_id}")
return self.recv_message("False")

def get_users_meetings(self, app_name, customer_id):
    with self.lock:
        self.send_message(f"GET_USERS_MEETINGS@{app_name}@{customer_id}")
        return self.recv_message([])

def delete_meeting(self, app_name, user, meeting_time):
    with self.lock:
self.send_message(f"DELETE_MEETING@{app_name}@{user}@{meeting_time[0]}")
return self.recv_message("False")

def insert_into_table(self, app_name, table_name, values):
    with self.lock:
        # ל-Base64 התמונות קידוד
        for key in values.keys():
            if key.startswith("picture") and values[key] != "X" and
values[key] != "Null":
                # ואינו "picture"-ב מתחיל העמודה שם אם בדיקת
                values[key] = reduce_image_binary(values[key])
                values[key] = base64.b64encode(values[key]).decode('utf-
8') # ל-Base64 קידוד

        send_values = RSA.encrypt(self.server_key,
f"{table_name}@{json.dumps(values)}")
        send_values = ",".join([str(n) for n in send_values])
        packets_num = len(send_values.encode('utf-8')) //
self.MAX_MSG_LENGTH

        self.send_message(f"{packets_num}@INSERT_INTO_TABLE@{app_name}")

        for i in range(packets_num):
            try:
                self.client_socket.send(send_values[i *
self.MAX_MSG_LENGTH:(i + 1) * self.MAX_MSG_LENGTH].encode())
            except (socket.error, ConnectionRefusedError,
BrokenPipeError, ConnectionResetError):
                self.retry_act(self.root)
                self.client_socket.close()
            if len(send_values) % self.MAX_MSG_LENGTH != 0:
                try:
                    self.client_socket.send(send_values[packets_num *
self.MAX_MSG_LENGTH:].encode())
                except (socket.error, ConnectionRefusedError,
BrokenPipeError, ConnectionResetError):
                    self.retry_act(self.root)
                    self.client_socket.close()

```

```

        return self.recv_message("Error: Server is not connected")

    def clear_app_database(self, app_name):
        with self.lock:
            self.send_message(f"CLEAR_APP_DATABASE@{app_name}")

    def get_meeting_system_vars(self, app_name):
        with self.lock:
            self.send_message(f"GET_MEETING_SYSTEM_VARS@{app_name}")
            return self.recv_message([0, 0, 0, 0, 0])

    def get_design(self, app_name):
        with self.lock:
            self.send_message(f"GET_DESIGN@{app_name}")
            return self.recv_message("cherry")

    def count_rows(self, app_name, table_name):
        with self.lock:
            self.send_message(f"COUNT_ROWS@{app_name}@{table_name}")
            return self.recv_message(0)

def reduce_image_binary(image_bytes, target_size_kb=15):
    """
    תמונה ומחזיר, הגודל את מקטין, צריך אם JPG-ל ממיר, בינארית תמונה מקבל
    """
    image = Image.open(io.BytesIO(image_bytes)) # מבינארי התמונה טעינת

    # ל-JPG והמרה PNG-ב מדובר אם בדיקה
    if image.format == "PNG":
        image = image.convert("RGB") # שקיפות ערוץ הסרת
        format_pic = "JPEG" # ל-PNG פורמט המרת
    else:
        format_pic = image.format # לא הוא אם המקורי הפורמט את שומר

    # ראשונית איכות קביעת
    quality = 95

    while True:
        buffer = io.BytesIO()
        image.save(buffer, format=format_pic, quality=quality, optimize=True)
        size_kb = buffer.tell() / 1024 # קובץ גודל מחשבים

        if size_kb <= target_size_kb or quality <= 10:
            break # נמוכה כבר התמונה איכות אם או מתאים הגודל אם עוצרים

        quality -= 5 # גודל להקטין כדי בהדרגה איכות הפחתת

    return buffer.getvalue() # בינארי בפורמט התמונה את מחזיר

```

:RSA

```

import random
# Euclid's algorithm for determining the greatest common divisor
# Use iteration to make it faster for larger integers

def gcd(a, b):

```

```

    while b != 0:
        a, b = b, a % b
    return a

# Euclid's extended algorithm for finding the multiplicative inverse of two
numbers

# Python program to demonstrate working of extended
# Euclidean Algorithm

# function for extended Euclidean Algorithm
def multiplicative_inverse(e, phi):
    d = 0
    x1 = 0
    x2 = 1
    y1 = 1
    temp_phi = phi

    while e > 0:
        temp1 = temp_phi // e
        temp2 = temp_phi - temp1 * e
        temp_phi = e
        e = temp2

        x = x2 - temp1 * x1
        y = d - temp1 * y1

        x2 = x1
        x1 = x
        d = y1
        y1 = y

    if temp_phi == 1:
        return d + phi

# Tests to see if a number is prime.
def is_prime(num):
    if num == 2:
        return True
    if num < 2 or num % 2 == 0:
        return False
    for n in range(3, int(num**0.5)+2, 2):
        if num % n == 0:
            return False
    return True

def generate_keypair(p1, q1):
    if not (is_prime(p1) and is_prime(q1)):
        raise ValueError('Both numbers must be prime.')
    elif p1 == q1:
        raise ValueError('p and q cannot be equal')
    # n = pq
    n = p1 * q1

```

```

# Phi is the totient of n
phi = (p1-1) * (q1-1)

# Choose an integer e such that e and phi(n) are coprime
e = random.randrange(2, phi)

# Use Euclid's Algorithm to verify that e and phi(n) are coprime
g = gcd(e, phi)
while g != 1:
    e = random.randrange(2, phi)
    g = gcd(e, phi)
# Use Extended Euclid's Algorithm to generate the private key
d = multiplicative_inverse(e, phi)

# Return public and private keypair
# Public key is (e, n) and private key is (d, n)
return (e, n), (d, n)

def encrypt(pk, plaintext):
    key, n = pk

    # Unpack the key into its components
    # Convert each letter in the plaintext to numbers based on the character
    # using a^b mod m
    cipher = [pow(ord(char), key, n) for char in plaintext]
    # cipher = [(ord(char) ** key) % n for char in plaintext]
    # Return the array of bytes
    return cipher

def decrypt(pk, ciphertext):
    # Unpack the key into its components
    key, n = pk
    # Generate the plaintext based on the ciphertext and key using a^b mod m
    plain = [chr(pow(char, key, n)) for char in ciphertext]
    # plain = [chr((char ** key) % n) for char in ciphertext]
    # Return the array of bytes as a string
    return ''.join(plain)

```

:OpeningFrame

```

import customtkinter as ctk
import CreatorFrame
import threading
import Tools
from PIL import Image

class OpeningFrame(ctk.CTkFrame):
    def __init__(self, root, background_frame, my_client):
        super().__init__(background_frame)
        self.root = root
        self.background_frame = background_frame
        self.client = my_client
        self.create_frame()

```

```

def go_to_creator_frame(self):
    self.destroy()
    t = threading.Thread(target=CreatorFrame.SettingsFrame,
args=(self.root, self.background_frame, self.client))
    t.start()

def go_to_app(self, var):
    self.background_frame.pack_forget()
    t = threading.Thread(target=Tools.Tools, args=(var, self.root,
self.client))
    t.start()

def create_frame(self):
    mode_switch = CreatorFrame.ColorMode(self.background_frame)
    mode_switch.mode_switch.place(x=10, y=10)

    self.background_frame.pack(fill="both", expand=True)
    self.background_frame.columnconfigure((0, 2), weight=1)

    self.grid(row=0, column=1, pady=10)
    self.columnconfigure((0, 3), weight=1)

    headline = ctk.CTkLabel(self, text="Welcome to AppSphere",
font=("Harrington", 70))
    headline.grid(row=0, column=1, columnspan=2, padx=40, pady=20)

    create_your_own_button = ctk.CTkButton(self, text="Create your own
app", command=self.go_to_creator_frame,
width=250, height=100,
font=("Ariel", 30))
    create_your_own_button.grid(row=1, rowspan=2, column=1, padx=60,
pady=40)

    go_to_app_label = ctk.CTkLabel(self, text="Please choose the app you
want to get into", font=("Ariel", 24))
    go_to_app_label.grid(row=1, column=2, padx=50, pady=30)

    try:
        result = self.client.get_apps_names()
        # בלבד מחזירות של לרשימה התוצאה המרת
        app_names = [row[0] for row in result]

        go_to_app_menu = ctk.CTkOptionMenu(self, values=app_names,
command=lambda x: self.go_to_app(x), width=200,
height=50, font=("Ariel", 20))
        go_to_app_menu.grid(row=2, column=2, padx=20, pady=20)
    except Exception as e:
        print("in opening frame ", e)

    pic = Image.open("random_pictures/computer_thingy-picture.png")
    picture = ctk.CTkLabel(self, text="",
image=ctk.CTkImage(light_image=pic,
size=(pic.size[0]-2500, pic.size[1]-2500)))
    picture.grid(row=3, column=1, columnspan=2, padx=50, pady=50)

```

:CreatorFrame

```

import customtkinter as ctk
from PIL import Image
import InputDatabaseFrame2

class ColorMode:
    def __init__(self, background_frame):
        self.mode = "dark"
        self.mode_switch = ctk.CTkSwitch(master=background_frame, text="Dark
Mode", onvalue="dark", offvalue="light",
                                         bg_color="transparent",
command=lambda: self.choose_mode(
self.mode_switch.get()))

    def choose_mode(self, var):
        self.mode = var
        ctk.set_appearance_mode(self.mode) # Modes: system (default), light,
dark

class SettingsFrame(ctk.CTkFrame):
    def __init__(self, root, background_frame, my_client):
        super().__init__(background_frame)
        self.root = root
        self.client = my_client
        self.background_frame = background_frame
        self.check_boxs_var = [ctk.StringVar(value="off"),
ctk.StringVar(value="off"), ctk.StringVar(value="off")]
        self.chosen_designs = ["design1", "design1", "design1"]
        self.color = "default"
        self.pictures = [{"template/Homepage_template_design_1.PNG",
"template/Homepage_template_design_2.PNG",
                        "template/Homepage_template_design_2.PNG"},
                        ["template/Catalog_template_design_1.PNG",
"template/Catalog_template_design_2.PNG",
                        "template/Catalog_template_design_3.PNG"],
                        ["template/meeting_system_design.png"]]
        self.colors = ["default", "autumn", "cherry", "coffee", "green",
"purple", "rime"]
        self.pic = [{"f"color_designs_pic/metal-light.png",
f"color_designs_pic/autumn-light.png",
                        f"color_designs_pic/cherry-light.png",
f"color_designs_pic/coffee-light.png",
                        f"color_designs_pic/marsh-light.png",
f"color_designs_pic/violet-light.png",
                        f"color_designs_pic/rime-light.png"},
                        [f"color_designs_pic/metal-dark.png",
f"color_designs_pic/autumn-dark.png",
                        f"color_designs_pic/cherry-dark.png",
f"color_designs_pic/coffee-dark.png",
                        f"color_designs_pic/marsh-dark.png",
f"color_designs_pic/violet-dark.png",
                        f"color_designs_pic/rime-dark.png"]]
        self.database_frame = ctk.CTkFrame(background_frame)

```

```

        self.create_settings_frame()

    def print_pic(self):
        for i in range(len(self.pictures)):
            label_pic = ctk.CTkLabel(self,
            image=ctk.CTkImage(light_image=Image.open(self.pictures[i][0]),
                                size=(200, (3-
            i)*120)), text="")
            label_pic.grid(row=3, column=1+i, padx=20, pady=20, sticky="N")

    def choose_design(self, var, index):
        self.chosen_designs[index] = var
        label_pic = ctk.CTkLabel(self,
            image=ctk.CTkImage(light_image=Image.open(self.pictures[index][int(var[-1])-
            1])),
                                size=(200, (3-
            index)*120)), text="")
        label_pic.grid(row=3, column=1 + index, padx=20, pady=20, sticky="N")

    def choose_color(self, var):
        self.color = str(var)
        i = self.colors.index(self.color)
        self.print_color_pic(i)

    def print_color_pic(self, i):
        light_pic = ctk.CTkLabel(self,
            image=ctk.CTkImage(light_image=Image.open(self.pic[0][i]),
                                size=(200, 100)),
            text="")
        light_pic.grid(row=5, column=2)
        dark_pic = ctk.CTkLabel(self,
            image=ctk.CTkImage(light_image=Image.open(self.pic[1][i]),
                                size=(200, 100)),
            text="")
        dark_pic.grid(row=5, column=3)

    def get_database_tables(self):
        self.database_frame.destroy()
        self.database_frame = InputDatabaseFrame2.InputDatabaseFrame(self,
        self.client)
        self.database_frame.create_tables()

    def create_settings_frame(self):
        mode_switch = ColorMode(self.background_frame)
        mode_switch.mode_switch.place(x=10, y=10)

        self.background_frame.pack(fill="both", expand=True)
        self.background_frame.columnconfigure((0, 2), weight=1)

        self.grid(row=0, column=1, pady=10)
        self.columnconfigure((0, 4), weight=1)

        headline = ctk.CTkLabel(self, text="Create Your Own App",
        font=("Ariel Black", 70))
        headline.grid(row=0, column=1, columnspan=3, padx=40, pady=20)

        explanation = ctk.CTkLabel(self, text="This is where you choose how

```

```

your app will look. "
                                "You can choose what your app
will contain,"
                                " the design of each part and
the color theme. ")
    explanation.grid(row=1, column=1, columnspan=3, padx=20, pady=10)

    self.print_pic()

    check_boxes = [ctk.CTkCheckBox(self, text="Homepage",
variable=self.check_boxes_var[0], onvalue="on",
                                offvalue="off"),
                    ctk.CTkCheckBox(self, text="Catalog",
variable=self.check_boxes_var[1], onvalue="on",
                                offvalue="off"),
                    ctk.CTkCheckBox(self, text="Meeting System",
variable=self.check_boxes_var[2], onvalue="on",
                                offvalue="off")]

    for i in range(len(check_boxes)):
        check_boxes[i].grid(row=2, column=1+i, padx=20, pady=20)

    designs = ["design1", "design2", "design3"]

    designs_options = [ctk.CTkOptionMenu(self, values=designs,
command=lambda x: self.choose_design(x, 0)),
                       ctk.CTkOptionMenu(self, values=designs,
command=lambda x: self.choose_design(x, 1))]

    for i in range(len(designs_options)):
        designs_options[i].grid(row=4, column=1+i, padx=20, pady=20)

    color_options = ctk.CTkOptionMenu(self, values=self.colors,
command=lambda x: self.choose_color(x))
    color_options.grid(row=5, column=1)
    self.print_color_pic(0)

    sub_button = ctk.CTkButton(self, text="Submit",
command=self.get_database_tables)
    sub_button.grid(row=7, column=2, padx=20, pady=40)

```

:InputDatabaseFrame

```

import customtkinter as ctk
import Tools
from tkinter import filedialog as fd
from tkinter.messagebox import showinfo
import threading
from functools import partial

class CreateTable(ctk.CTkFrame):
    def __init__(self, background_frame, var_arr, headlines):
        super().__init__(background_frame)
        self.next_row = 0
        self.columns = len(headlines)
        self.picture_places = []

```



```

self.textbox_places = []
for i in range(len(headlines)):
    if headlines[i].startswith("Picture"):
        self.picture_places.append(i)
    elif headlines[i] == "Content":
        self.textbox_places.append(i)
self.var_arr = var_arr
self.headlines = headlines
self.plus_button = ctk.CTkButton(self)
self.arr = []
self.choose_block = ctk.CTkOptionMenu(self)
self.print_column_headlines()

def print_column_headlines(self):
    for i in range(len(self.headlines)):
        label = ctk.CTkLabel(self, text=self.headlines[i], font=("Arial
Black", 14))
        label.grid(row=0, column=i, pady=10, padx=10)
        self.next_row += 1
    self.create_new_row()

def select_file(self, arr, row_label, i):
    filetypes = (('picture file', '*.jpg'), ('picture file', '*.png'),
('All files', '.'))
    file_name = fd.askopenfilename(title='Open a file', initialdir='/',
filetypes=filetypes)
    showinfo(title='Selected File', message=file_name)
    try:
        with open(file_name, 'rb') as file:
            binary_content = file.read()
            arr[i] = binary_content
            label = ctk.CTkLabel(self, text=file_name.split("/")[-
1]+"/"+file_name.split("/")[-2], padx=10)
            label.grid(row=row_label, column=i, sticky="N", padx=0)
    except (FileNotFoundError, IndexError):
        pass

def blocks(self, row, block):
    if block == "left picture_and_content" or block ==
"right picture_and_content":
        self.arr[4] = "X"
        self.arr[5] = "X"
    elif block == "only_content":
        self.arr[3] = "X"
        self.arr[4] = "X"
        self.arr[5] = "X"
    else:
        self.arr[2] = "X"
    block_type = ctk.CTkLabel(self, text=block)
    block_type.grid(row=row, column=0, sticky="N", padx=10)
    self.choose_block.destroy()
    self.arr[0] = block
    self.print_row()

def create_new_row(self):
    self.plus_button.destroy()
    self.arr = []

```

```

        for i in range(self.columns):
            if i in self.textbox_places:
                self.arr.append(ctlk.CTkTextbox(self, height=60, width=160))
            elif i in self.picture_places:
                self.arr.append("Null")
            else:
                self.arr.append(ctlk.CTkEntry(self))
            if self.headlines[0] == "Block type":
                self.choose_block = ctlk.CTkOptionMenu(self,
values=["left_picture_and_content", "only_content",
"pictures_only", "right_picture_and_content"],
command=partial(self.blocks, self.next_row))
                self.choose_block.grid(row=self.next_row, column=0, sticky="N")
            else:
                self.print_row()

        def print_row(self):
            for i in range(len(self.arr)):
                if self.arr[i] == "X" or self.arr[i] in
["left_picture_and_content", "only_content", "pictures_only",
"right_picture_and_content"]:
                    pass
                elif i in self.picture_places:
                    open_button = ctlk.CTkButton(self, text='Upload picture',
command=partial(
                        self.select_file, self.arr, self.next_row, i))
                    open_button.grid(row=self.next_row, column=i, sticky="N")
                else:
                    self.arr[i].grid(row=self.next_row, column=i, sticky="N")
                    self.var_arr.append(self.arr)
                    self.next_row += 1
                    self.plus_button = ctlk.CTkButton(self, text="+",
command=self.create_new_row, width=60,
font=("Ariel Black", 18))
                    self.plus_button.grid(row=self.next_row, column=0,
columnspan=len(self.headlines))

class InputDatabaseFrame(ctlk.CTkFrame):
    def __init__(self, settings_frame, my_client):
        super().__init__(settings_frame.background_frame)
        self.settings_frame = settings_frame
        self.client = my_client
        self.next_row = 0
        self.homepage_values = []
        self.catalog_values = []
        self.meeting_system_values = []
        self.input_update_frame = ctlk.CTkFrame(self)
        self.name_entry = ctlk.CTkEntry(self)

    def homepage_table(self):
        headline = ctlk.CTkLabel(self, text="Homepage Content", font=("Ariel
Black", 25))

```

```

        headline.grid(row=self.next_row, column=1, columnspan=2, padx=10,
pady=10)
        self.next_row += 1
        label1 = ctk.CTkLabel(self, text="Choose the frame type and fill the
content.\n"
                                "Notice that the order of the frames
matters. ")
        label1.grid(row=self.next_row, column=1, columnspan=2, padx=30,
pady=10)
        self.next_row += 1

        homepage_table = CreateTable(self, self.homepage_values, ["Block
type", "Headline", "Content",
"Picture1", "Picture2", "Picture3"])
        homepage_table.grid(row=self.next_row, column=1, columnspan=2,
padx=20, pady=10)
        self.next_row += 1

        def catalog_table(self):
            headline = ctk.CTkLabel(self, text="Catalog Content", font=("Ariel
Black", 25))
            headline.grid(row=self.next_row, column=1, columnspan=2, padx=10,
pady=10)
            self.next_row += 1
            label = ctk.CTkLabel(self, text="Enter all of your product's details
according to the columns.\n "
                                "Notice that the order of the
products matters. ")
            label.grid(row=self.next_row, column=1, columnspan=2, padx=10,
pady=10)
            self.next_row += 1

            catalog_table = CreateTable(self, self.catalog_values, ["Product
name", " Price ($)", "Description", "Picture"])
            catalog_table.grid(row=self.next_row, column=1, columnspan=2,
padx=30, pady=10)
            self.next_row += 1

        def change_hours(self, i, new_time):
            self.meeting_system_values[3 + i] = self.meeting_system_values[3 + i]
% 60 + int(new_time)*60

        def change_minutes(self, i, new_time):
            self.meeting_system_values[3 + i] = (self.meeting_system_values[3 +
i] - self.meeting_system_values[3 + i] % 60
                                                + int(new_time))

        def meeting_system_table(self):
            headline = ctk.CTkLabel(self, text="Meeting System Setting",
font=("Ariel Black", 25))
            headline.grid(row=self.next_row, column=1, columnspan=2, padx=10,
pady=10)
            self.next_row += 1

            frame = ctk.CTkFrame(self)
            frame.grid(row=self.next_row, column=1, columnspan=2, padx=10,

```

```

pady=10)
    label1 = ctk.CTkLabel(frame, text="Please enter the length of each
meeting in units of minutes. ")
    label1.grid(row=0, column=0, columnspan=3, padx=20)
    self.meeting_system_values.append(ctk.CTkEntry(frame))
    self.meeting_system_values[0].grid(row=1, column=0, columnspan=3,
padx=10, pady=10)

    label2 = ctk.CTkLabel(frame, text="How mach time are the breaks? (In
units of minutes)")
    label2.grid(row=2, column=0, padx=10)
    label3 = ctk.CTkLabel(frame, text="Every how many meetings do you
take a break? ")
    label3.grid(row=2, column=1, padx=10)
    for i in range(2):
        self.meeting_system_values.append(ctk.CTkEntry(frame))
        self.meeting_system_values[1+i].grid(row=3, column=0+i, padx=10,
pady=10)
    min_start_time = ctk.CTkLabel(frame, text="Please enter min start
time to the first meeting")
    min_start_time.grid(row=4, column=0, padx=10)
    max_ending_time = ctk.CTkLabel(frame, text="Please enter max end time
to the last meeting")
    max_ending_time.grid(row=5, column=0, padx=10)
    for i in range(2):
        clock_frame = ctk.CTkFrame(frame)
        clock_frame.grid(row=4+i, column=1)
        self.meeting_system_values.append(0)
        hours = ctk.CTkOptionMenu(clock_frame, values=["00", "01", "02",
"03", "04", "05", "06", "07", "08", "09",
"10", "11", "12",
"13", "14", "15", "16", "17", "18", "19",
"20", "21", "22",
"23"],
                                command=partial(self.change_hours, i),
width=30, bg_color=frame.cget("fg_color"))
        hours.grid(row=0, column=0)
        label = ctk.CTkLabel(clock_frame, text=":", font=("Ariel Black",
25), bg_color=frame.cget("fg_color"))
        label.grid(row=0, column=1)
        minutes = ctk.CTkOptionMenu(clock_frame, values=["00", "15",
"30", "45"], width=30,
                                command=partial(self.change_minutes,
i), bg_color=frame.cget("fg_color"))
        minutes.grid(row=0, column=2)
        self.next_row += 1

    def insert_arr_to_database(self):
        self.input_update_frame.destroy()
        self.input_update_frame = ctk.CTkFrame(self)
        self.input_update_frame.grid(row=self.next_row, column=1,
columnspan=2)
        row = 0
        error = False
        if self.name_entry.get() == '' or self.name_entry.get() is None:
            row += 1

```

```

        label = ctk.CTkLabel(self.input_update_frame, text="Please enter
the app's name",
                             text_color="red", padx=10)
        label.grid(row=row, column=0)
        elif self.client.check_unique_name(self.name_entry.get()) != 0:
            row += 1
            label = ctk.CTkLabel(self.input_update_frame, text="Please enter
another name, this one is already taken",
                             text_color="red", padx=10)
            label.grid(row=row, column=0)
        elif " " in self.name_entry.get():
            row += 1
            label = ctk.CTkLabel(self.input_update_frame, text="Name can't
contain spaces",
                             text_color="red", padx=10)
            label.grid(row=row, column=0)
        else:
            if len(self.meeting_system_values) > 0:
                label = ctk.CTkLabel(self.input_update_frame,
text=self.client.insert_into_table(self.name_entry.get(),
'Apps', {
                    'app_name': self.name_entry.get(),
                    'design': self.settings_frame.color,
                    'meeting_length': self.meeting_system_values[0].get(),
                    'break_length': self.meeting_system_values[1].get(),
                    'meeting_in_between':
self.meeting_system_values[2].get(),
                    'starting_time': self.meeting_system_values[3],
                    'ending_time': self.meeting_system_values[4]}),
text_color="red", padx=10)
                row += 1
                if (label.cget("text").startswith("Error") and
                    label.cget("text") != "Error: All values are empty.
No data inserted."):
                    error = True
                    label.grid(row=row, column=0)
                    if self.meeting_system_values[3] >
self.meeting_system_values[4]:
                        error = True
                        label = ctk.CTkLabel(self.input_update_frame,
text="Error: The time input doesn't make sense.",
                             text_color="red", padx=10)
                        label.grid(row=row, column=0)
                        row += 1
                    else:
                        label = ctk.CTkLabel(self.input_update_frame,
text=self.client.insert_into_table(self.name_entry.get(),
'Apps', {
                            'app_name': self.name_entry.get(),
                            'design': self.settings_frame.color,
                            'meeting_length': "X",
                            'break_length': "X",
                            'meeting_in_between': "X",
                            'starting_time': "X",
                            'ending_time': "X"}), text_color="red", padx=10)

```

```

        row += 1
        if (label.cget("text").startswith("Error") and
            label.cget("text") != "Error: All values are empty.
No data inserted."):
            error = True
            label.grid(row=row, column=0)
        for i in range(len(self.homepage_values)):
            try:
                content = self.homepage_values[i][2].get("1.0", "end-1c")
            except AttributeError:
                content = "X"

            label = ctk.CTkLabel(self.input_update_frame,
text=self.client.insert_into_table(self.name_entry.get(),
"Homepage", {
                'block_type': self.homepage_values[i][0],
                'headline': self.homepage_values[i][1].get(),
                'content': content,
                'picture1': self.homepage_values[i][3],
                'picture2': self.homepage_values[i][4],
                'picture3': self.homepage_values[i][5]}),
text_color="red", padx=10)
            row += 1
            if (label.cget("text").startswith("Error") and
                label.cget("text") != "Error: All values are empty.
No data inserted."):
                error = True
                label.grid(row=row, column=0)
            for i in range(len(self.catalog_values)):
                label = ctk.CTkLabel(self.input_update_frame,
text=self.client.insert_into_table(self.name_entry.get(),
'Catalog', {
                    'product_name': self.catalog_values[i][0].get(),
                    'price': self.catalog_values[i][1].get(),
                    'description': self.catalog_values[i][2].get(),
                    'picture': self.catalog_values[i][3]}), text_color="red",
padx=10)
                row += 1
                if (label.cget("text").startswith("Error") and
                    label.cget("text") != "Error: All values are empty.
No data inserted."):
                    error = True
                    label.grid(row=row, column=0)
                if not error:
                    self.settings_frame.background_frame.pack_forget()
                    t = threading.Thread(target=Tools.Tools,
args=(self.name_entry.get(),
self.settings_frame.root, self.client))
                    t.start()
                else:
                    self.client.clear_app_database(self.name_entry.get())

    def create_tables(self):
        self.grid_forget()

```

```

self.grid(row=1, column=1, pady=10)
self.columnconfigure((0, 3), weight=1)

if self.settings_frame.check_boxs_var[0].get() == "on":
    self.homepage_table()
if self.settings_frame.check_boxs_var[1].get() == "on":
    self.catalog_table()
if self.settings_frame.check_boxs_var[2].get() == "on":
    self.meeting_system_table()
if (self.settings_frame.check_boxs_var[0].get() == "off" and
self.settings_frame.check_boxs_var[1].get() ==
    "off" and self.settings_frame.check_boxs_var[2].get() ==
"off"):
    label = ctk.CTkLabel(self, text="Error! No part selected.")
    label.grid(row=self.next_row, column=1, columnspan=2, padx=20)
else:
    name = ctk.CTkLabel(self, text="App's name:", font=("Ariel
Black", 25))
    name.grid(row=self.next_row, column=1, padx=20, pady=30,
sticky="E")
    self.name_entry.grid(row=self.next_row, column=2, padx=20,
pady=30, sticky="W")
    self.next_row += 1
    label = ctk.CTkLabel(self, text="*If you accidentally create more
rows than required,"
                                " leave them blank and they will
not be entered into the database.",
                                font=("Ariel", 10))
    label.grid(row=self.next_row, column=1, columnspan=2, padx=20)
    self.next_row += 1
    create_button = ctk.CTkButton(self, text="Create App",
command=self.insert_arr_to_database)
    create_button.grid(row=self.next_row + 1, column=1, columnspan=2,
padx=20, pady=30)

```

:Tools

```

import customtkinter as ctk
import Homepage
import Catalog
import Meeting System
import CreatorFrame
import json
import threading
import Users
from PIL import Image

class Tools(ctk.CTkFrame):
    def __init__(self, name, root, my_client):
        self.name = name
        self.root = root
        self.client = my_client

        self.design = self.client.get_design(self.name)

        self.j = json.load(open(f"designs/{self.design}_design.json"))

```

```

        ctk.set_default_color_theme(f"designs/{self.design}_design.json")
        super().__init__(self.root, fg_color=(self.j["CTk"]["fg_color"]))
        self.background_frame = ctk.CTkScrollableFrame(self.root,
fg_color=(self.j["CTk"]["fg_color"]))
        self.user = ""
        self.column = 0
        self.create_tools()

    def create_homepage(self):
        self.background_frame.pack_forget()
        self.background_frame = ctk.CTkScrollableFrame(self.root,
fg_color=(self.j["CTk"]["fg_color"]))
        self.background_frame.pack(fill="both", expand=True)
        gif_label = GIFLabel(self.background_frame,
"random_pictures/loading_gif.gif")
        gif_label.pack(expand=True) # לחלון GIF-ה הוספת
        gif_label.animate()
        t = threading.Thread(target=Homepage.HomePageDesign,
args=(self.background_frame, self.name, self.client, 1,
gif_label))
        t.start()

    def create_catalog(self):
        self.background_frame.pack_forget()
        self.background_frame = ctk.CTkScrollableFrame(self.root,
fg_color=(self.j["CTk"]["fg_color"]))
        self.background_frame.pack(fill="both", expand=True)
        gif_label = GIFLabel(self.background_frame,
"random_pictures/loading_gif.gif")
        gif_label.pack(expand=True) # לחלון GIF-ה הוספת
        gif_label.animate()
        t = threading.Thread(target=Catalog.CatalogDesign,
args=(self.background_frame, self.name, self.client, 1,
gif_label))
        t.start()

    def create_meeting_system(self):
        self.background_frame.pack_forget()
        self.background_frame = ctk.CTkScrollableFrame(self.root,
fg_color=(self.j["CTk"]["fg_color"]))
        self.background_frame.pack(fill="both", expand=True)

        meeting_system_values =
self.client.get_meeting_system_vars(self.name)
        if self.client.client_socket.fileno() != -1: # עדיין הסוקט האם
פועל
            t = threading.Thread(target=Meeting_System.MeetingSystemDesign,
args=(self.background_frame, self.user,
meeting_system_values,
self.design, self.root,
self.after_log_in, self.name,
self.client,))

```



```

        t.start()

    def create_tools(self):
        self.pack(fill="both")
        self.columnconfigure((1, 3), weight=1)

        mode_switch = CreatorFrame.ColorMode(self)
        mode_switch.mode_switch.grid(row=0, column=0)

        name = ctk.CTkLabel(self, text=self.name, font=("Gill Sans Nova Ultra
Bold", 25))
        name.grid(row=0, column=2)

        start = False

        # בטבלה שורות לספירת שאילתה
        count = int(self.client.count_rows(self.name, "Homepage")) # מחזיר
את השורות מספר את

        if count > 0:
            button_homepage = ctk.CTkButton(self, text="Back to the
homepage", command=self.create_homepage)
            button_homepage.grid(row=0, column=4+self.column, padx=5, pady=5)
            self.column += 1
            self.create_homepage()
            start = True

        # בטבלה שורות לספירת שאילתה
        count = int(self.client.count_rows(self.name, "Catalog")) # מחזיר
את השורות מספר
        # מחזיר את מספר את

        if count > 0:
            button_catalog = ctk.CTkButton(self, text="View Catalog",
command=self.create_catalog)
            button_catalog.grid(row=0, column=4+self.column, padx=5, pady=5)
            self.column += 1
            if not start:
                self.create_catalog()
                start = True

        # הערך לבדיקת שאילתה כתיבת
        result = self.client.get_meeting_system_vars(self.name)[0]

        if result != "X":
            button_meeting_system = ctk.CTkButton(self, text="Meeting
system", command=self.create_meeting_system)
            button_meeting_system.grid(row=0, column=4+self.column, padx=5,
pady=5)
            self.column += 1
            if not start:
                self.create_meeting_system()

        if self.user == "":
            button_log_in = ctk.CTkButton(self, text="Log In",
command=lambda: self.log_in())

```

```

        button_log_in.grid(row=0, column=4+self.column, padx=5, pady=5,
sticky="E")
    else:
        button_log_out = ctk.CTkButton(self, text="Log Out",
command=lambda: self.log_out())
        button_log_out.grid(row=0, column=4 + self.column, padx=5,
pady=5, sticky="E")

    def log_out(self):
        self.user = ""
        frame_name_packed =
type(self.background_frame.wininfo_children()[0]).__name__
        if frame_name_packed == "MeetingSystemDesign":
            self.create_meeting_system()

        button_log_in = ctk.CTkButton(self, text="Log In", command=lambda:
self.log_in())
        button_log_in.grid(row=0, column=4 + self.column, padx=5, pady=5,
sticky="E")

    def log_in(self):
        Users.CreateRoot(self.design, self.root, self.after_log_in, 0, 0, "",
False, self.name, self.client)

    def after_log_in(self, user_name, meeting_row, meeting_column,
meeting_time, meeting_act):
        self.user = user_name

        button_log_out = ctk.CTkButton(self, text="Log Out", command=lambda:
self.log_out())
        button_log_out.grid(row=0, column=4 + self.column, padx=5, pady=5,
sticky="E")

        frame_name_packed =
type(self.background_frame.wininfo_children()[0]).__name__
        if frame_name_packed == "MeetingSystemDesign":
            self.create_meeting_system()

        if meeting_time != "":
            meeting_act(meeting_row, meeting_column, meeting_time, self.user)

class GIFLabel(ctk.CTkLabel):
    def __init__(self, master, image_path, **kwargs):
        self._gif_image = Image.open(image_path)
        # set the size of the label to the same as the GIF image
        kwargs.setdefault("width", self._gif_image.width)
        kwargs.setdefault("height", self._gif_image.height)
        # don't show the text initially
        kwargs.setdefault("text", "")
        # delay for the after loop
        self._duration = kwargs.pop("duration", None) or
self._gif_image.info["duration"]
        super().__init__(master, **kwargs)
        # load all the frames
        self._frames = []
        for i in range(self._gif_image.n_frames):

```

```

        self._gif_image.seek(i)
        self._frames.append(ctk.CTkImage(self._gif_image.copy(),
size=(self["width"], self["height"])))

    def animate(self, idx=0):
        self.configure(image=self._frames[idx])
        self.after(self._duration, self.animate, (idx + 1) %
len(self._frames))

```

:Homepage

```

import customtkinter as ctk
from PIL import Image, ImageOps
from io import BytesIO
import textwrap
from tkinter.font import Font
import time

def change_picture_to_size(picture, size):
    return picture.size[0] - (min(picture.size) - size), picture.size[1] -
(min(picture.size) - size)

def wrap_label_text_in_frame(frame, label, column_index):
    frame.update_idletasks()
    max_width_pixels = (frame.winfo_width()-frame.grid_bbox(column_index, 0,
column_index, frame.grid_size()[1] - 1)[2])

    font = Font(font=label.cget("font"))
    average_char_width = font.measure("0")
    max_width_chars = max_width_pixels // average_char_width

    text = label.cget("text")
    try:
        wrapped_text = "\n".join([textwrap.fill(line, width=max_width_chars)
for line in text.split('\n')])
    except ValueError: # עקרונית מקום ואין מסך פיצול עקב הוקטן המחשב מסך אם
לחונן
        wrapped_text = text
    label.configure(text=wrapped_text)
    return label

class HomePageDesign:
    def __init__(self, background_frame, name, my_client, design_num, gif):
        self.name = name
        self.background_frame = background_frame
        self.client = my_client
        self.gif = gif
        self.rows = self.client.get_homepage_content(self.name)
        self.design_num = design_num
        time.sleep(0.1)
        self.create_frames()

    def create_left_picture_and_content(self, i):
        frame = ctk.CTkFrame(self.background_frame)

```

```

frame.pack(fill="both", expand=True)
frame.columnconfigure((1, 3), weight=1)

image_data = self.rows[i][4]
image = Image.open(BytesIO(image_data))
size = change_picture_to_size(image, 300)

sparkles_image = Image.open("homepage_design_pictures/sparkles2.png")
sparkles = ctk.CTkLabel(frame, text="", image=ctk.CTkImage(
    light_image=sparkles_image, size=(size[0]+100, size[1]+190)))
sparkles.grid(row=0, rowspan=3, column=0, sticky="NW")

gold_frame_image =
Image.open("homepage_design_pictures/gold_frame.png")
gold_frame = ctk.CTkLabel(frame, text="",
image=ctk.CTkImage(light_image=gold_frame_image, size=size))
gold_frame.grid(row=0, rowspan=3, column=0, padx=170, pady=80,
sticky="NW")

picture = ctk.CTkLabel(frame, text="",
image=ctk.CTkImage(light_image=gold_frame_image, size=size))
picture.grid(row=0, rowspan=3, column=0, padx=200, pady=100,
sticky="NE")

headline = ctk.CTkLabel(frame, text=self.rows[i][2],
font=("Broadway", 80))
headline = wrap_label_text_in_frame(frame, headline, 0)
headline.grid(row=0, column=2, sticky="S", pady=20)
content = ctk.CTkLabel(frame, text=self.rows[i][3], font=("Arial",
40))
content = wrap_label_text_in_frame(frame, content, 0)
content.grid(row=1, column=2, sticky="N", pady=20)

def create_right_picture_and_content(self, i):
    frame = ctk.CTkFrame(self.background_frame)
    frame.pack(fill="both", expand=True)
    frame.columnconfigure((0, 2), weight=1)

    image_data = self.rows[i][4]
    image = Image.open(BytesIO(image_data))
    size = change_picture_to_size(image, 300)

    sparkles_image =
ImageOps.mirror(Image.open("homepage_design_pictures/sparkles2.png"))
    sparkles = ctk.CTkLabel(frame, text="", image=ctk.CTkImage(
        light_image=sparkles_image, size=(size[0] + 100, size[1] + 190)))
    sparkles.grid(row=0, rowspan=3, column=3, sticky="NE")

    gold_frame_image =
Image.open("homepage_design_pictures/gold_frames.png")
    gold_frame = ctk.CTkLabel(frame, text="",
image=ctk.CTkImage(light_image=gold_frame_image,
size=(size[0] + 40, size[1] + 40)))
    gold_frame.grid(row=0, rowspan=3, column=3, padx=150, pady=80,
sticky="nsew")

```

```

        picture = ctk.CTkLabel(frame, text="",
image=ctk.CTkImage(light_image=image, size=size))
        picture.grid(row=0, rowspan=3, column=3, padx=170, pady=100,
sticky="nsew")

        headline = ctk.CTkLabel(frame, text=self.rows[i][2],
font=("Broadway", 80))
        headline = wrap_label_text_in_frame(frame, headline, 3)
        headline.grid(row=0, column=1, sticky="S", pady=20)
        content = ctk.CTkLabel(frame, text=self.rows[i][3], font=("Arial",
40))
        content = wrap_label_text_in_frame(frame, content, 3)
        content.grid(row=1, column=1, sticky="N", pady=20)

    def create_only_content(self, i):
        frame = ctk.CTkFrame(self.background_frame)
        frame.pack(side="top", fill="both", pady=10)
        frame.columnconfigure((0, 2), weight=1)

        bg_image = Image.open("homepage_design_pictures/gold_background.png")
        background_label = ctk.CTkLabel(frame, text="", image=ctk.CTkImage(
            light_image=bg_image, size=(2000, 800)))
        background_label.grid(row=0, rowspan=3, column=0, columnspan=3,
sticky="nsew")

        text_frame = ctk.CTkFrame(frame)
        text_frame.grid(row=1, column=1)

        headline = ctk.CTkLabel(text_frame, text=self.rows[i][2],
font=("Broadway", 80))
        headline = wrap_label_text_in_frame(frame, headline, 4)
        headline.grid(row=0, column=1, pady=10, sticky="S")
        content = ctk.CTkLabel(text_frame, text=self.rows[i][3],
font=("Arial", 40))
        content = wrap_label_text_in_frame(frame, content, 4)
        content.grid(row=1, column=1, pady=10, sticky="N")

    def create_pictures_only(self, i):
        frame = ctk.CTkFrame(self.background_frame)
        frame.pack(side="top", fill="both", pady=10)
        frame.columnconfigure((0, 4), weight=1)

        sparkles_image = Image.open("homepage_design_pictures/sparkles1.png")
        sparkles = ctk.CTkLabel(frame, text=self.rows[i][2],
font=("Broadway", 80), image=ctk.CTkImage(
            light_image=sparkles_image, size=(1500, 400)))
        sparkles = wrap_label_text_in_frame(frame, sparkles, 4)
        sparkles.grid(row=0, column=0, columnspan=5, sticky="NW")

        for j in range(3):
            image_data = self.rows[i][4+j]
            image = Image.open(BytesIO(image_data))
            size = change_picture_to_size(image, 200)

            gold_frame_image =
Image.open("homepage_design_pictures/gold_frame2.png")
            gold_frame = ctk.CTkLabel(frame, text="",

```

```

image=ctk.CTkImage(light_image=gold_frame_image,
size=(size[0]+40, size[1]+80)))
gold_frame.grid(row=1, column=j+1, padx=30)

picture = ctk.CTkLabel(frame, text="",
image=ctk.CTkImage(light_image=image, size=size))
picture.grid(row=1, column=j+1, padx=30, pady=30)

def create_frames(self):
    for i in range(len(self.rows)):
        method = getattr(self, f"create_{self.rows[i][1]}", None)
        if i == 0:
            self.gif.pack_forget()
        if method is not None:
            method(i)

```

:Catalog

```

import customtkinter as ctk
from PIL import Image
from io import BytesIO
import textwrap
from tkinter.font import Font
import time

def change_picture_to_size(picture):
    ratio = 600 / float(picture.size[0])
    height = int((float(picture.size[1]) * float(ratio)))
    return 600, height

def wrap_label_text_in_frame(frame, label, column_index):
    frame.update_idletasks()
    max_width_pixels = (frame.winfo_width()-frame.grid_bbox(column_index, 0,
column_index, frame.grid_size()[1] - 1)[2]
- 80)

    font = Font(font=label.cget("font"))
    average_char_width = font.measure("0")
    max_width_chars = max_width_pixels // average_char_width

    text = label.cget("text")
    wrapped_text = "\n".join([textwrap.fill(line, width=max_width_chars) for
line in text.split('\n')])
    label.configure(text=wrapped_text)
    return label

class CatalogDesign:
    def __init__(self, background_frame, name, my_client, design_num, gif):
        self.name = name
        self.background_frame = background_frame
        self.client = my_client
        self.gif = gif
        self.rows = self.client.get_catalog_content(self.name)

```

```

        self.design_num = design_num
        time.sleep(0.1)
        self.create_frames()

    def create_product_frame(self, i):
        frame = ctk.CTkFrame(self.background_frame)
        frame.pack(fill="both", expand=True, pady=10)
        frame.columnconfigure((0, 3), weight=1)

        image_data = self.rows[i][4]
        image = Image.open(BytesIO(image_data))
        size = change_picture_to_size(image)

        gold_frame_image =
Image.open("homepage design pictures/gold_frame.png")
        gold_frame = ctk.CTkLabel(frame, text="",
image=ctk.CTkImage(light_image=gold_frame_image, size=size))
        gold_frame.grid(row=0, column=1, columnspan=2,
padx=frame.winfo_width()/2-size[0]/2, pady=20, sticky="SW")

        picture = ctk.CTkLabel(frame, text="",
image=ctk.CTkImage(light_image=image, size=size))
        picture.grid(row=0, column=1, columnspan=2,
padx=frame.winfo_width()/2-size[0]/2-30, pady=40, sticky="SE")

        product_name = ctk.CTkLabel(frame, text=self.rows[i][1],
font=("Broadway", 80))
        product_name = wrap_label_text_in_frame(frame, product_name, 0)
        product_name.grid(row=1, column=1, pady=5, sticky="E")

        price = ctk.CTkLabel(frame, text=str(self.rows[i][2])+"$",
font=("Arial", 40))
        price.grid(row=1, column=2, sticky="W")

        description = ctk.CTkLabel(frame, text=self.rows[i][3],
font=("Arial", 40))
        description = wrap_label_text_in_frame(frame, description, 4)
        description.grid(row=2, column=1, columnspan=2, pady=10, sticky="N")

    def create_frames(self):
        for i in range(len(self.rows)):
            self.create_product_frame(i)
            if i == 0:
                self.gif.pack_forget()

```

:Meeting System

```

import customtkinter as ctk
import datetime
from functools import partial
import Users
from PIL import Image

class MeetingSystemDesign(ctk.CTkFrame):
    def __init__(self, background_frame, user, input_arr, design, root,
after_log_in, name, my_client):

```

```

        super().__init__(background_frame)
        self.name = name
        self.background_frame = background_frame
        self.user = user
        self.design = design
        self.root = root
        self.after_log_in = after_log_in
        self.client = my_client
        self.input_arr = input_arr # length of a meeting (minutes), length
of a break (minutes),
        # how many meetings between every break, starting time, ending time
        self.meetings_arr = []
        self.frame = ctk.CTkFrame(self.background_frame)
        self.pack(fill="both", expand=True, pady=10)
        self.columnconfigure((0, 8), weight=1)
        self.create_frame()

    def book_a_meeting(self, row, column, meeting_time, new_user):
        self.user = new_user
        if self.user == "":
            self.user = Users.CreateRoot(self.design, self.root,
self.after_log_in, row, column, meeting_time,
self.client)
            self.book_a_meeting, self.name,
        else:
            response = self.client.book_meeting(self.name, meeting_time,
self.user)
            if response == "True":
                if isinstance(self.meetings_arr[column][row], ctk.CTkButton):
# checking that the placement is correct
                    text = self.meetings_arr[column][row].cget("text")
                    self.meetings_arr[column][row] = ctk.CTkLabel(self,
text=text + " (taken)")

                    for widget in self.winfo_children():
                        widget.destroy()

                    self.create_frame()

    def create_frame(self):
        headline = ctk.CTkLabel(self, text="Book A Meeting With Us",
font=("Broadway", 80))
        headline.grid(row=0, column=1, columnspan=7)

        for i in range(7):
            day = datetime.datetime.today() + datetime.timedelta(days=i)
            day_name = day.strftime('%A')
            label = ctk.CTkLabel(self, text=f"{day_name}\n {day.strftime('%Y-
%m-%d')}", font=("Black Ariel", 18))
            label.grid(row=1, column=1 + i)
            arr = []
            start_time = self.input_arr[3]
            end_time = start_time + int(self.input_arr[0])
            row = 2

            while start_time < int(self.input_arr[4]):
                for j in range(int(self.input_arr[2])):

```



```

        if end_time > int(self.input_arr[4]):
            pass
        else:
            meeting_time = (f"{day_name}\n {day.strftime('%Y-%m-%d')}")
                                f" {int(start_time /
60):02}:{start_time % 60:02}")
            meeting_available =
self.client.check_meeting_availability(self.name, meeting_time)

            if meeting_available == "False":
                arr.append(ctk.CTkLabel(self,
text=f"{int(start_time / 60):02}:{start_time % 60:02}-"
f"{int(end_time / 60):02}:{end_time % 60:02} (taken)"))
                arr[-1].grid(row=row, column=i + 1, padx=20,
pady=5)
            else:
                arr.append(ctk.CTkButton(self,
text=f"{int(start_time / 60):02}:{start_time % 60:02}-"
f"{int(end_time / 60):02}:{end_time % 60:02}",
                                command=partial(self.book_a_meeting,
len(arr), i, meeting_time, self.user)))
                arr[-1].grid(row=row, column=i + 1, padx=20,
pady=5)

            row += 1
            start_time = end_time
            end_time = start_time + int(self.input_arr[0])

        if start_time < int(self.input_arr[4]):
            label = ctk.CTkLabel(self, text="-----")
            label.grid(row=row, column=i + 1)
            row += 1
            start_time += int(self.input_arr[1])
            end_time = start_time + int(self.input_arr[0])

        self.meetings_arr.append(arr)
        if self.user != "":
            self.frame.pack_forget()
            self.my_meetings()

    def delete_meeting(self, user, time):
        response = self.client.delete_meeting(self.name, user, time)
        if response == "True":
            for widget in self.wininfo_children():
                widget.destroy()
            self.create_frame()

    def my_meetings(self):
        self.frame = ctk.CTkFrame(self.background_frame)
        self.frame.pack(pady=20)
        self.frame.columnconfigure((0, 5), weight=1)

        headline = ctk.CTkLabel(self.frame, text="My Meetings",
font=("Broadway", 60))

```

```

headline.grid(row=0, column=1, columnspan=4, padx=10, pady=10)

time_label_headline = ctk.CTkLabel(self.frame, text="Date & Time",
font=("Black Ariel", 20))
time_label_headline.grid(row=1, column=2, padx=20, pady=20)

meetings = self.client.get_users_meetings(self.name, self.user)
meetings = [m for m in meetings if
datetime.datetime.strptime(m[0].split("\n")[1].strip(), "%Y-%m-%d %H:%M")
    >= datetime.datetime.now()] # שעברו תאריכים של הורדה
meetings = sorted(meetings, key=lambda x: datetime.datetime.
    strftime(x[0].split("\n")[1].strip(), "%Y-%m-%d
%H:%M")) # ושמע תאריך לפי חיון

index = 2
for meeting in meetings:
    time_label_headline = ctk.CTkLabel(self.frame, text=meeting)
    time_label_headline.grid(row=index, column=2, padx=10, pady=10)
    pic = Image.open("random_pictures/trash.png.png")
    trash_button = ctk.CTkButton(self.frame, text="",
image=ctk.CTkImage(light_image=pic, size=(30, 30)),
                                command=partial(self.delete_meeting,
self.user, meeting), width=30,
                                height=30)
    trash_button.grid(row=index, column=3, padx=10, pady=10)
    index += 1

```

:User

```

import customtkinter as ctk
import json
import re
import hashlib

class CreateRoot:
    def __init__(self, design, last_root, after_log_in, meeting_row,
meeting_column, meeting_time, meeting_act, name,
my_client):
        self.name = name
        self.design = design
        self.root = ctk.CTkToplevel(last_root)
        self.root.after(100, self.root.lift)
        self.after_log_in = after_log_in
        self.meeting_row = meeting_row
        self.meeting_column = meeting_column
        self.meeting_time = meeting_time
        self.meeting_act = meeting_act
        self.result = None
        self.client = my_client
        self.root.title("Log In/Sign Up")
        ctk.set_default_color_theme(f"designs/{self.design}_design.json")

        window_width = 450
        window_height = 350
        screen_width = self.root.winfo_screenwidth()
        screen_height = self.root.winfo_screenheight()

```

```

        position_x = int((screen_width / 2) - (window_width / 2)) # window
open in the middle of the screen
        position_y = int((screen_height / 2) - (window_height / 2))

self.root.geometry(f"{window_width}x{window_height}+{position_x}+{position_y}
")

        self.initialize_frame()

        self.run()

    def initialize_frame(self):
        login_frame = LogIn(self.root, self.design, self.name, self.client)
        login_frame.pack(expand=True, fill="both")
        login_frame.callback = self.set_result

    def set_result(self, result):
        self.result = result
        try:
            self.root.destroy()
            self.after_log_in(self.result, self.meeting_row,
self.meeting_column, self.meeting_time, self.meeting_act)
        except Exception as e:
            print(f"Error while closing root: {e}")

    def run(self):
        self.root.mainloop()

class Authentication(ctk.CTkFrame):
    def __init__(self, root, design, name, my_client):
        super().__init__(root)
        self.name = name
        self.root = root
        self.design = design
        self.client = my_client
        self.callback = None
        self.error_label = ctk.CTkLabel(self)
        self.password_entry = ctk.CTkEntry(self)
        self.gmail_entry = ctk.CTkEntry(self)

    def go_to_sign_up(self, frame):
        frame.destroy()
        sign_up_frame = SingUp(self.root, self.design, self.name,
self.client)
        sign_up_frame.callback = self.callback
        sign_up_frame.pack(expand=True, fill="both")

    def go_to_log_in(self, last_frame):
        last_frame.destroy()
        login_frame = LogIn(self.root, self.design, self.name, self.client)
        login_frame.callback = self.callback
        login_frame.pack(expand=True, fill="both")

class LogIn(Authentication):

```

```

def __init__(self, root, design, name, my_client):
    super().__init__(root, design, name, my_client)
    self.password_shown = False
    self.result = None
    self.create_frame()

def create_frame(self):
    self.columnconfigure((0, 4), weight=1)
    headline = ctk.CTkLabel(self, text="Log In", font=("Broadway", 40))
    headline.grid(row=0, column=1, columnspan=2, pady=10)

    gmail = ctk.CTkLabel(self, text="Gmail:")
    gmail.grid(row=1, column=1, padx=10, pady=5)
    self.gmail_entry = ctk.CTkEntry(self)
    self.gmail_entry.grid(row=1, column=2, padx=10, pady=5)

    password = ctk.CTkLabel(self, text="Password:")
    password.grid(row=2, column=1, padx=10, pady=5)
    self.password_entry = ctk.CTkEntry(self, show="*")
    self.password_entry.grid(row=2, column=2, padx=10, pady=5)

    toggle_password_button = ctk.CTkButton(self, width=20, text="*",
font=("Ariel Black", 20),
command=self.toggle_password_visibility)
    toggle_password_button.grid(row=2, column=3, pady=10)

    login_button = ctk.CTkButton(self, text="Log In",
command=self.login_action)
    login_button.grid(row=5, column=1, columnspan=2, pady=10)
    j = json.load(open(f"designs/{self.design}_design.json"))
    sing_up_button = ctk.CTkButton(self, text="Sing Up",
text_color=(j["CTkButton"]["fg_color"]),
fg_color=(j["CTk"]["fg_color"]),
command=lambda: self.go_to_sign_up(self))
    sing_up_button.grid(row=6, column=1, columnspan=2, pady=10)
    forgot_password_button = ctk.CTkButton(self, text="Forgot Password",
fg_color=(j["CTkFrame"]["fg_color"]),
text_color=(j["CTkButton"]["fg_color"]), command=self.forget_password)
    forgot_password_button.grid(row=7, column=1, columnspan=2, pady=10)

    def toggle_password_visibility(self):
        if self.password_shown:
            self.password_entry.configure(show="*")
            self.password_shown = False
        else:
            self.password_entry.configure(show="")
            self.password_shown = True

    def forget_password(self):
        self.destroy()
        forgot_password_frame = ctk.CTkFrame(self.root)
        forgot_password_frame.pack(expand=True, fill="both")
        forgot_password_frame.columnconfigure((0, 3), weight=1)
        label = ctk.CTkLabel(forgot_password_frame, text="Forgot Password",
font=("Black Ariel", 24))

```

```

label.grid(row=0, column=1, columnspan=2, pady=10)

gmail = ctk.CTkLabel(forgot_password_frame, text="Gmail:")
gmail.grid(row=1, column=1, padx=10, pady=5)
gmail_entry = ctk.CTkEntry(forgot_password_frame)
gmail_entry.grid(row=1, column=2, padx=10, pady=5)

send_button = ctk.CTkButton(forgot_password_frame, text="Send",
                             command=lambda:
self.send_new_password(forgot_password_frame, gmail_entry.get()))
send_button.grid(row=2, column=1, columnspan=2, pady=10)
j = json.load(open(f"designs/{self.design}_design.json"))
back_to_log_in = ctk.CTkButton(forgot_password_frame, text="Back to
Log In", fg_color=(j["CTk"]["fg_color"]),
text_color=(j["CTkButton"]["fg_color"]), command=lambda:
self.go_to_log_in(forgot_password_frame))
back_to_log_in.grid(row=3, column=1, columnspan=2, pady=10)

def login_action(self):
self.error_label.destroy()
email_pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
if self.gmail_entry.get() == '' or self.gmail_entry.get() == 'Null':
self.error_label = ctk.CTkLabel(self, text="Please Enter Your
Username", text_color="red")
self.error_label.grid(row=4, column=1, columnspan=2)
elif not re.match(email_pattern, self.gmail_entry.get()):
self.error_label = ctk.CTkLabel(self, text="Invalid email format.
Example: example@gmail.com.",
text_color="red")
self.error_label.grid(row=4, column=1, columnspan=2)
elif self.password_entry.get() == '' or self.password_entry.get() ==
'Null':
self.error_label = ctk.CTkLabel(self, text="Please Enter Your
Password", text_color="red")
self.error_label.grid(row=4, column=1, columnspan=2)
else:
result = self.client.login_to_system(self.name,
self.gmail_entry.get(),
hashlib.sha256(self.password_entry.get().encode()).hexdigest())
if result == "False":
self.error_label = ctk.CTkLabel(self, text="Your gmail or
password are incorrect", text_color="red")
self.error_label.grid(row=4, column=1, columnspan=2)
else:
self.result = result[0]
if self.callback:
self.callback(self.result)
else:
print("Callback is not set.")

def send_new_password(self, frame, gmail):
email_pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
if not re.match(email_pattern, gmail):
error_label = ctk.CTkLabel(frame, text="Invalid email format.

```

```

Example: example@gmail.com.",
                                text_color="red")
    error_label.grid(row=4, column=1, columnspan=2, pady=10)
else:
    gmail_exists = self.client.check_gmail_in_app(self.name, gmail)
    if gmail_exists == "False":
        error_label = ctk.CTkLabel(frame, text="You are not in the
system.\n Please go to sign up",
                                text_color="red")
        error_label.grid(row=5, column=1, columnspan=2, pady=10)
    else:
        self.client.send_new_password(self.name, gmail)

class SingUp(Authentication):
    def __init__(self, root, design, name, my_client):
        super().__init__(root, design, name, my_client)
        self.name_entry = ctk.CTkEntry(self)
        self.phone_num_entry = ctk.CTkEntry(self)
        self.create_frame()

    def create_frame(self):
        self.columnconfigure((0, 4), weight=1)
        headline = ctk.CTkLabel(self, text="Sing Up", font=("Broadway", 40))
        headline.grid(row=0, column=1, columnspan=2, pady=10)

        name = ctk.CTkLabel(self, text="Name:")
        name.grid(row=1, column=1, padx=10, pady=5)
        self.name_entry = ctk.CTkEntry(self)
        self.name_entry.grid(row=1, column=2, padx=10, pady=5)

        gmail = ctk.CTkLabel(self, text="Gmail:")
        gmail.grid(row=2, column=1, padx=10, pady=5)
        self.gmail_entry = ctk.CTkEntry(self)
        self.gmail_entry.grid(row=2, column=2, padx=10, pady=5)

        phone_num = ctk.CTkLabel(self, text="Phone Number:")
        phone_num.grid(row=3, column=1, padx=10, pady=5)
        self.phone_num_entry = ctk.CTkEntry(self)
        self.phone_num_entry.grid(row=3, column=2, padx=10, pady=5)

        password = ctk.CTkLabel(self, text="Password:")
        password.grid(row=4, column=1, padx=10, pady=5)
        self.password_entry.grid(row=4, column=2, padx=10, pady=5)

        sing_up_button = ctk.CTkButton(self, text="Sing Up",
command=self.sign_up_action)
        sing_up_button.grid(row=6, column=1, columnspan=2, pady=10)

    def sign_up_action(self):
        self.error_label.destroy()
        errors = []

        phone_pattern = r"^\+?[0-9]\d{9,14}$"
        if not re.match(phone_pattern, self.phone_num_entry.get()):
            errors.append("Invalid phone number format.\n"
                " Use international format with 10-15 digits, e.g.,

```

```

+1234567890.")

    email_pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
    if not re.match(email_pattern, self.gmail_entry.get()):
        errors.append("Invalid email format. Example:
example@gmail.com.")

    password_pattern = r"^(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&:*/#$](<>)) [A-
Za-z\d@$!%*?&:*/#$](<>){8,}$"
    if not re.match(password_pattern, self.password_entry.get()):
        errors.append("Password must be at least 8 characters long,\n"
            " contain one uppercase letter, one digit, and one
special character.")

    if errors:
        text = "\n".join([f"{s}" for i, s in enumerate(errors)])

        self.error_label = ctk.CTkLabel(self, text=text,
text_color="red", padx=20)
        self.error_label.grid(row=5, column=1, columnspan=2)

    else:
        user_var_dict = {
            'name': self.name_entry.get(), 'gmail':
self.gmail_entry.get(),
            'phone_number': self.phone_num_entry.get(),
            'password':
hashlib.sha256(self.password_entry.get().encode()).hexdigest()

        # database
        # תקין אכן שהוא לראות מנת על הבדיקות את עובר רק התוכן אלא
        self.error_label = ctk.CTkLabel(self,
text=self.client.insert_into_table(
            self.name, 'Users', user_var_dict), text_color="red",
padx=10)

        if self.error_label.cget("text").startswith("Error"):
            self.error_label.grid(row=5, column=1, columnspan=2)
        else:
            self.destroy()

        confirmation_frame = ctk.CTkFrame(self.root)
        confirmation_frame.pack(expand=True, fill="both")
        confirmation_frame.columnconfigure((0, 2), weight=1)
        label = ctk.CTkLabel(confirmation_frame, text="Please enter
your confirmation code",
                                font=("Black Ariel", 20))
        label.grid(row=0, column=1, pady=20)
        entry = ctk.CTkEntry(confirmation_frame)
        entry.grid(row=1, column=1, pady=20)
        send_button = ctk.CTkButton(confirmation_frame, text="Send",
command=lambda:
self.check_confirmation(entry.get(), confirmation_frame,
user_var_dict))
        send_button.grid(row=2, column=1, pady=10)
        send_again_button = ctk.CTkButton(confirmation_frame,

```

```

text="Send Again", command=lambda:
self.client.send_confirmation_again(user_var_dict["gmail"]))
    send_again_button.grid(row=3, column=1, pady=10)
    j = json.load(open(f"designs/{self.design}_design.json"))
    back_to_sign_up = ctk.CTkButton(confirmation_frame,
text="Back to Sing Up",
fg_color=(j["CTk"]["fg_color"]),
text_color=(j["CTkButton"]["fg_color"]), command=lambda:
self.go_to_sign_up(confirmation_frame))
    back_to_sign_up.grid(row=4, column=1, pady=10)

    def check_confirmation(self, entry, frame, user_vars):
        correct = self.client.check_confirmation(entry)
        if correct == "True":
            self.client.insert_user(self.name, user_vars)
            self.go_to_log_in(frame)
        else:
            error_label = ctk.CTkLabel(frame, text="Code is incorrect",
text_color="red")
            error_label.grid(row=5, column=1, pady=10)

```